**HTBL Hollabrunn**

**Höhere Lehranstalt für Elektronik**
**und Technische Informatik**

# DIPLOMARBEIT

Gesamtprojekt

## Digitale Audio Effekt Box

**Hardware Digital Audio Effekt Box**
Lukas Gritsch 5AHEL Betreuer: Dipl.-Ing. Josef Reisinger

**Software Digital Audio Effekt Box**
Michael Donner 5AHEL Betreuer: Dipl.-Ing. Josef Reisinger

Schuljahr 2024/25

Abgabevermerk:

Datum: 07.04.2025 übernommen von:

# Höhere Technische Bundeslehranstalt Hollabrunn
## Höhere Lehranstalt für Elektronik und Technische Informatik

**EIDESSTATTLICHE ERKLÄRUNG**

**Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.**

## Lukas Gritsch

## Michael Donner

Hollabrunn, am 7. April 2025

## HINWEISE

Die vorliegende Diplomarbeit wurde für die Abteilung Elektronik und Technische Informatik der HTL Hollabrunn ausgeführt.

Die in dieser Diplomarbeit entwickelten Prototypen und Software-Produkte dürfen ganz oder auch in Teilen von Privatpersonen oder Firmen nur dann in Verkehr gebracht werden, wenn sie diese selbst geprüft und für den vorgesehenen Verwendungszweck für geeignet befunden haben.
Es wird keinerlei Haftung übernommen für irgendwelche Schäden, die aus der Nutzung der hier entwickelten oder beschriebenen Bestandteile des Projekts resultieren.

Für alle Entwicklungen gilt die GNU General Public License [http://www.gnu.org/licenses/gpl.html] der Free Software Foundation, Boston, USA in der Version 3.

Die Diplomarbeit erfüllt die “Standards für Ingenieur- und Technikerprojekte” entsprechend dem Rundschreiben Nr. 60 aus 1999 des BMBWK (GZ.17.600/101-II/2b/99).
[https://www.bmb.gv.at/ministerium/rs/1999_60.html]

## SCHLÜSSELBEGRIFFE

Audioverarbeitungssystem

I2S-Protokoll

MDDS-System

Audiocodec

## DANKSAGUNGEN

Besonderer Dank geht an unseren Diplomarbeitbetreuer Dipl.-Ing Reisinger Josef, welcher uns selbstverständlich während der gesamten Diplomarbeit tatkräftig unterstützt hat. Dank gilt auch dem BEd Wolfgang Kauer, welcher beim Bestücken der Platinen mit seiner Expertise und seiner präzisen Handwerksarbeit unterstützt hat. Außerdem geht Dank an Herr Spindler Valentin und Herr Wurst Alexander für das Unterstützen des Bestückens der Platinen. Weiters geht Dank an den Dipl.-Ing Kühtreiber Andreas, welcher während den Sommerferien mit seiner Expertise ein Problem im Programm Altium Designer gelöst hat.

| | |
|---|---|
| htl bildung mit zukunft | **HÖHERE TECHNISCHE BUNDESLEHRANSTALT HOLLABRUNN** |
| | Fachrichtung: **Elektronik und Technische Informatik** |

# DIPLOMARBEIT
## DOKUMENTATION

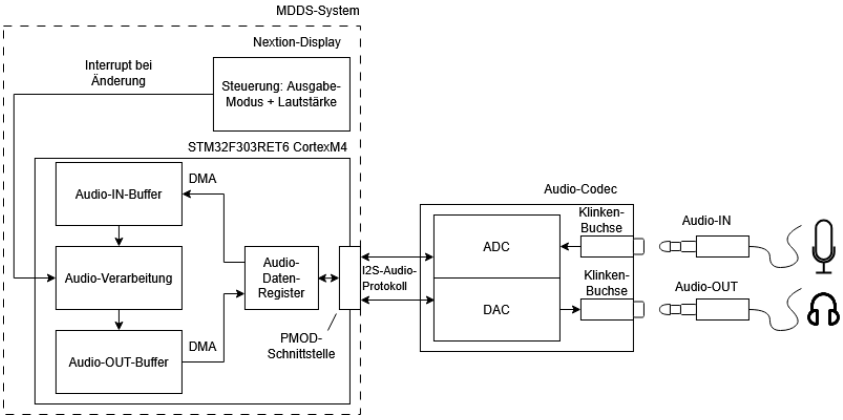
| | |
|---|---|
| Namen der Verfasser/innen | Donner Michael, Gritsch Lukas |
| Jahrgang Schuljahr | 5AHEL 2024/25 |
| Thema der Diplomarbeit | Digitale Audio Effekt Box |
| Kooperationspartner | |

| | |
|---|---|
| Aufgabenstellung | Ziel der Diplomarbeit ist den Audiocodec der bestehenden Audioplatine für das MDDS System mit dem TLV320AIC23B, welcher nicht mehr hergestellt wird durch einen Neuen zu ersetzen. Des Weiteren soll eine Software entwickelt werden, die es erlaubt verschiedene Audioeffekte in Form einer Laborübung zu realisieren. |

| | |
|---|---|
| Realisierung | Die Audioplatine wurde mithilfe von Altium Designer entwickelt, gefertigt und in Betrieb genommen. Als Vorbild wurde das PMOD I2S Modul von der Firma Digilant genommen und an das MDDS-System angepasst. Als ADC wurde der CS5343 und als DAC der CS4344 genommen. Die Platine wurde von Firma PCB-Way produziert und in der HTL Hollabrunn bestückt.<br><br>Parallel wurde eine Software für den STM32F107RBT6 entwickelt, welche mit dem Digilant Development Kit getestet wurde. Die Software übermittelt die Audiodaten mit dem I2S Protokoll, welche von der Software verarbeitet werden. Mit dem Nextion Display kann zwischen verschiedenen Audioeffekten und Signalformen gewählt werden. |

| | |
|---|---|
| Ergebnisse | Die Platine mit dem ADC CS5343 und DAC CS4344 wurde getestet und entspricht der den Zielanforderungen. Es wurde im Vergleich zur alten Diplomarbeit geschafft, mit nur einer PMOD-Schnittstelle Audiodaten empfangen und senden zu können.<br><br>Zuerst wurde die Software für den STM32F107RBT6 entwickelt und wegen der voll duplexen I2S-Schnittstelle auf den STM32F303RBT6 umgestiegen. Die Kommunikation mit dem Audiocodec über das I2S-Protokoll sowie mit dem Nextion-Display wurde erfolgreich hergestellt. Die Software unterstützt Audioeffekte wie Pitch-Shift, Frequenztrennung und Reverb sowie Signalgeneratoren für Rechteck-, Sägezahn-, Dreieck- und Sinussignale sowie AM-Modulation. |

| | |
|---|---|
| **HÖHERE TECHNISCHE BUNDESLEHRANSTALT HOLLABRUNN** | |
| Fachrichtung: | **Elektronik und Technische Informatik** |

| | |
|---|---|
| Typische Grafik, Foto etc.<br>(mit Erläuterung) |  <br><br>An das Audio-Modul kann über die Klinken-Buchse ein Audio-Signal eingespeist werden und Audio empfangen werden. Dabei wird typischerweise Audio von einem Handy oder durch ein verstärktes Mikrofon eingegeben und die Audio wird typischerweise durch Lautsprecher oder Kopfhörer ausgegeben.<br><br>**Audio-Kreislauf:**<br>Der eingespeiste Audio-Stream wird durch einen ADC digitalisiert und diese Daten werden dann über das I2S-Protokoll an den Cortex Mikrocontroller gesendet und in einem Audio-Daten-Register gespeichert. Von dort aus wird es in einen internen Audio-Eingangs-Buffer kopiert. Danach werden die Audio-Daten je nach ausgewähltem Modus verarbeitet und in den Audio-Ausgangs-Buffer geschrieben. Diese Daten werden wieder in das Audio-Daten-Register kopiert. Von dort aus werden die Audio-Daten über das I2S-Protokoll an den DAC übertragen, welcher das digitale Signal wieder in ein analoges Signal wandelt. Dieses Audio-Signal kann dann mit einem Lautsprecher ausgegeben werden.<br><br>Der Modus der Audio-Verarbeitung kann über das Nextion-Display gesteuert werden. Dieses schickt bei Änderung eine Nachricht an den Mikrocontroller, welcher dann den Modus ändert. |

| | |
|---|---|
| Teilnahme an Wettbewerben, Auszeichnungen | keine |

| | | |
|---|---|---|
| Möglichkeiten der Einsichtnahme in die Arbeit | HTL Hollabrunn<br>Anton Ehrenfriedstraße 10<br>2020 Hollabrunn | |
| Approbation<br>(Datum / Unterschrift) | Prüfer/Prüferin | Direktor/Direktorin<br>Abteilungsvorstand/Abteilungsvorständin |

| | |
|---|---|
| htl bildung mit zukunft | **HÖHERE TECHNISCHE BUNDESLEHRANSTALT HOLLABRUNN**<br>**COLLEGE of ENGINEERING** |
| | Department: **Electronics and computer engineering** |

# DIPLOMA THESIS
## Documentation

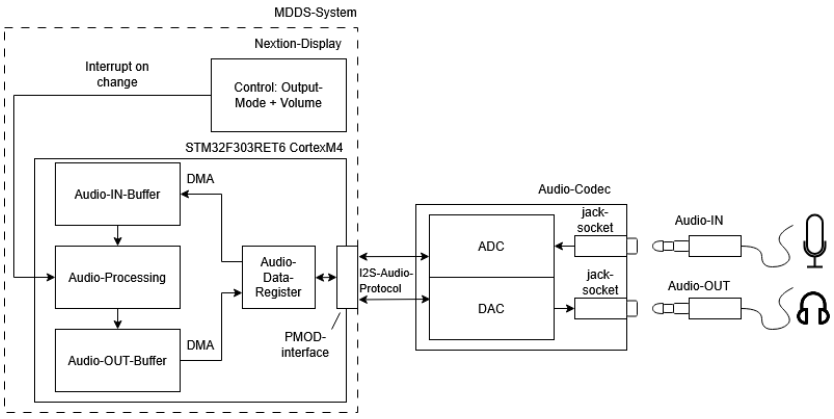
| | |
|---|---|
| Author(s) | Donner Michael, Gritsch Lukas |
| Form<br>Academic year | 5AHEL 2024/25 |
| Topic | Digital Audio Effect Box |
| Co-operation partners | |

| | |
|---|---|
| Assignment of tasks | The goal of the diploma thesis is to replace the audio codec of the existing audio board for the MDDS system with a new one, as the TLV320AIC23B is no longer being manufactured. Furthermore, software should be developed to enable the implementation of various audio effects in the form of a laboratory exercise. |

| | |
|---|---|
| Realisation | The audio board was developed, manufactured, and commissioned using Altium Designer. The design was based on the PMOD I2S module from Digilent and adapted to the MDDS system. The CS5343 was used as the ADC and the CS4344 as the DAC. The board was manufactured by PCB-Way and assembled at HTL Hollabrunn.<br>In parallel, software was developed and tested with the Digilent Development Kit. The software transmits audio data using the I2S protocol, which is then processed accordingly. The Nextion display allows users to select between different audio effects and signal forms. |

| | |
|---|---|
| Results | The board with the ADC CS5343 and DAC CS4344 has been tested and meets the target requirements. Compared to the previous diploma thesis, it was successfully achieved to transmit and receive audio data using only a single PMOD interface.<br>Communication with the audio codec via the I2S protocol, as well as with the Nextion display, was successfully established.<br>The software supports audio effects such as pitch shift, frequency separation, and reverb, as well as signal generators for square, sawtooth, triangle, and sine waves, along with AM modulation. |

| | |
|---|---|
| htl bildung mit zukunft | **HÖHERE TECHNISCHE BUNDESLEHRANSTALT HOLLABRUNN**<br>**COLLEGE of ENGINEERING** |
| | Department: **Electronics and computer engineering** |

| | |
|---|---|
| Illustrative graph, photo (incl. explanation) |  <br>An audio signal can be input and received through the audio module's 3.5mm jack. Typically, audio is input from a mobile phone or an amplified microphone, while the output is sent to speakers or headphones.<br><br>**Audio Loop:**<br>The input audio stream is digitized by an ADC and the data is transmitted to the Cortex microcontroller via the I2S protocol, where it is stored in an audio data register. From there, it is copied into an internal audio input buffer. The audio data is then processed according to the selected mode and written into the audio output buffer. After processing, the data is copied back into the audio data register. Finally, the audio data is sent via I2S to the DAC, which converts the digital signal back into an analog signal. This audio signal can then be played through a speaker.<br><br>The audio processing mode can be controlled via the Nextion display. When a mode change is requested, the display sends a message to the microcontroller, which then updates the processing mode accordingly. |

| | |
|---|---|
| Participation in competitions<br>Awards | none |

| | |
|---|---|
| Accessibility of final project thesis | HTL Hollabrunn<br>Anton Ehrenfriedstraße 10<br>2020 Hollabrunn |

| | | |
|---|---|---|
| Approval<br>(Date / Signature) | Examiner/s | Head of Department / College |

Berufsbildende Schulen

# Erklärung

Die unterfertigten Kandidaten/Kandidatinnen haben gemäß den geltenden schulrechtlichen Bestimmungen die Ausarbeitung einer abschließenden Arbeit (Diplomarbeit bzw. Abschlussarbeit) mit folgender Aufgabenstellung gewählt:

**Digital Audio Effekt Box**

Individuelle Aufgabenstellungen im Rahmen des Gesamtprojektes:

- Michael Donner (5AHEL): **Software Digital Audio Effekt Box**
- Lukas Gritsch (5AHEL): **Hardware Digital Audio Effekt Box**

Die Kandidaten/Kandidatinnen nehmen zur Kenntnis, dass die abschließende Arbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können, die jedenfalls als solche entsprechend kenntlich zu machen sind.

Die Abgabe der vollständigen abschließenden Arbeit hat in digitaler und in zweifach ausgedruckter Form bis spätestens **07.04.2025** beim zuständigen Betreuer/der zuständigen Betreuerin zu erfolgen.

Die Kandidaten/Kandidatinnen nehmen auch zur Kenntnis, dass ein Abbruch der abschließenden Arbeit nicht möglich ist.

| **Kandidaten/Kandidatinnen:** | **Datum und Unterschrift bzw. Handysignatur:** |
|---|---|
| Michael Donner (5AHEL) | 18.9.2024, Michael Donner |
| Lukas Gritsch (5AHEL) | 18.9.2024, Lukas Gritsch |

# Inhaltsverzeichnis


1 Einleitung ..... 13
1.1 Abkürzungsverzeichnis ..... 13
1.2 Begriffsverzeichnis ..... 14
2 Blockschaltbild des Gesamtsystems ..... 15
3 Theorie ..... 16
3.1 I2S ..... 16
3.1.1 Allgemein ..... 16
3.1.2 Abtastrate ..... 16
3.1.3 Wortbreite ..... 16
3.1.4 Signale ..... 17
3.1.5 Ports ..... 18
3.1.6 Timing-Diagramm ..... 18
3.2 Audiocodec ..... 19
4 Hardware Audio Effekt Box ..... 20
4.1 Audioplatine mit Audio-Codec TLV320AIC32 ..... 20
4.2 Einführung in den Audiocodec TLV320AIC32 ..... 20
4.3 Musterschaltung ..... 21
4.4 Schaltung ..... 22
4.4.1 Analoge vs. Digitale Masse/Versorgung ..... 22
4.4.2 Buck-Converter 1,8V (MAX38640BELT18+T) ..... 23
4.4.3 Audio-Switch TS5A22364 ..... 25
4.4.4 Taktgenerator 12MHz (ASEMB-12 000MHZ-LY-T) ..... 26
4.4.5 Beschaltung des TLV320AIC32 ..... 26
4.4.6 Gesamtschaltung ..... 29
4.5 Layout ..... 30
4.6 Fehlersuche ..... 31
4.6.1 Messungen ..... 31
4.6.2 Erkenntnisse ..... 34
5 Audioboard mit Audio-Codec TAC5142 ..... 35
5.1 Einführung in den TAC5142 ..... 35
5.2 Beispielschaltung ..... 36
5.3 Schaltung ..... 37
5.3.1 Audio-Switch TS5A22364 ..... 37
5.3.2 Beschaltung des TAC5142 ..... 37
5.3.3 Gesamtschaltung ..... 40
5.4 Layout ..... 41
5.5 Demo Software ..... 41
5.5.1 Konfigurations-Pins ..... 41
5.5.2 Demosoftware ..... 43
**5.5.2.1** CubeMX-Konfiguration ..... 43
**5.5.2.2** Programmcode ..... 44
5.6 Breakoutboard ..... 45
5.6.1 Schaltung ..... 45
5.6.2 Layout ..... 46

5.7 Fehlersuche ..... 47
5.7.1 Messungen ..... 47
5.7.2 Erkenntnisse ..... 50
6 Audioboard mit ADC CS5343 und DAC CS4344 ..... 51
6.1 Einführung in den CS5343 und CS4344 ..... 51
6.2 Musterschaltung ..... 51
6.3 Schaltung ..... 52
6.3.1 Master/Slave-Auswahl ..... 52
6.3.2 Gesamtschaltung ..... 53
6.4 Layout ..... 53
6.5 Ergebnisse ..... 54
6.5.1 Messungen ..... 54
6.5.2 Ergebnisse ..... 57
7 Software Audio Effekt Box ..... 58
7.1 Software-Architektur ..... 58
7.1.1 Einleitung ..... 58
7.1.2 Blockschaltbild ..... 58
7.1.3 Beschreibung ..... 58
7.2 Entwicklungs-Hardware ..... 59
7.2.1 Allgemein ..... 59
7.2.2 Audio-Codec ..... 59
7.2.3 Platine ..... 59
7.2.4 Base Board Ports ..... 60
7.3 Auswahl des Mikrocontrollers ..... 60
7.3.1 Allgemein ..... 60
7.3.2 STM32F107RBT6 ..... 60
7.3.3 STM32F407G-DISC1 ..... 61
7.3.4 STM32F303RET6 ..... 61
7.4 Konfiguration Mikrocontroller STM32F303RET6 ..... 62
7.4.1 Allgemein ..... 62
7.4.2 STM32CubeMX ..... 62
7.4.3 Nicht verwendete Projekte ..... 62
7.4.3.1 Allgemein ..... 62
7.4.3.2 STM32F107RCT6 Master-Master Version ..... 63
7.4.3.3 STM32F107RCT6 Master-Slave Version ..... 64
7.4.3.4 Discovery-Board STM32F407G-DISC1 ..... 64
7.4.4 Projekt Erstellung mit STM32F303RET6 Mikrocontroller ..... 65
7.4.5 Debug ..... 66
7.4.6 Clock Konfiguration ..... 66
7.4.7 I2S2 Audio-Protokoll Konfiguration ..... 67
7.4.8 USART3 Konfiguration ..... 71
7.4.9 Pinbelegung des Cortex ..... 72
7.4.10 Code Generation ..... 73
7.6 Framework Audioverarbeitung ..... 74
7.6.1 DMA ..... 74
7.6.2 Interrupt ..... 74

7.6.3 Grundprinzip ..... 74
7.6.4 Allgemeiner Ablauf ..... 76
7.6.5 Interrupt Auslösung ..... 76
7.6.6 Erstelltes Projekt ..... 77
7.6.7 Main.c ..... 77
7.6.8 Interrupt-Service-Routinen ..... 78
7.6.8.1 DMA-Interrupt ..... 78
7.6.8.2 UART3-Interrupt ..... 79
7.6.9 Ausgabe-Modus ..... 80
7.6.10 Lautsärkenregelung ..... 80
7.7 Signalgeneratoren ..... 81
7.7.1 Allgemein ..... 81
7.7.2 Sägezahn ..... 81
7.7.3 Dreieck ..... 82
7.7.4 Rechteck ..... 83
7.7.5 Sinus ..... 84
7.7.6 AM ..... 85
7.8 Modus Auswahl mit Nextion Display ..... 86
7.8.1 Allgemein ..... 86
7.8.2 Nextion-Editor ..... 86
7.8.2.1 Projekt Erstellung ..... 86
7.8.2.2 Debug ..... 89
7.8.3 UART3-Kommunikation ..... 90
7.8.3.1 Allgemein ..... 90
7.8.3.2 Port ..... 90
7.8.3.3 Protokoll ..... 91
7.8.3.4 Protokoll Diagramme ..... 91
7.9 Audio-Effekte ..... 93
7.9.1 Allgemein ..... 93
7.9.2 Frequency Split ..... 93
7.9.2.1 Allgemein ..... 93
7.9.2.2 IIR-Filter ..... 93
7.9.2.3 Matlab Filter Designer ..... 94
7.9.2.4 Code ..... 95
7.9.3 Pitch Shift (Mickey Mouse Voice) ..... 96
7.9.3.1 Allgemein ..... 96
7.9.3.2 Grundprinzip ..... 97
7.9.3.3 Code ..... 98
7.9.4 Reverb ..... 100
7.9.4.1 Allgemein ..... 100
7.9.4.2 Grundprinzip ..... 100
7.9.4.3 Code ..... 101
7.9.5 Wah-Wah-Effekt ..... 103
7.9.5.1 Allgemein ..... 103
7.9.5.2 Grundprinzip ..... 103
7.9.5.3 Code ..... 104

8 Inbetriebnahme des Systems ..... 106
9 Verwendete Software ..... 108
9.1 Altium Designer ..... 108
9.2 STM32CubeMX ..... 108
9.3 Keil uVision5 ..... 108
9.4 Logic 1.1.15 ..... 108
9.5 Matlab R2024a ..... 109
9.6 Nextion Editor ..... 109
9.7 Draw.io ..... 109
10 Zeitaufwand ..... 110
10.1 Lukas Gritsch ..... 110
10.2 Michael Donner ..... 110
11 Kostendarstellung ..... 111
12 Projektplan ..... 112
13 Quellen ..... 113
13.1 Gedruckte Medien ..... 113
13.2 Online ..... 113
14 Verzeichnis der Abbildungen ..... 116

# 1 Einleitung

Im Rahmen dieser Diplomarbeit wurde eine Audioplatine entwickelt, welche an das MDDS-System angeschlossen werden kann. Mit dieser Audioplatine sollen mit dem I2S-Protokoll Audiosignale eingelesen und ausgegeben werden können. Zusätzlich sollen verschiedene Audioeffekte programmiert werden, um die Grundlagen der Audioverarbeitung aufzuzeigen. So eine Platine wurde schon 2019 von einer ehemaligen Diplomarbeitsgruppe entwickelt, jedoch ist der verwendete Audiocodec nicht mehr in Produktion, weswegen eine Platine mit erhältlichen Bauteilen entwickelt werden soll.
Mit dieser Platine sollen Schüler der HTL Hollabrunn zukünftig in Form von einem DIC-Projekt oder einer Laborübung Audioverarbeitung mit dem ihnen bekannten MDDS-System kennenlernen.

## 1.1 Abkürzungsverzeichnis

| | |
|---|---|
| ADC | Analog-Digital Converter |
| DAC | Digital-Analog Converter |
| DMA | Direct Memory Access |
| GPIO | General Purpose Input Output |
| HPF | High Pass Filter |
| I2C | Inter Integrated Circuit |
| I2S | Inter IC Sound |
| IIR | Infinite Impulse Response |
| MCU | Micro Controller Unit |
| PCB | Leiterplatte |
| PLL | Phase-locked Loop |
| PMOD | Peripheral Module Interface |
| USART | Universal Synchronous Asynchronous Receiver Transmitter |
| V4 | bezeichnet die Platine der alten Diplomarbeit |
| µC | Microcontroller |

## 1.2 Begriffsverzeichnis

| | |
|---|---|
| **Audio-Sample** | bezeichnet einen momentanen Abtastwert eines analogen Audio-Signals. |
| **Buck Converter** | ist ein IC, welcher eine Gleichspannung in eine Kleinere umwandelt. |
| **Full-Duplex** | bedeutet, dass gleichzeitig empfangen und gesendet werden kann. |
| **Half-Duplex** | bedeutet, dass entweder gesendet oder empfangen werden kann. |
| **I2C** | ist ein serielles Kommunikationsprotokoll. |
| **I2S Target Mode** | ist der Modus, in dem sich ein Gerät als I2S-Slave verhält. |
| **Interrupt** | ist ein Mechanismus, der die Programmausführung unterbricht. |
| **Lookup-table** | Ein Speicherort, wo Informationen die öfter gebraucht werden, abgespeichert sind (z.B.: Sinus-Werte über eine Periodendauer). |
| **Master** | ist derjenige, der für die Steuerung der Übertragung zuständig ist. |
| **MDDS System** | ist ein schuleigenes System, bei dem ein µC mit der Peripherie interagiert. |
| **PLL** | ist eine Regelschleife, welche mit Zusatzbeschaltung aus eine Frequenz in eine andere umwandeln kann. |
| **PMOD** | ist ein Stecker Standard der Firma Digilant mit dem zweimal Versorgung und zweimal Masse sowie acht Datenpins verknüpft werden. |
| **Slave** | ist derjenige, der Daten liest oder schickt, aber nicht für die Steuerung der Übertragung zuständig ist. |
| **Stereo** | bedeutet, dass das Audiosignal in links und rechts aufgeteilt ist. |
| **Zweierkomplement** | ist eine Darstellung einer Integer-Zahl, wobei das erste Bit aussagt, ob die Zahl negativ = 1 oder positive = 0 ist. |

# 2 Blockschaltbild des Gesamtsystems

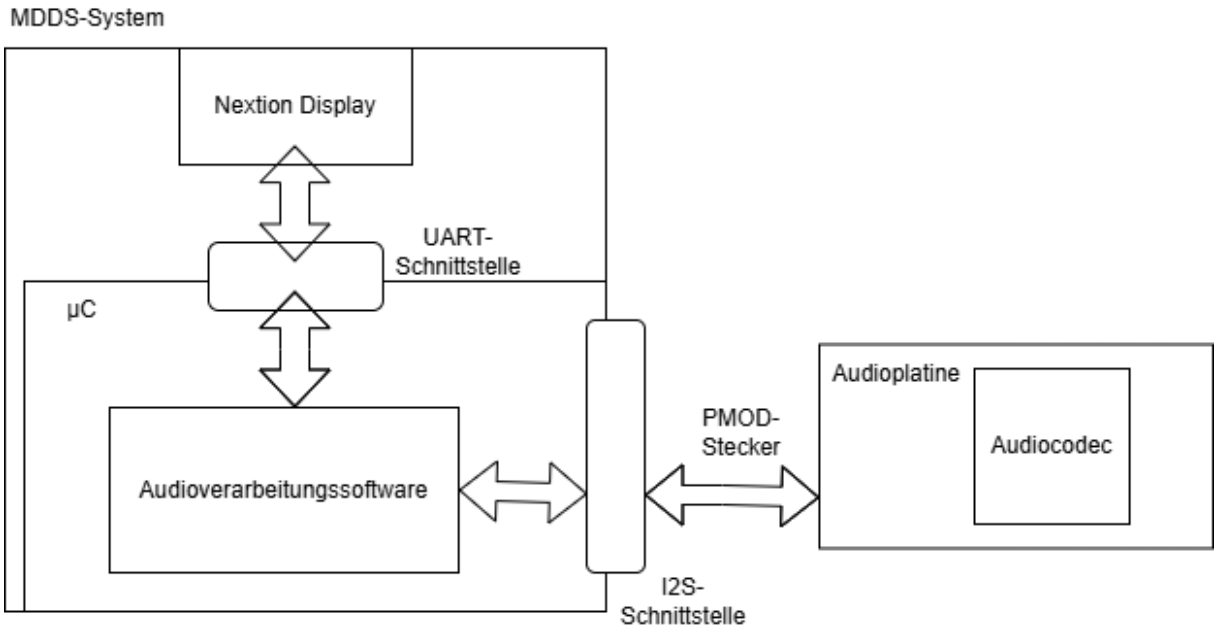



*Abbildung 1: Blockschaltbild des Gesamtsystems*

Die Digitale Audio Effekt Box kann in zwei Teile aufgeteilt werden. Der erste Teil ist das MDDS-System, welches für Verarbeitung der Audio zuständig ist. Dieses System ist ein von der HTL Hollabrunn entwickeltes System, was bedeutet, dass für diesen Teil nur Software entwickelt werden muss. Die Entwicklung dieser Software ist im Kapitel 7 zu finden. Der zweite Teil ist die Audioplatine, die zwischen Audiosignal und I2S-Daten wandelt. Für die Audioplatine muss hauptsächlich die Hardware entwickelt werden. Das Entwickeln dieser Hardware ist im Kapitel 4, 5 und 6 zu finden.

Das MDDS-System kann in µC und Nextion Display unterteilt werden. Das Nextion Display ist für die Kommunikation mit dem Nutzer zuständig. Die Eingaben des Nextion Displays werden über eine UART-Schnittstelle übermittelt. Auf dem µC läuft eine Audioverarbeitungssoftware, welche die Daten der Audioplatine und die Eingaben auf dem Nextion Display verarbeitet. Zurzeit wird als µC der STM32F303RET6 verwendet.

Auf der Audioplatine befindet sich ein Audiocodec, der für das Umwandeln der Daten zuständig ist. Die Audioplatine ist durch einen PMOD-Stecker mit dem MDDS-System verbunden. Der PMOD-Stecker ist ein Standard der Firma Digilant. Die Kommunikation zwischen Audioplatine und µC findet über eine I2S-Schnittstelle statt.

# 3 Theorie

## 3.1 I2S

### 3.1.1 Allgemein

Das I2S (Inter-IC Sound) Audio-Protokoll ist ein serielles Kommunikationsprotokoll, welches für die Übertragung von digitalen Audiosignalen zwischen integrierten Schaltkreisen (ICs) entwickelt wurde. In unserem Fall ist das der Audio-Codec und Mikrocontroller. Es unterscheidet dabei zwischen 2 Audiokanälen (left, right), welche typischerweise für das linke und rechte Ohr verwendet werden.

Dabei gibt es verschiedene Protokollversionen von I2S, wie „I2S Philips standard“, „MSB-justified Standard“, „LSB-justified Standard“ und „PCM Standard“. In unserem Fall wurde das meist verbreitete „I2S Philips standard“-Protokoll verwendet.

### 3.1.2 Abtastrate

Die Abtastrate ist eine Frequenz, die angibt wie oft in der Sekunde ein Audiosample eingelesen bzw. ausgegeben wird. Je höher die Abtastrate ist, desto genauer kann das tatsächliche Signal digital nachgebildet werden und desto besser ist die Audio-Qualität. In unserem Fall beträgt die Abtastrate **$f_s$ 48kHz**.

Diese Abtastrate hat einen speziellen Grund, der mit dem Frequenzbereich zu tun hat, den der Mensch hört. Dieser ist ungefähr von 20Hz – 20kHz.
Laut dem Nyquist-Kriterium kann eine Frequenz ohne Störung digital erfasst werden, solange die Abtastrate das Doppelte oder größer als die zu abtastende Frequenz ist.
**2 * 20kHz** (maximale Frequenz) = **40kHz** und liegt somit unter **48kHz,** was das Kriterium erfüllt jede hörbare Frequenz störungsfrei zu erfassen.

### 3.1.3 Wortbreite

Die Auflösung der Audio-Samples kann unterschiedlich sein. Je mehr Bit Auflösung desto genauer kann das Originalsignal digitalisiert werden. Für das I2S-Protokoll gibt es standardmäßig 16bit, 24bit oder 32bit. Da beide unserer ADC und DAC eine Auflösung von 24bit haben, wurde die Wortbreite bei uns auf 24bit gewählt.

Es kann jedoch nur auf einem Frame von 16bit oder 32bit übertragen werden, weshalb ein Audio-Sample mit 24bit auf einem Frame von 32bit übertragen wird. Die restlichen 8bit werden mit „0“ gefüllt.

### 3.1.4 Signale

Alle Signale des I2S-Protokolls richten sich in ihrer Frequenz nach der Abtastraten-Frequenz und zum Teil auch nach der Wortbreite.

Bei dem I2S Audio-Protokoll gibt es grundsätzlich 4 Signale:

| | |
|---|---|
| **SD** = Serial Data | Leitung, die die Daten des Audio-Samples überträgt. |
| **WS** = Word Select | Clock: Gibt an, ob gerade ein Audio-Sample für den left- oder right-Channel geschickt wird. **Frequenz = Abtastrate $f_s$ = 48kHz** |
| **BCLK** = Bit-Clock | Clock: Gibt den Takt an, in dem die SD-Leitung Daten überträgt. **Frequenz = Bitanzahl des Frames * Anzahl der Kanäle * $f_s$ = 32 * 2 * 48kHz = 3.072Mhz** |
| **MCLK** = Master Clock (optional) | Clock: Diese ist optional, wird aber von manchen Audio-Codec's gebraucht und synchronisiert alle Signale. **Frequenz = 256 * fs = 12.288Mhz** |

Aus Sicht des Cortex werden zwei verschiedene Modi unterschieden:

| | |
|---|---|
| **Master** | Der Mikrocontroller erzeugt alle Clock-Signale. |
| **Slave** | Der Audio-Codec erzeugt alle Clock-Signale. |

In unserem Fall erzeugt der Mikrocontroller alle Clock-Signale und ist daher der Master.

Der aktuell beschriebene Aufbau ist nur Half-Duplex, da nur Audio-Daten in eine Richtung gesendet werden können. Da aber Audio-Samples eingelesen und ausgegeben werden sollen, gibt es den Full-Duplex I2S-Betrieb, welchen wir benutzen.

Dieser erlaubt es mit derselben Synchronisation der bestehenden Clocks eine weitere Datenleitung zu benutzen. Damit müssen die Clock-Signale nicht zweifach generiert werden.

| | |
|---|---|
| **SD** = Serial Data | Audio-Sample Daten die der Mikrocontroller aussendet. |
| **SD_ext** = Serial Data external | Audio-Sample Daten die der Mikrocontroller empfängt. |

### 3.1.5 Ports

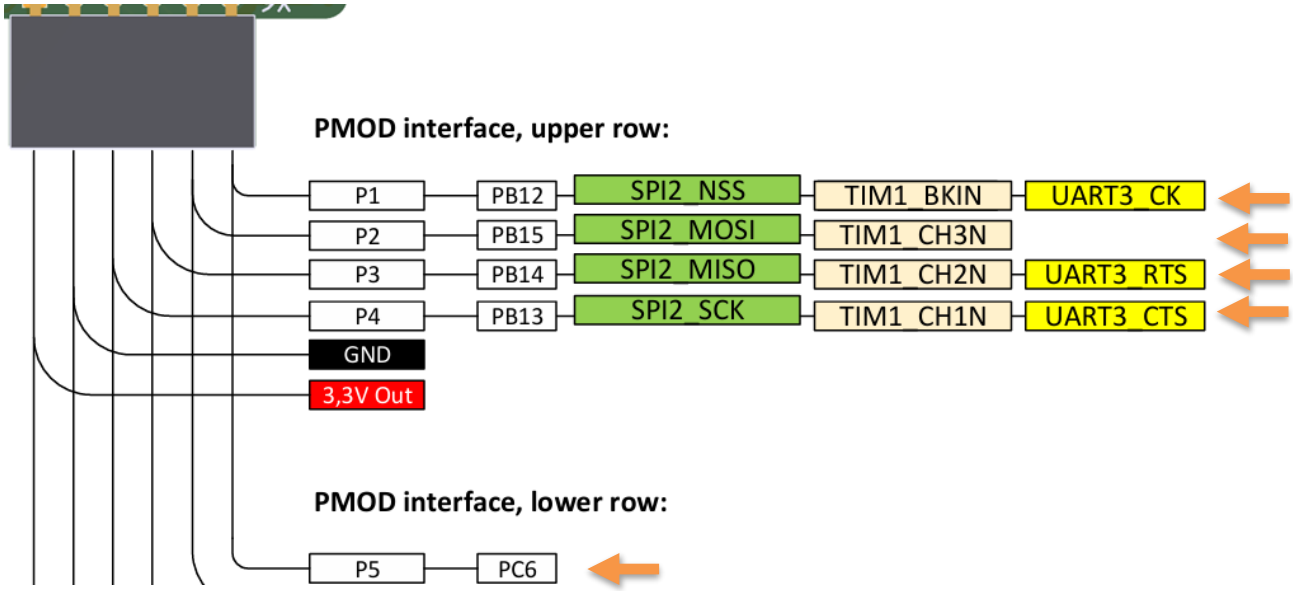



*Abbildung 2: Ports des I2S-Protokoll*

Alle Ports, die verwendet werden, sind auf der Controller-Platine auf einer einzigen PMOD-Schnittstelle zu finden. Da die Hardware-Architektur der SPI-Kommunikation sehr ähnlich zu der des I2S-Protokolls ist, sind diese auf denselben Ports.

| | |
|---|---|
| PB12 | WS |
| PB15 | SD |
| PB14 | SD_ext |
| PB13 | BCLK |
| PC6 | MCLK |

### 3.1.6 Timing-Diagramm

Dieses Timing-Diagramm zeigt die Zusammenhänge zwischen BCLK bzw. CK, WS und SD für 24bit Wortbreite, die in unserem Fall verwendet wurde.

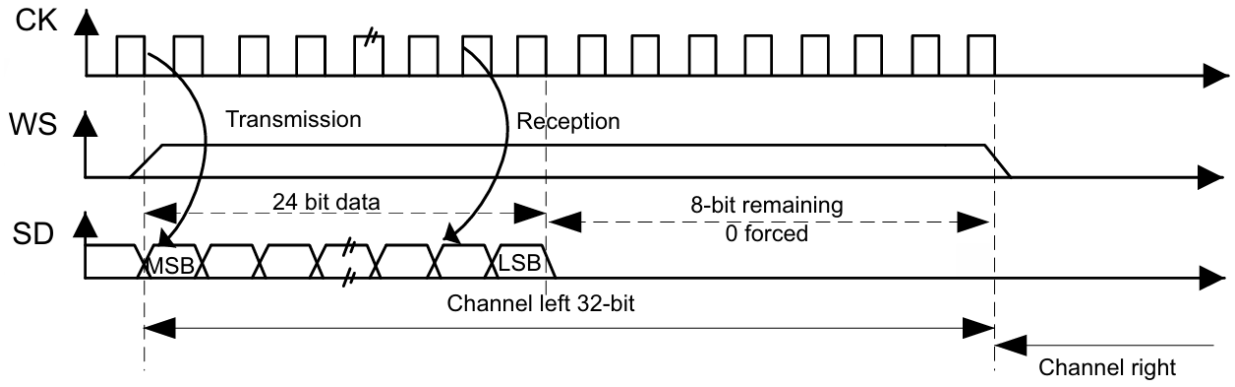



*Abbildung 3: I2S Timing-Diagramm [STC25]*

Wenn der Left-Channel angesprochen wird, ist WS high. Wenn der Right-Channel angesprochen wird, ist WS low.
Wenn BLCK Low ist, ändert sich die SD-Leitung auf den nächsten Wert.
Wenn BLCK High ist, sind die Daten auf der Datenleitung SD gültig.
Die letzten 8bit in jedem WS-Frame sind alle Low, da sie nicht verwendet werden.

## 3.2 Audiocodec

Ein Audiocodec wandelt Audiosignale von analog zu digital mit einem ADC und von digital zu analog mit einem DAC. Im digitalen Bereich gibt es verschiedene Audioprotokolle, wie I2S, Left/Right-Justified und TDM, die ein Audiocodec unterstützen kann, um Audiosignale digital darzustellen. Die verwendeten µC, wie zum Beispiel STM32F107T6 unterstützen das I2S-Protokoll. Ein Audiocodec kann entweder durch Konfigurationspins oder durch ein Steuerprotokoll, wie zum Beispiel I2C konfiguriert werden. Des Weiteren unterstützen die meisten Audiocodecs einen „Mic Bias“, welcher für die Versorgung von passiven Mikrophonen genutzt wird.

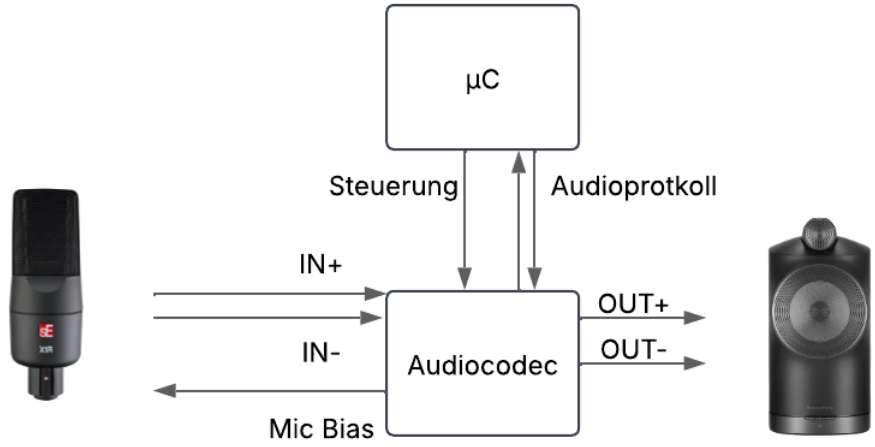



*Abbildung 4: Blockschaltbild des Audio*

# 4 Hardware Audio Effekt Box

Dieses Kapitel beschreibt die Entwicklung der Hardware von die Audioeffektbox. Von einer ehemaligen Diplomarbeit wurde schon eine ähnliche Audioplatine entworfen. Die alte Diplomarbeit heißt „GLADOS – Guided Low Altitude Detection and Observation System“. Die Audioplatine dieser Diplomarbeit ist die V4. Unsere Diplomarbeit entwickelt eine neue Version dieser Audioplatine und ist somit die V5.

## 4.1 Audioplatine mit Audio-Codec TLV320AIC32

Diese Audioplatine ist die Version 5.0.0. Sie ist der erste Versuch, die ehemalige Audioplatine V4 ersetzen. Hier wurde der TLV320AIC32 eingesetzt, weil dieser viele Optionen zur Konfiguration des I2S-Protokolls unterstützt. Der Audiocodec wird durch das I2C-Protokoll konfiguriert, wodurch schnell viele verschiedene Konfigurationen des I2S-Protokolls getestet werden können. Diese Platine ist eine Kombination aus der Schaltung der Audioplatine V4 und der Musterschaltung des Datenblatt des TLV320AIC32.

## 4.2 Einführung in den Audiocodec TLV320AIC32

Dieser Audiocodec verfügt über sechs Stereo Inputs und sechs Stereo Outputs. Er kann über eine I2C-Schnittstelle konfiguriert werden. Mit ihr kann der Microphone Bias, die integrierte PLL und das Audioprotokoll konfiguriert werden. Der Audiocodec unterstützt I2S, sowie andere Audioübertragungsprotokolle. Außerdem kann zwischen 16, 20, 24 und 32 Bit Daten sowie einer Abtastrate von 8 bis 96kHz ausgewählt werden. Des Weiteren braucht er eine 3,3V und eine 1,8V Versorgung. [TI25b]

## 4.3 Musterschaltung

Dies ist die Audioplatine V4. Sie wurde von der alten Diplomarbeitsgruppe entwickelt. Die Audioplatine kann an das MDDS-System angeschlossen werden. Sie soll Audiosignale von einem Klinkenstecker und einer Cinchbuchse in ein I2S-Protokoll umwandeln. Ein Audioswitch erlaubt die Umschaltung zwischen Cinchbuchse und Klinkenbuchse. Mit einem Jumper kann zwischen den beiden Eingängen ausgewählt werden. Außerdem soll ein I2S-Protokoll empfangen und in ein Audiosignal umwandeln werden. Das Audiosignal soll über einen Klinkenstecker und einen Cinchstecker abgegriffen werden können.

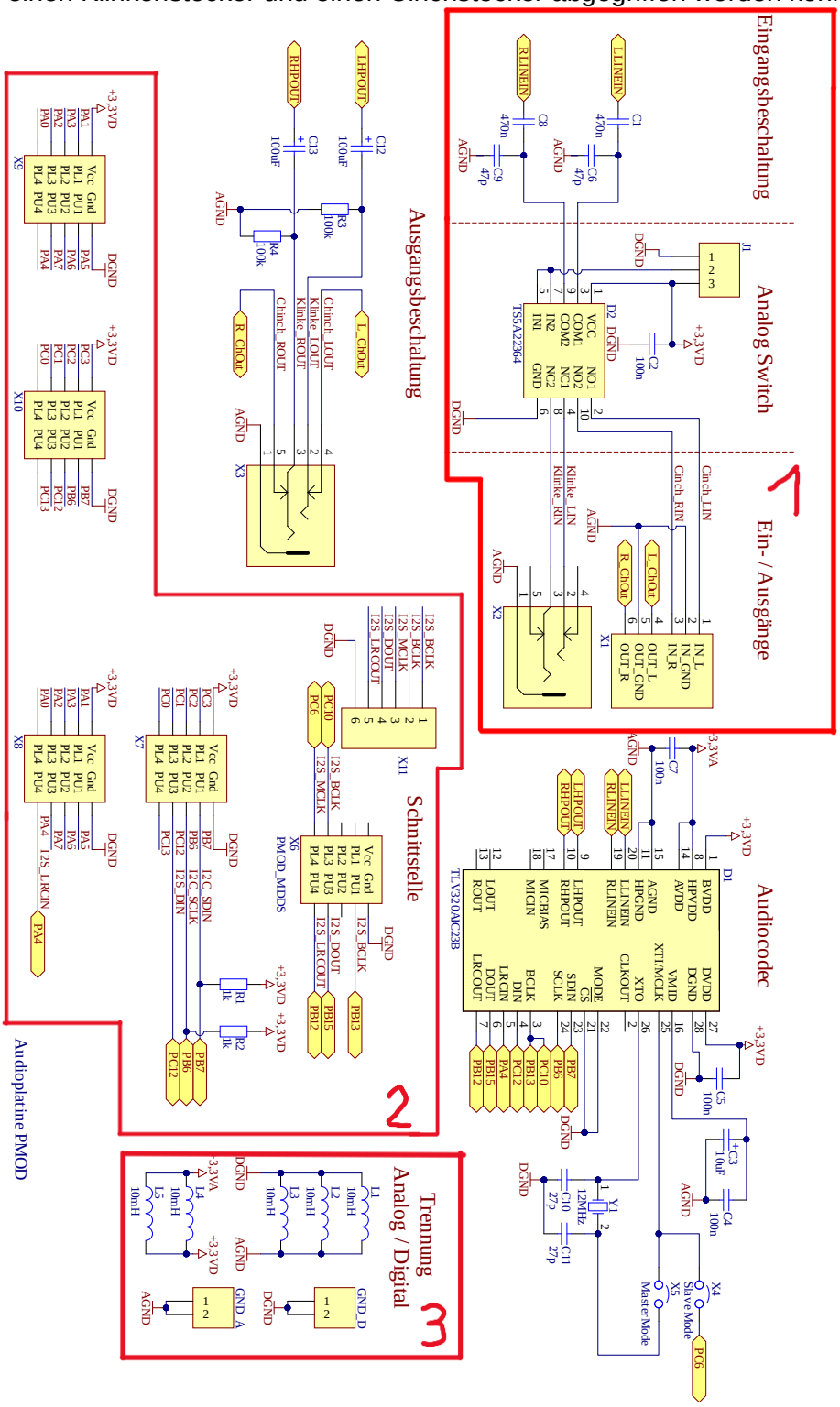



*Abbildung 5: Schemetic der Audioplatine V4 [GLA25]*

Aus der Schaltung in Abbildung 5 wurde die Beschaltung des Audioswitch TS5A22364 übernommen. Der mit 1 markierte Bereich wurde kopiert. Die Leitung, welche das Audiosignal zum Audiocodec führt, wird in Abbildung 5 mit 4 Kondensatoren bestückt. Bei der Audioplatine mit dem TLV320AIC32 werden andere Kondensatoren verwendet. Es werden stattdessen die Kondensatoren verwendet, welche bei Abbildung 6 mit 2 markiert sind. In Abbildung 5 wird die Trennung zwischen analoger und digitaler Masse, sowie die Trennung zwischen analoger und digitaler Versorgung mit 3 markiert. Dieser Teil der Schaltung wird 1:1 übernommen. Die Schnittstellen, die in Abbildung 5 mit 2 markiert sind werden übernommen und sind in der Gesamtschaltung mit 7 markiert.

Diese Musterschaltung wurde dem Datenblatt des Audiocodecs TLV320AIC32 entnommen. Aus dieser Schaltung wurde die Beschaltung der Versorgung wie in Abbildung 6 mit 1 markiert, kopiert. Des Weiteren wurde sich am Audioeingang, der mit 2 markiert wurde, orientiert. Der Audioausgang, der mit 3 markiert ist, wurde nicht ganz übernommen. Der Unterschied ist, dass HPLCOM und HPRCOM verbunden sind und als GND für den Klinkenstecker und die Cinchbuchse genutzt wird. Dies sorgt für eine bessere Rauschunterdrückung, weil der GND der Platine und der Klinkenbuchse getrennt sind. In der Gesamtschaltung ist dieser Teil mit 3 markiert.

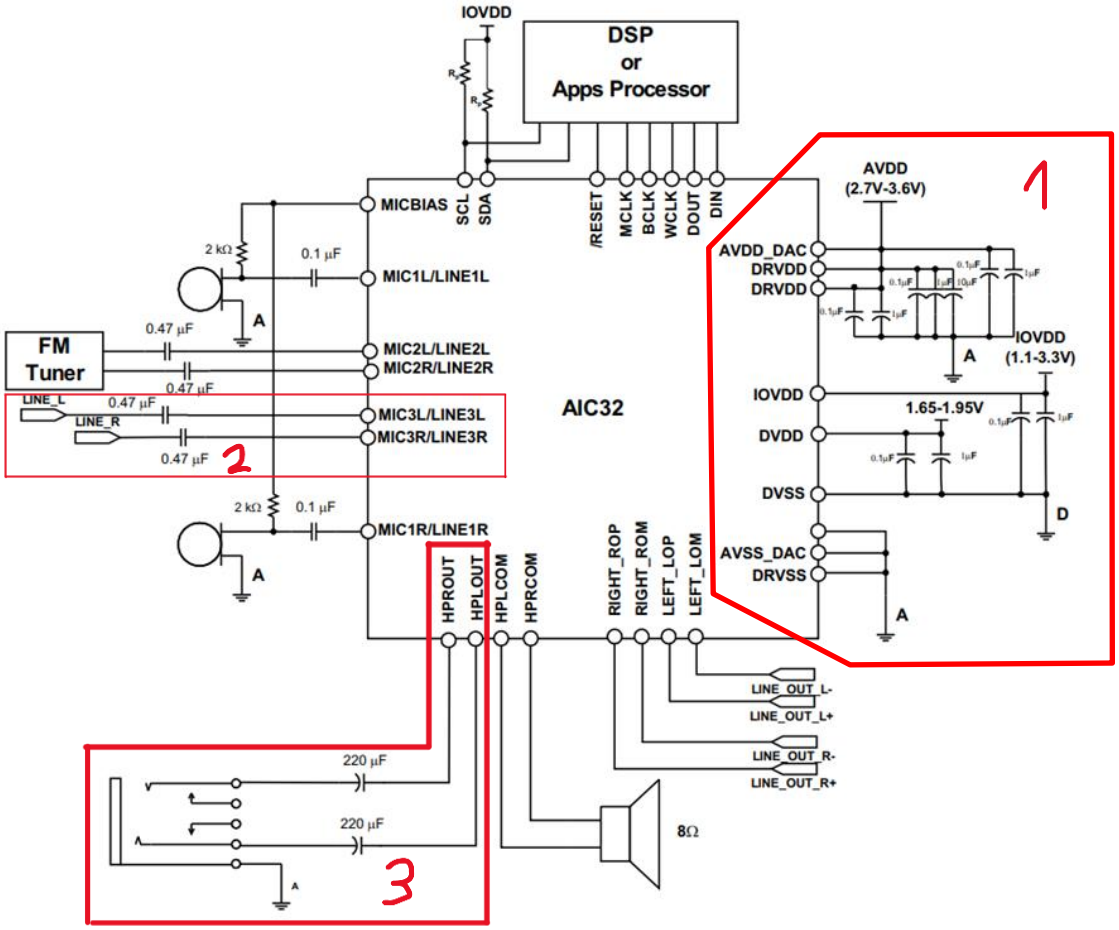



*Abbildung 6: Musterschaltung aus dem Datenblatt des TLV320AIC32 [TI25b]*

## 4.4 Schaltung

### 4.4.1 Analoge vs. Digitale Masse/Versorgung

Die Masse und die Versorgung für die analogen und digitalen Bauteile werden getrennt. Die beiden Teile sind mit Ferrit Beads verbunden, welche hochfrequente Störungen nicht durchlassen und somit können die Störungen von den digitalen Schaltelementen nicht auf die analogen übertragen werden. [MIK25]

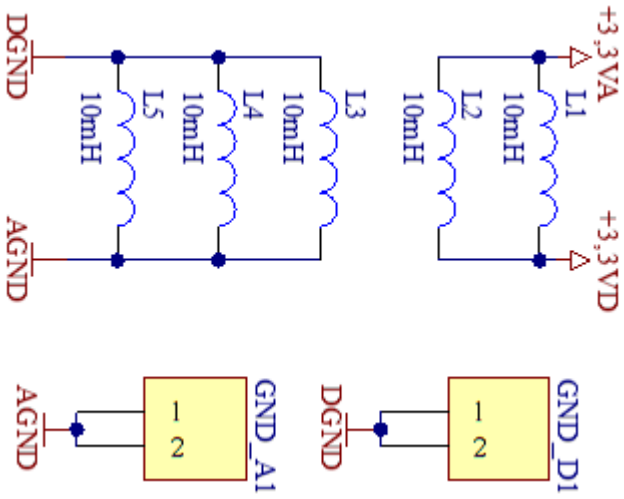



*Abbildung 7: Schaltung der Trennung von digital und analog*

Hier ist die Trennung der analogen und digitalen Schaltelemente aufgezeigt. Sie wird für diese Version der Audioplatine benutzt. In der Gesamtschaltung ist dieser Teil der Schaltung mit 1 markiert. Im Kapitel 4.4 Layout wird die Trennung von digital und analog noch einmal dargestellt.

### 4.4.2 Buck-Converter 1,8V (MAX38640BELT18+T)

Der Buck-Converter wandelt die 3,3V vom MDDS-System in 1,8V für den DVDD-Pin des Audiocodec TLV320AIC32 um. Dieser Teil ist in der Gesamtschaltung mit 4 markiert.

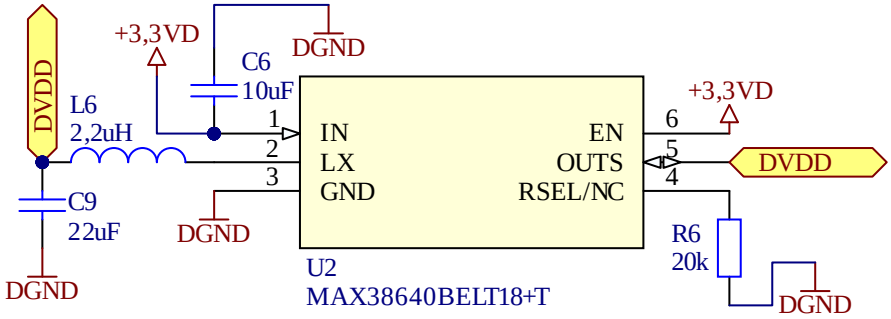



*Abbildung 8: Schaltung des Buck Converters*

Das ist die Schaltung mit dem Buck-Converter, der für die Version der Audioplatine verwendet wurde. Die Pins IN und EN sind mit digitalen 3,3V verbunden und IN hat noch einen 10uF Stützkondensator verbaut. EN aktiviert den Buck Converter. Zwischen OUTS und LX ist eine Induktivität verbaut, weil diese für die Regelung gebraucht wird und nicht direkt im IC ist.

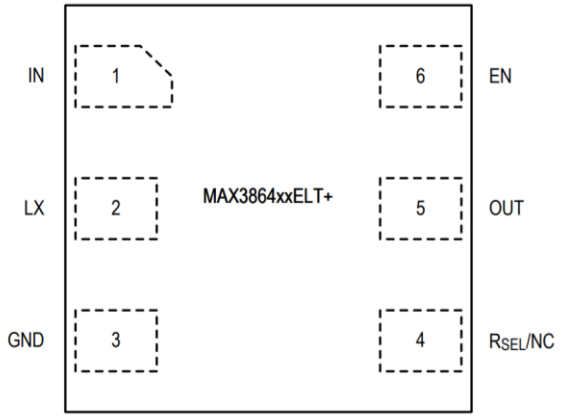



*Abbildung 9: Buck-Converter aus dem Datenblatt [MOU25c]*

| PIN | | NAME | FUNCTION |
|---|---|---|---|
| MAX3864xxELT+ | MAX3864xxENT+ | | |
| 1 | A1 | IN | Regulator Supply Input. Connect to a voltage between 1.8V and 5.5V and bypass with a ceramic capacitor from IN to GND. |
| 2 | A2 | LX | Switching Node. Connect an inductor between LX and the regulator output. |
| 3 | A3 | GND | Ground. |
| 4 | B3 | $R_{SEL}$/NC | **MAX38640A/1A/2A/3A:** Output Voltage Select Input. Connect a resistor from $R_{SEL}$ to GND to program the output voltage and IN undervoltage threshold based on Table 1.<br><br>**MAX38640B/1B/2B/3B:** No Connect. Leave floating. |
| 5 | B2 | OUT | Output Voltage Sense Input. Connect to the load at a point where accurate regulation (output capacitor) is required to eliminate resistive metal drops. |
| 6 | B1 | EN | Enable Input. Force this pin high to enable the buck converter. Force this pin low to disable the part and enter shutdown. |

*Abbildung 10: Pinning aus dem Datenblatt [MOU25c]*

Mit der Information aus diesem Pinning wurde die Beschaltung des Buck-Converters entwickelt. Der Pin 4 $(R_{SEL})$ wird wie in der Tabelle (Abbildung 11) beschrieben mit 20kΩ zu GND verbunden, damit die 3,3V Eingangsspannung zu 1,8V umgewandelt wird.

| TARGET OUTPUT VOLTAGE (V) | $R_{SEL}$ (kΩ) | INPUT UVLO THRESHOLD, RISING (V) |
|---|---|---|
| 2.75 | 34 | 2.6 |
| 2.5 | 28 | 2.6 |
| 2 | 23.7 | 2.6 |
| 1.8 | 20 | 2.6 |
| 1.5 | 16.9 | 2.6 |
| 1.25 | 14 | 2.6 |
| 1.2 | 11.8 | 2.6 |
| 1.15 | 10 | 2.6 |
| 1.1 | 8.45 | 2.6 |
| 1 | 7.15 | 2.6 |
| 0.95 | 5.9 | 2.6 |
| 0.9 | 4.99 | 2.6 |
| 0.8 | SHORT TO GND | 2.6 |

*Abbildung 11: Tabelle zur Konfiguration der Ausgangsspannung aus dem Datenblatt [MOU25c]*

Die linke Spalte zeigt die erzeugte Spannung, die mittlere den notwendigen Widerstand und die rechte die Spannung, die mindestens anliegen muss, damit die gewollte Spannung erzeugt werden kann.

### 4.4.3 Audio-Switch TS5A22364

Prinzipiell wurde die Schaltung für den Audioswitch der Musterschaltung in Abbildung 5 exakt übernommen. Die Schaltung ist in Abbildung 5 mit 1 markiert. Der Audio-Switch wählt aus, ob der Eingang von der Cinchbuchse oder von der 3,5mm-Klinkenbuchse zum Audiocodec übertragen werden soll. Dieser Teil der Platine ist in der Gesamtschaltung mit 2 markiert.

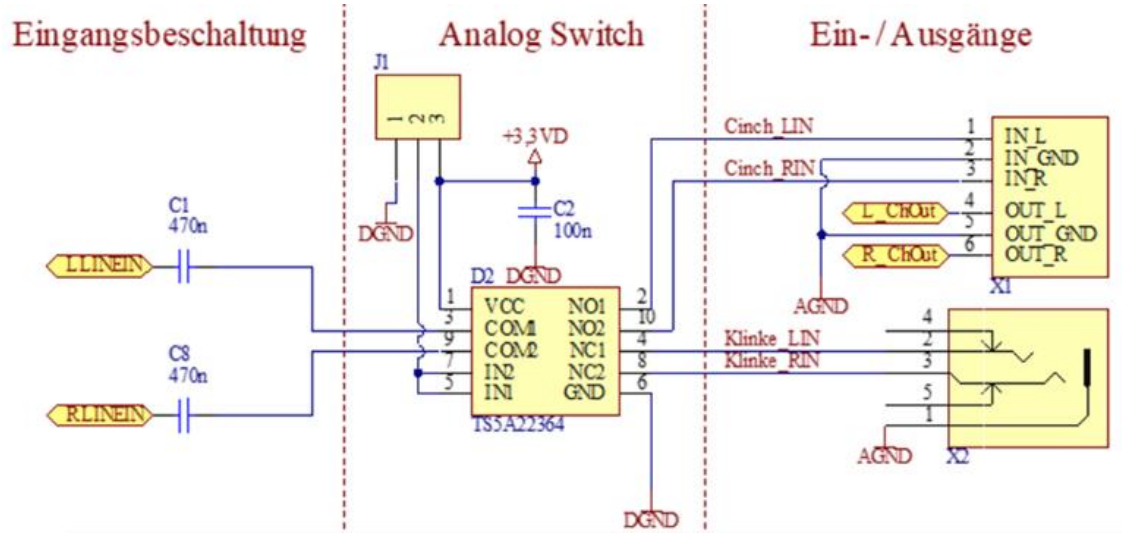



*Abbildung 12: Schaltung mit dem Audioswitch*

Der Audioswitch hat pro Eingang zwei Pins. Die beiden Pins werden für das linke und rechte Audiosignal genutzt. Der Eingang mit den Pins NO1 und NO2 ist mit den beiden Signalen der Cinchbuchse verbunden. Der Eingang mit den Pins NC1 und NC2 ist mit den beiden Signalen des Klinkensteckers verbunden. IN1 steuert die Auswahl zwischen NO1 und NC1. IN2 steuert die Auswahl zwischen NO2 und NC2. Die Pins IN1 und IN2 werden kurzgeschlossen, weil entweder NO1 und NO2 oder NC1 und NC2 ausgewählt werden soll. Für die Auswahl gibt es einen Jumper, mit dem die Leitungen entweder auf 3,3V oder auf Masse gezogen werden können.

Das folgende Pinning ist ein Datenblattauszug des TS5A22364. Aus diesem Pinning wurden die Informationen für die Entwicklung der Schaltung entnommen.

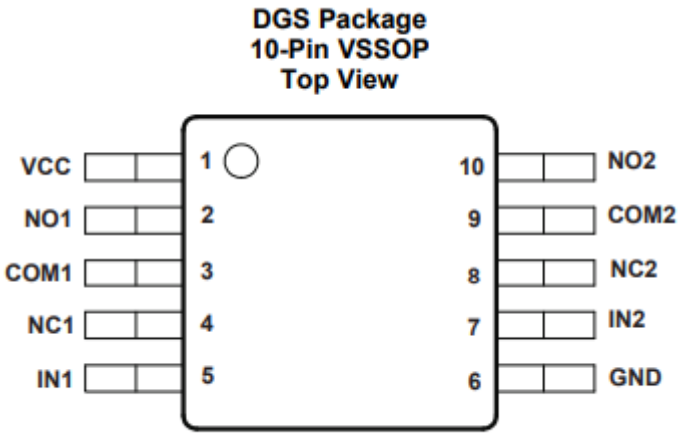



*Abbildung 13: Pinning des Audioswitchs [TI25c]*

**Pin Functions**

| PIN | | | TYPE | DESCRIPTION |
|---|---|---|---|---|
| NAME | DRC / DGS | YZP | | |
| VCC | 1 | A2 | — | Power Supply |
| NO1 | 2 | A3 | I/O | Normally Open (NO) signal path, Switch 1 |
| COM1 | 3 | B3 | I/O | Common signal path, Switch 1 |
| NC1 | 4 | C3 | I/O | Normally Closed (NC) signal path, Switch 1 |
| IN1 | 5 | D3 | I | Digital control pin to connect COM1 to NO1, Switch 1 |
| GND | 6 | D2 | — | Ground |
| IN2 | 7 | D1 | I | Digital control pin to connect COM2 to NO2, Switch 2 |
| NC2 | 8 | C1 | I/O | Normally Closed (NC) signal path, Switch 2 |
| COM2 | 9 | B1 | I/O | Common signal path, Switch 2 |
| NO2 | 10 | A1 | I/O | Normally Open (NO) signal Path, Switch 2 |

*Abbildung 14: Pinningtabelle des Audioswitch [TI25c]*

## 4.4.4 Taktgenerator 12MHz (ASEMB-12 000MHZ-LY-T)

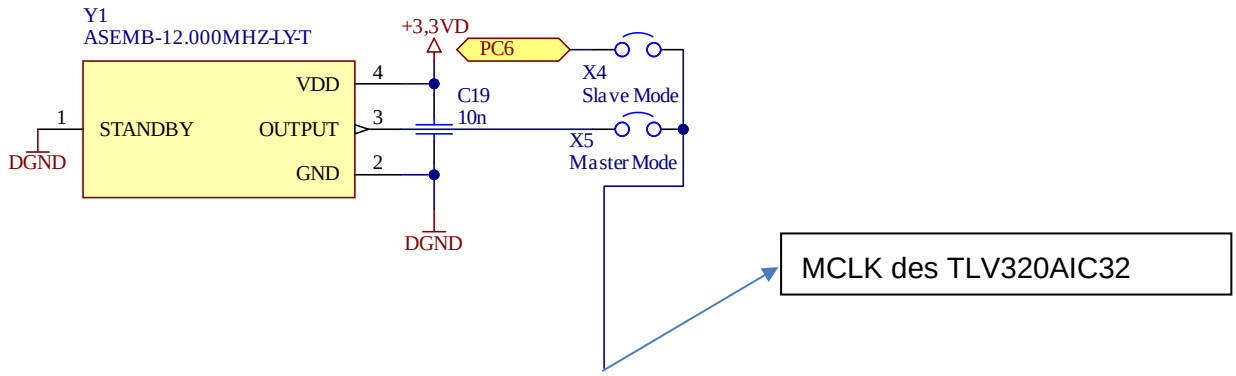



*Abbildung 15: Schaltung des Taktgenerators*

Der Oszillator AESMB-12.000MHZ-LY-T generiert ein 12MHz Signal. Dafür braucht er einen Stützkondensator von 10nF zwischen VDD und GND, damit kann er ein 12MHz Signal am OUTPUT-Pin erzeugen, welches auf die Stiftleiste X5 geht. Wenn ein Jumper auf diese Stiftleiste gesetzt wird, ist er im Master-Betrieb. Wenn die Stiftleiste X4 überbrückt wird, dann befindet er sich im Slave Betrieb und kriegt das MCLK-Signal vom MDDS-System. [MOU25d]

Dieser Teil der Platine ist in der Gesamtschaltung mit 5 markiert.

## 4.4.5 Beschaltung des TLV320AIC32

Für die Entwicklung der Beschaltung wurde sich an folgendem Pinning in Abbildung 16 und 17 orientiert. Weiters wurde sich an der Schaltung, die in Abbildung 6 mit 1 markiert ist, orientiert. Nach dem Pinning befindet sich eine Tabelle, welche zeigt, wie jeder Pin des Audiocodecs TLV320AIC32 verbunden ist. Außerdem zeigt sie, wie der Pin heißt, sowie notwendige Begründungen für eine solche Beschaltung des Audiocodecs TLV320AIC32. Dieser Teil ist in der Gesamtschaltung mit 6 markiert.

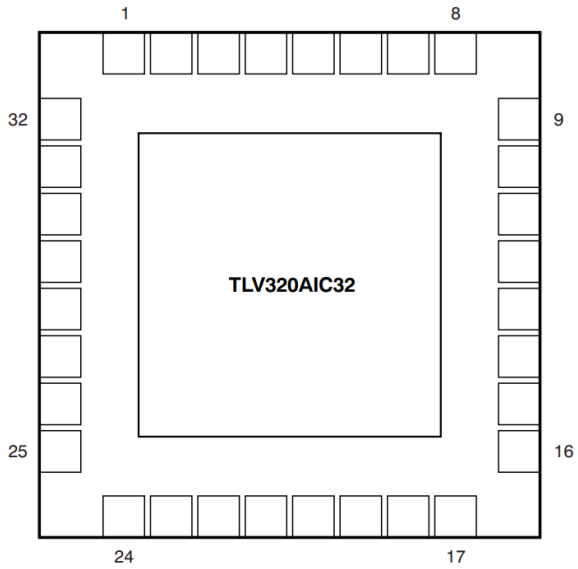



*Abbildung 16: Pinning des TLV320AIC32 [TI25b]*

| TERMINAL | | | DESCRIPTION |
|---|---|---|---|
| NAME | QFN NO. | I/O | |
| MCLK | 1 | I | Master clock input |
| BCLK | 2 | I/O | Audio serial data bus bit clock input/output |
| WCLK | 3 | I/O | Audio serial data bus word clock input/output |
| DIN | 4 | I | Audio serial data bus data input |
| DOUT | 5 | O | Audio serial data bus data output |
| DVSS | 6 | I/O | Digital core / I/O Ground Supply, 0 V |
| IOVDD | 7 | I/O | Digital I/O voltage supply, 1.1 V – 3.6 V |
| SCL | 8 | I/O | I2C serial clock input |
| SDA | 9 | I/O | I2C serial data input/output |
| MIC1L/LINE1L | 10 | I | Left input 1 |
| MIC1R/LINE1R | 11 | I | Right input 1 |
| MIC2L/LINE2L | 12 | I | Left input 2 |
| MIC2R/LINE2R | 13 | I | Right input 2 |
| MIC3L/LINE3L | 14 | I | Left input 3 |
| MICBIAS | 15 | O | Microphone bias voltage output |
| MIC3R/LINE3R | 16 | I | Right input 3 |
| AVSS1 | 17 | I | Analog ADC ground supply, 0 V |
| DRVDD | 18 | O | Analog ADC and output driver voltage supply, 2.7 V – 3.6 V |
| HPLOUT | 19 | O | High power output driver (left +) |
| HPLCOM | 20 | O | High power output driver (left - or multi-functional) |
| DRVSS | 21 | O | Analog output driver ground supply, 0 V |
| HPRCOM | 22 | O | High power output driver (right - or multi-functional) |
| HPROUT | 23 | O | High power output driver (right +) |
| DRVDD | 24 | O | Analog output driver voltage supply, 2.7 V – 3.6 V |
| AVDD | 25 | I | Analog DAC voltage supply, 2.7 V – 3.6 V |
| AVSS2 | 26 | I | Analog DAC ground supply, 0 V |
| LEFT_LOP | 27 | O | Left line output (+) |
| LEFT_LOM | 28 | O | Left line output (-) |
| RIGHT_LOP | 29 | O | Right lineo output (+) |
| RIGHT_LOM | 30 | O | Right line output (-) |
| $\overline{\text{RESET}}$ | 31 | | Reset |
| DVDD | 32 | I | Digital core voltage supply, 1.65 V – 1.95 V |

*Abbildung 17: Pinningtabelle des TLV320AIC32 [TI25b]*

| Anschlusstabelle | | |
|---|---|---|
| Pin | Name | Verbunden mit |
| 1 | MCLK | PC6 vom MDDS-System |
| 2 | BCLK | PC10, PB13 vom MDDS-System |
| 3 | WCLK | PA4 vom MDDS-System |
| 4 | DIN | PC12 vom MDDS-System |
| 5 | DOUT | PB15 vom MDDS-System |
| 6 | DVSS | DGND wie in Abbildung 3 |
| 7 | IOVDD | Digitale 3,3V mit Stützkondensatoren wie in Abbildung 2 |
| 8 | SCL | PB6 vom MDDS-System |
| 9 | SDA | PB7 vom MDDS-System |
| 10 | MIC1L/LINE1L | COM1-Ausgang des Audio-Switchs |
| 11 | MIC1R/LINE1R | COM2-Ausgang des Audio-Switchs |
| 12 | MIC2L/LINE2L | AGND, weil Eingang nicht benützt wird. |
| 13 | MIC2R/LINE2R | AGND, weil Eingang nicht benützt wird. |
| 14 | MIC3L/LINE3L | AGND, weil Eingang nicht benützt wird. |
| 15 | MICBIAS | nC, weil kein passives Mikrofon benutzt wird. |
| 16 | MIC3R/LINE3R | AGND, weil Eingang nicht benützt wird. |
| 17 | AVSS1(GND des analog-digital-Converters) | AGND wie in Abbildung 3 |
| 18 | DRVDD | Analoge 3,3V mit Stützkondensatoren wie in Abbildung 3 |
| 19 | HPLOUT | Linker Channel des Klinkensteckers bzw. der Chinch-Buchse mit Ausgangsbeschaltung wie in V4 |
| 20 | HPLCOM | Mit GND des Klinkensteckers verbunden |
| 21 | DRVSS | AGND wie in Abbildung 3 |
| 22 | HPRCOM | Mit GND des Klinkensteckers verbunden |
| 23 | HPROUT | Rechter Channel des Klinkensteckers bzw. der Chinch-Buchse mit Ausgangsbeschaltung wie in Abbildung 2 |
| 24 | DRVDD | Analoge 3,3V mit Stützkondensatoren wie in Abbildung 3 |
| 25 | AVDD | Analoge 3,3V mit Stützkondensatoren wie in Abbildung 3 |
| 26 | AVSS2( GND des Digital-Analog-Converters) | AGND wie in Abbildung 3 |
| 27 | LEFT-LOP | nC, weil dieser Ausgang nicht benützt wird. |
| 28 | LEFT-LOM | nC, weil dieser Ausgang nicht benützt wird. |
| 29 | RIGHT-LOP | nC, weil dieser Ausgang nicht benützt wird. |
| 30 | RIGHT-LOM | nC, weil dieser Ausgang nicht benützt wird. |
| 31 | RESET-N | 10kOhm Pullup, was bedeutet, dass der Reset nicht betätigt werden kann. |
| 32 | DVDD | 1,8V vom Buck-Konverter mit Stützkondensatoren wie in Abbildung 3 |
| 33 | GND-Platte | AGND wie in Abbildung 3 |

## 4.4.6 Gesamtschaltung

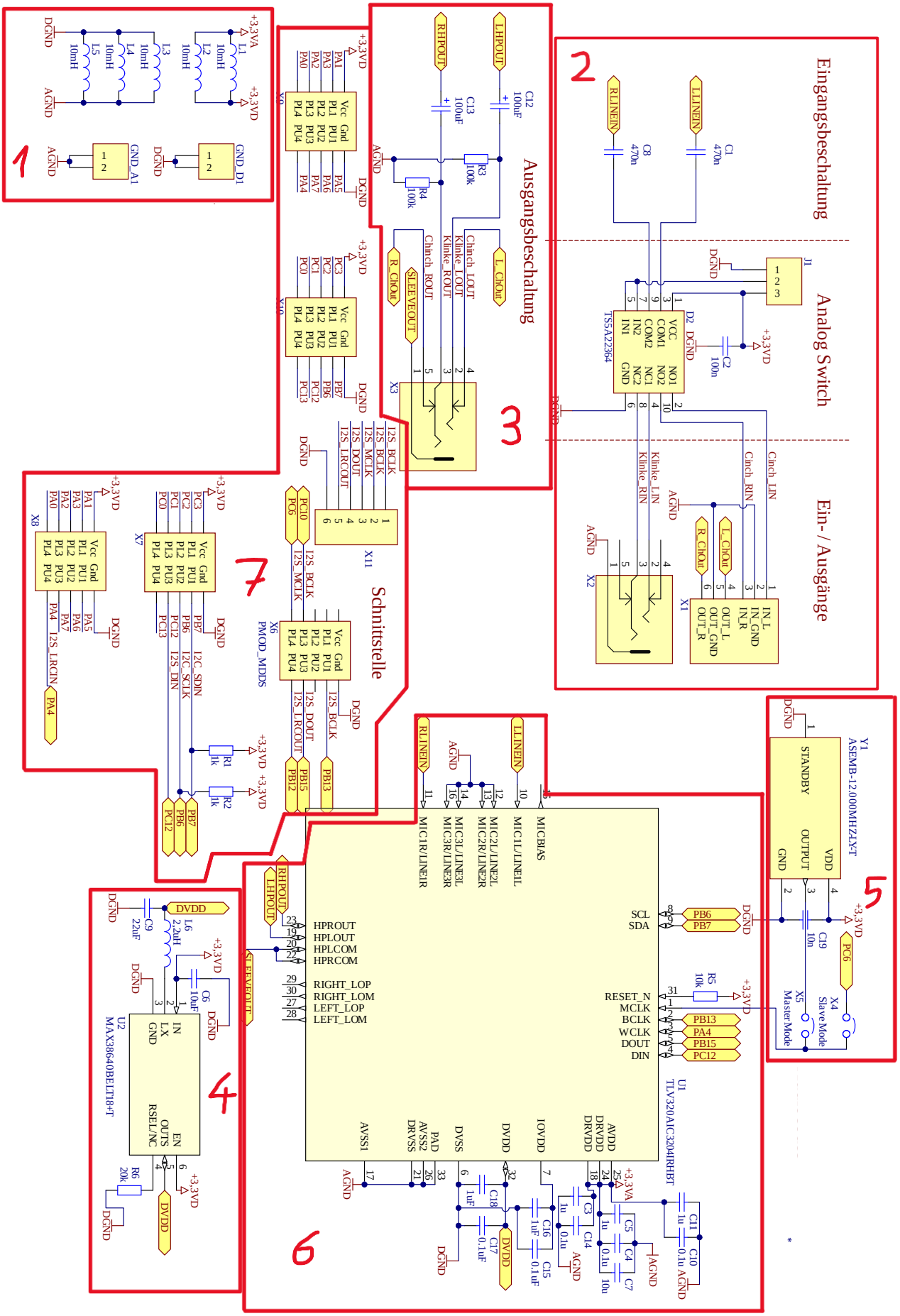



*Abbildung 18: Schemetic der Platine V5.0.0*

## 4.5 Layout

Beim Layout wurde sich an dem der V4 orientiert, welche von einer alten Diplomarbeit „GLADOS – Guided Low Altitude Detection and Observation System“ designt wurde. Die Größe der Platine und die Position der Stiftleisten, Klinkenbuchsen und Chinch-Buchsen wurden übernommen. Außerdem wurde, wie auch bei der V4, eine 4-Layerplatine mit Split-Planes verwendet. Das bedeutet, dass die Platine oben und unten einen Signal-Layer hat und dazwischen eine GND-Plane und eine VCC-Plane ist. Diese Planes sind jeweils in analoge und digitale Masse aufgeteilt, wie in 4.3.1 erklärt.

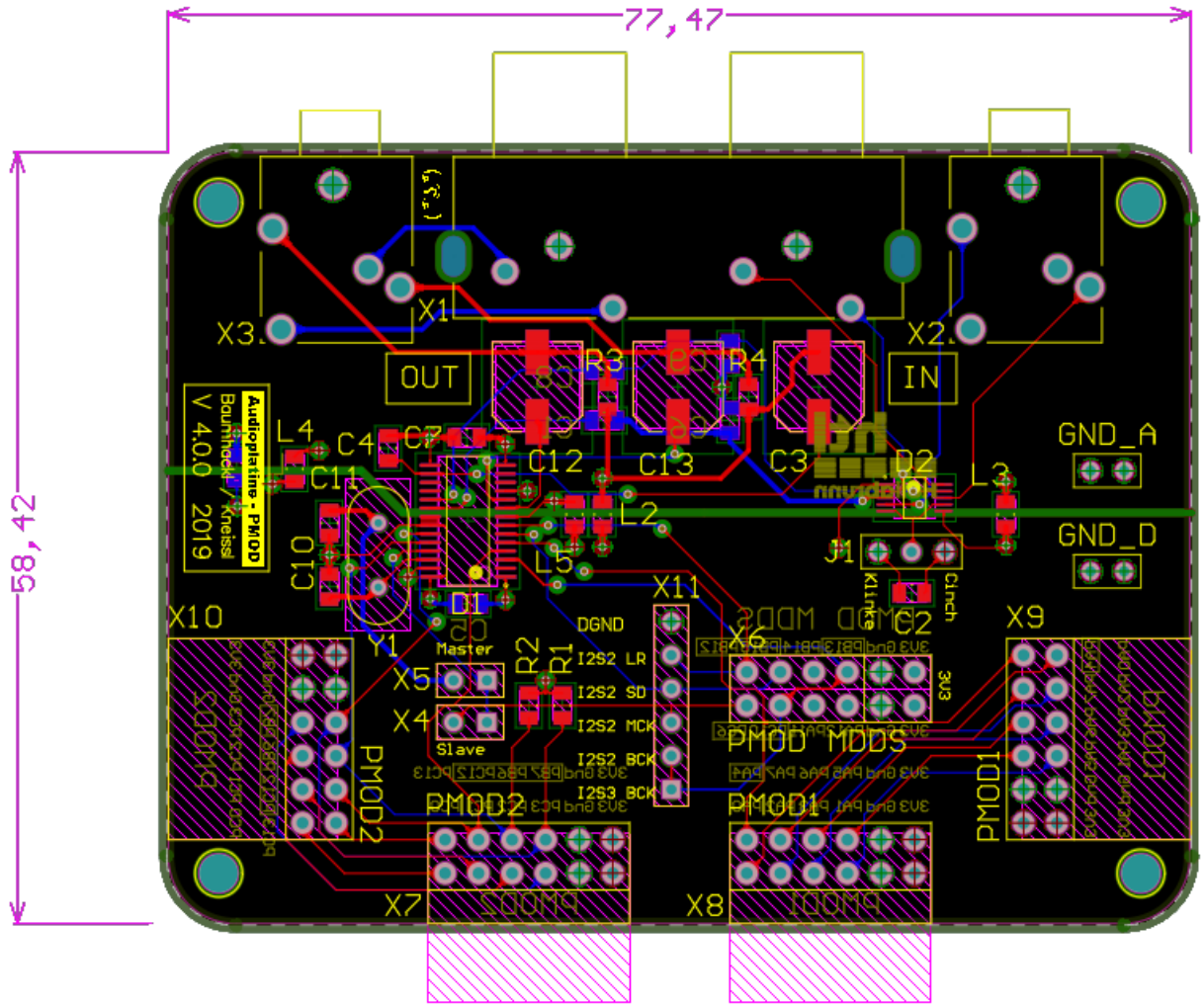



*Abbildung 19: Layout der Platine V4 [GLA25]*

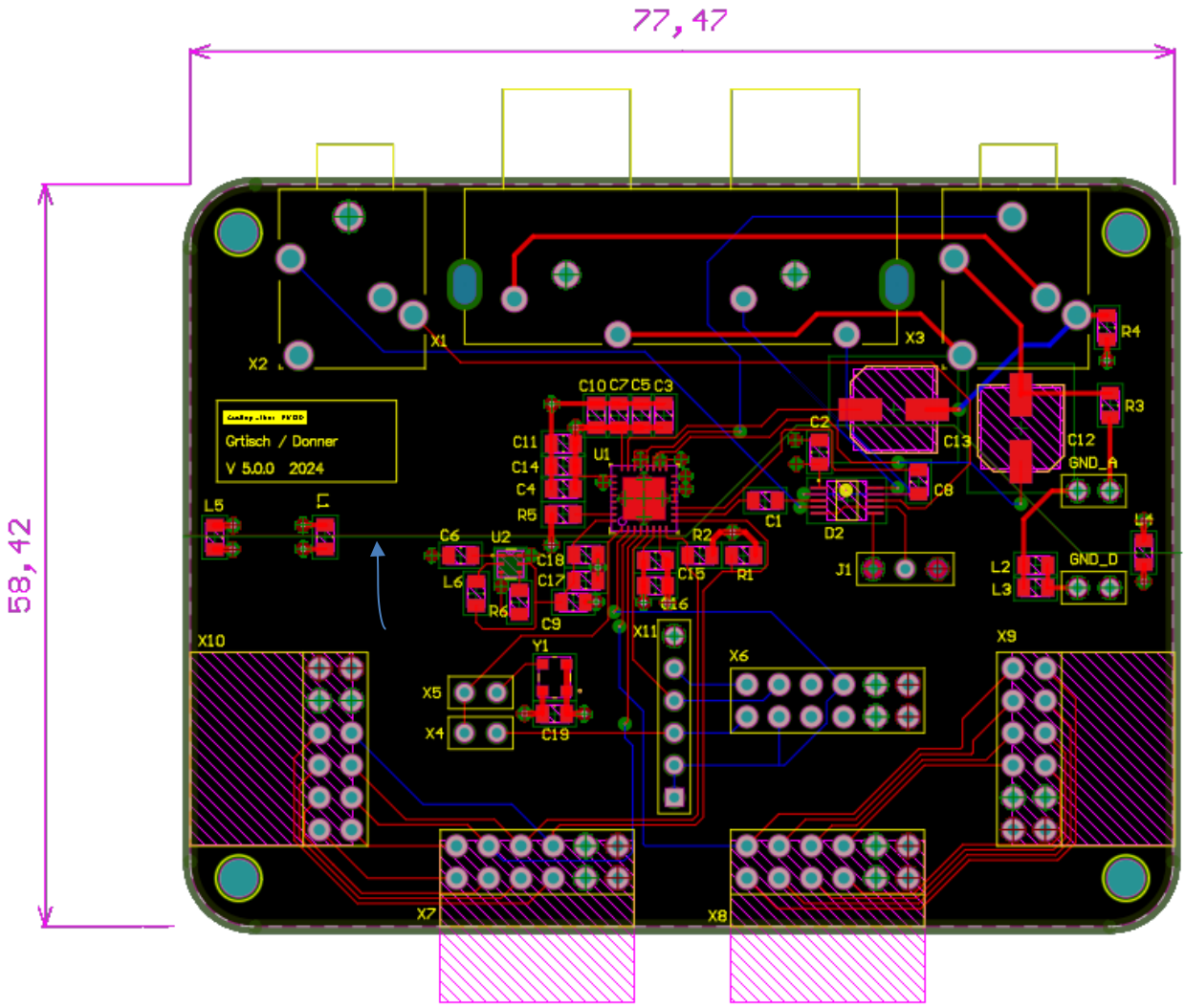



*Abbildung 20: Layout der Platine V5.0.0*

Das Design der alten Diplomarbeit V4 hat die exakt selben Maße wie die V5. Die grüne Linie, die in Abbildung 20 mit einem Pfeil markiert ist, zeigt in beiden die Trennung der analogen und digitalen Masse. Auf der exakt selben Position, wie die grüne Linie, liegt eine Weitere, die die Trennung zwischen analoger und digitaler Versorgung darstellt. Die Linie wäre braun, ist aber nicht sichtbar, weil sie bei dieser Platine von der grünen Linie bedeckt wird. Für mehr Informationen über die analoge und digitale Masse siehe Kapitel 4.3.1.

## 4.6 Fehlersuche

### 4.6.1 Messungen

Die Platine wurde in der HTL bestückt und dann wurden die einzelnen Pins unter dem Mikroskop betrachtet und mit dem Ohmmeter gemessen, um herauszufinden, ob es sich um einen Kurzschluss handelt. Jedoch wurde keiner gefunden. Danach wurde die fertig gelötete Platine an das MDDS-System angeschlossen.

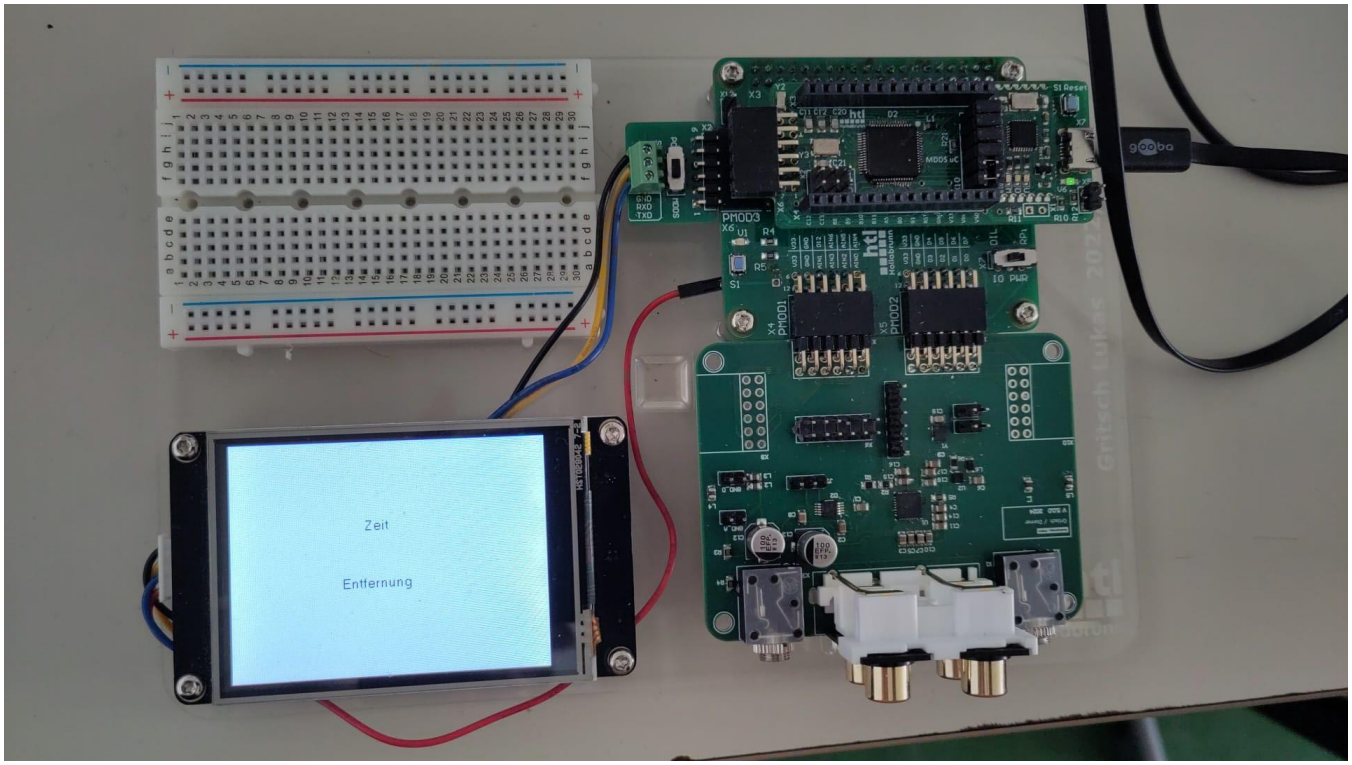



*Abbildung 21: Platine an MDDS-System angeschlossen*

Als Erstes wurde die Versorgungsspannung nachgemessen, um sicher zu stellen, dass die Audioplatine richtig versorgt wird und es keinen Kurzschluss gibt.




*Abbildung 22: Messung der Versorgung mit TLV320AIC32*

Hier ist erkennbar, dass die Versorgung die ganze Zeit ein- und ausschaltet, weil die Audioplatine offensichtlich zu viel Strom verbraucht.
Deswegen wurde der nächste Versuch mit einem externen Netzteil, welches die Audioplatine mit 3,3V versorgt, gemacht. Die Strombegrenzung wurde bis zu 2,4A erhöht, wo das Netzgerät dann 3,3V liefern konnte. Daraus wurde geschlossen, dass irgendwo ein Kurzschluss ist, weil die Audioplatine auf keinen Fall eine Leistung von 8W brauchen sollte.

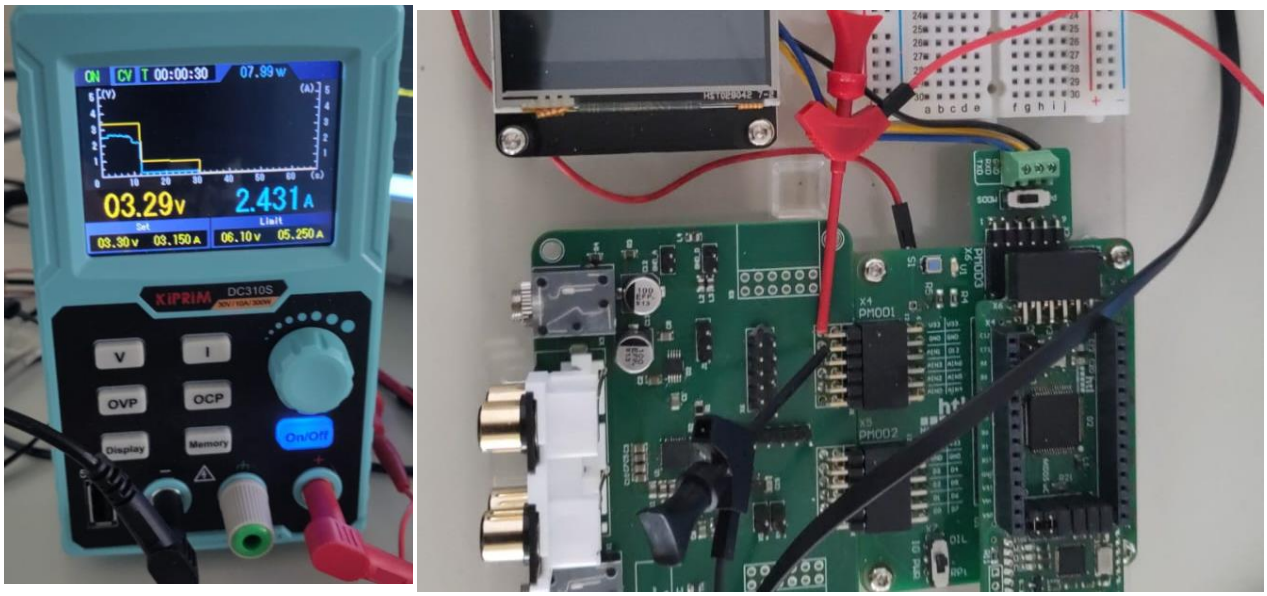



*Abbildung 23: Aufbau mit externem Netzgerät*

In weiterer Folge wurde die Audioplatine mit der Wärmebildkamera gemessen. Die Messung hat ergeben, dass mit steigender Strombegrenzung nur die Temperatur des Audiocodecs ansteigt.

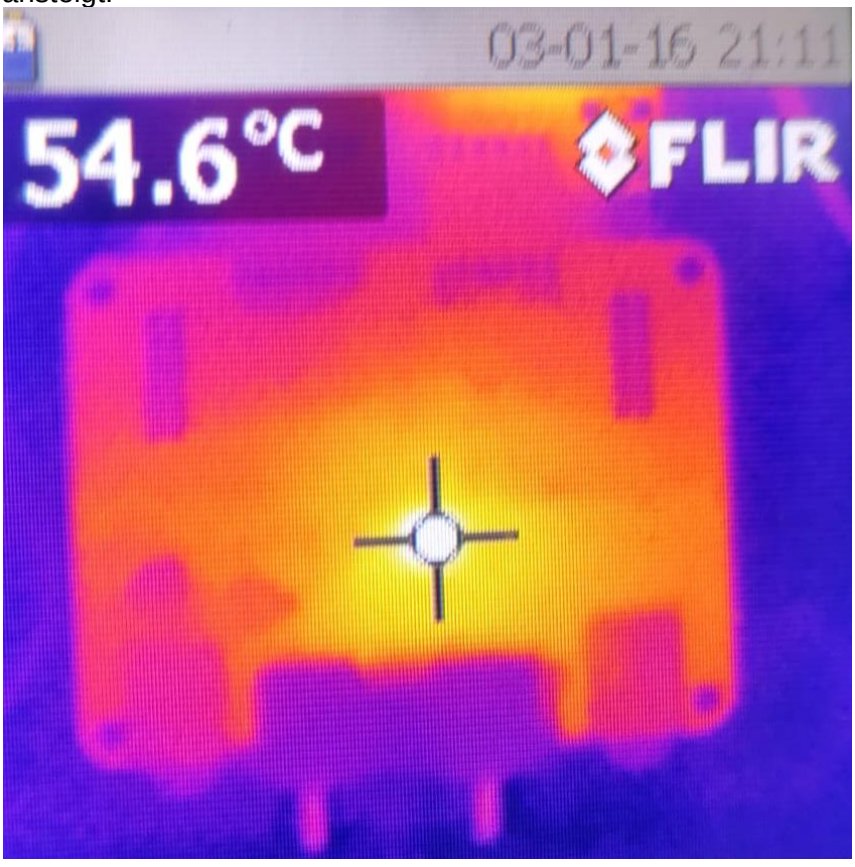



*Abbildung 24: Wärmebildaufnahme der Platine mit 100mA Strombegrenzung*




*Abbildung 25: Wärmebildaufnahme bei 2,4A*

Weiters wurde mit einem Ohmmeter zwischen Versorgung und Masse gemessen, um herauszufinden, ob es ein ohmscher Kurzschluss ist.




Der gemessene Widerstand beträgt 1,34kOhm. Bei einer Spannung von 3,3V, sollte also ein Strom von $I = \frac{U}{R} = \frac{3{,}3V}{1340\Omega} = 2{,}46mA$ fließen. Jedoch fließt ein Strom von über 2,4A, was bedeutet, dass es kein ohmscher Kurzschluss ist.

*Abbildung 26: Ohmmeter zwischen Versorgung und Masse gemessen*

## 4.6.2 Erkenntnisse

Da es kein ohmscher Kurzschluss ist, dürfte es ein Beschaltungsfehler des Audiocodec sein. Weil kein offensichtlicher Fehler in der Schaltung festgestellt werden konnte, wurde beschlossen, eine Platine mit einem weniger komplexen Audiocodec anzufertigen, welcher mehr dem der alten Diplomarbeit entspricht. Die Entwicklung der Platine hat im Sommer 2024 begonnen und die Fehler wurden im November 2024 festgestellt.
Auf der Texas Instruments Webseite sind kompatible Audiocodecs vorgeschlagen. Der verwendete Audiocodec war jedoch keine Empfehlung, deswegen sollte in einer neuen Audioplatine einer der kompatiblen Audiocodecs verwendet werden.

# 5 Audioboard mit Audio-Codec TAC5142

Diese Platine ist die Version 5.1.1 und wurde mit dem Audiocodec TAC5142 entwickelt. Das Ziel dieser Platine ist es, eine Platine zu entwickeln, welche möglichst genau der alten Diplomarbeit entspricht. Dies reduziert die Fehlerquellen auf ein Minimum. Deswegen wurde der TAC5142 eingesetzt, welcher dem TLV320AIC23B bis auf paar Details im Pin-out und der Funktion entspricht. Weiters wurde die Schaltung aus einer Kombination der alten Diplomarbeit „GLADOS – Guided Low Altitude Detection and Observation System“ und der Beispielschaltung in 4.2 aufgebaut. Zusätzlich wurde noch ein Breakoutboard entwickelt, welches die Möglichkeit eröffnen soll, Schaltungsänderungen auf einem Steckbrett auszutesten.

## 5.1 Einführung in den TAC5142

Der TAC5142 ist ein Hardware-gesteuerter Stereo-Audio-Codec mit 2 ADC- und 2 DAC-Channels und zwei analogen Ein- und Ausgängen. Der Audiocodec unterstützt das I2S, sowie andere Audioverarbeitungsprotokolle. Außerdem hat dieser Audiocodec Features wie Low Power, „Mic bias“ und integrierten PLL. [TI25a]

## 5.2 Beispielschaltung

Die Beispielschaltung wurde dem Datenblatt des TAC5142 entnommen, sie stellt eine typische Anwendung des TAC5142 mit zwei AC-gekoppelten, passiven Mikrofonen und zwei Ausgängen, die zu einem Audioverstärker geführt sind, dar. Diese Beispielschaltung verwendet eine Konfiguration für das I2S-Protokoll und eine 3,3V Versorgung, optional kann auch eine 1,8V Versorgung verwendet werden. [TI25a]

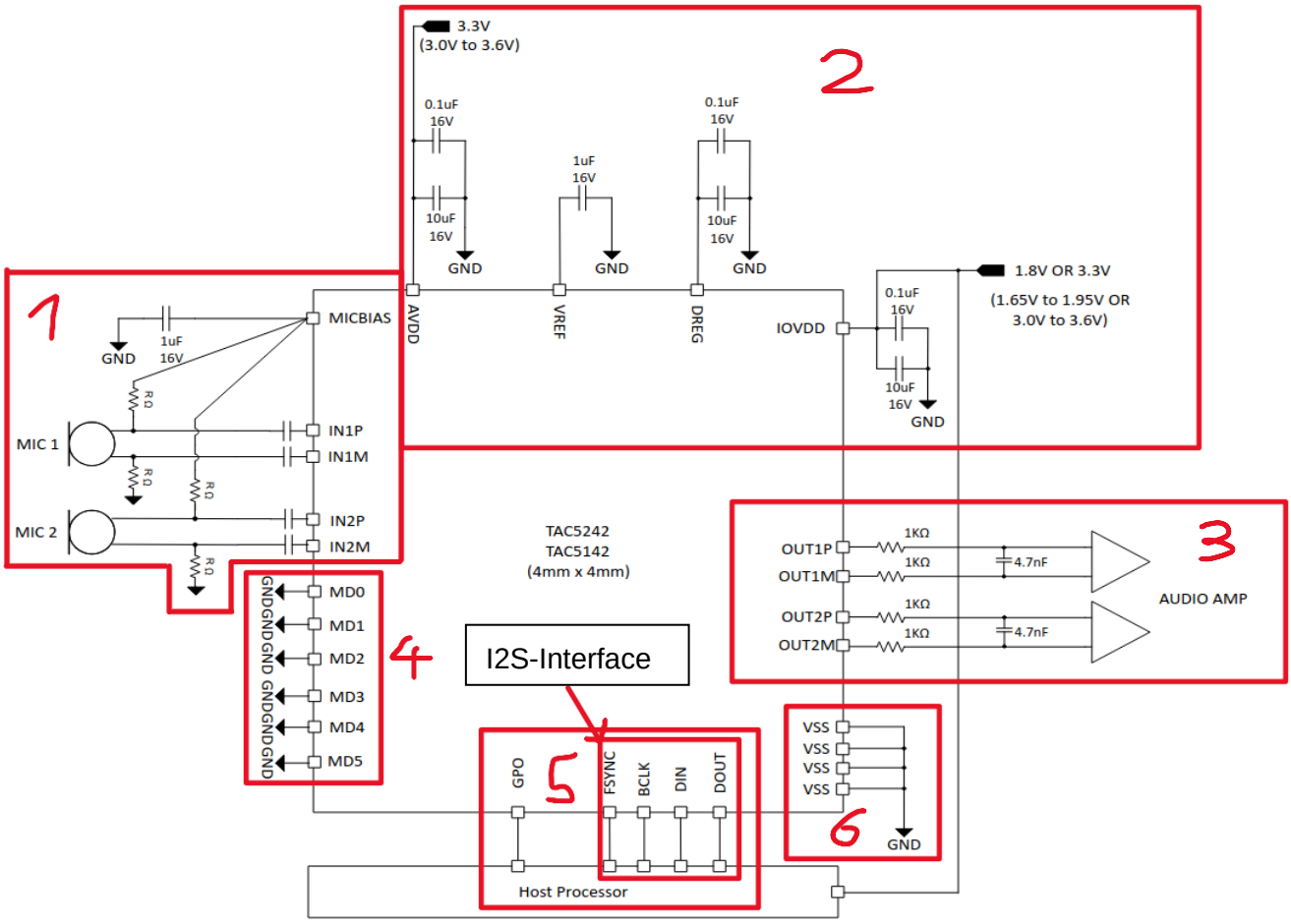



*Abbildung 27: Beispielschaltung für den TAC5142 [TI25a]*

| Schaltungsteil | Funktion |
|---|---|
| 1 | Eingangsbeschaltung mit 2 passiven Mikrofonen |
| 2 | Korrekte Versorgung des Audiocodec TAC5142 |
| 3 | Ausgangsbeschaltung für zwei Audioverstärker |
| 4 | Konfigurationspins für eine Konfiguration des I2S-Target-Modes |
| 5 | Kommunikation mit dem µC |
| 6 | Verbindung der GND-Pads |

Aus der Schaltung wurde der Teil 2,5 und 6 übernommen.

Die zweite Beispielschaltung, an der ich mich orientiert habe, ist die Audioplatine V4, welche im Kapitel 4.3 unter Abbildung 5 zu finden ist.

# 5.3 Schaltung

## 5.3.1 Audio-Switch TS5A22364

Die Beschaltung des Audio-Switch ist wie in Kapitel 4.4.3

## 5.3.2 Beschaltung des TAC5142

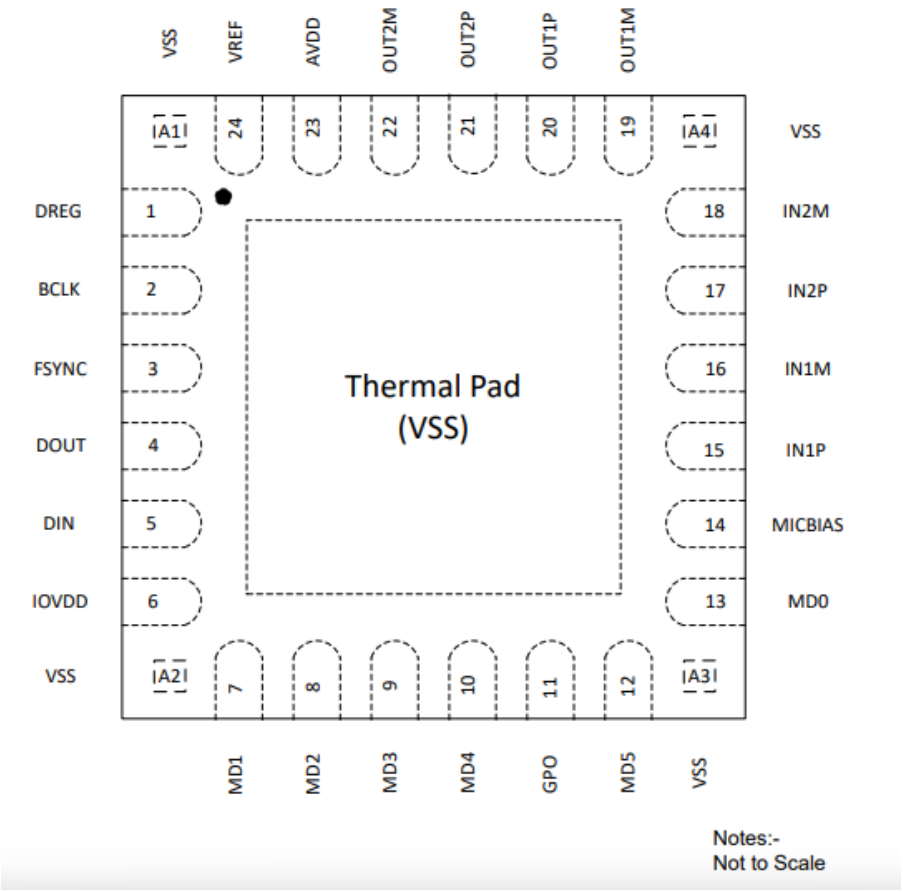



*Abbildung 28: Pinning des Audiocodec TAC5142 [TI25a]*

| Pin | Beschreibung |
|---|---|
| VSS | GND-Planes direkt mit GND verbunden. |
| DREG, VREF | Wie in der Beispielschaltung mit GND verbunden (Abbildung 27). |
| BCLK, FSYNC, DOUT, DIN | Pins für I2S Kommunikation (BCLK, WS, DOUT, DIN) (Abbildung 27). |
| IOVDD, AVDD | Wie in der Beispielschaltung mit der 3,3V Spannungsversorgung verbunden (Abbildung 27). |
| MDO, MD1, MD2, MD3, MD4, MD5 | Konfigurationspins, für die derzeitige Konfiguration siehe Kapitel 2.3.6.1. |
| GPO | Der Audiocodec übermittelt über diese Leitung dem µC, wenn die Clock gestoppt wurde oder ein Clock-Error aufgetreten ist. |
| MICBAIS | Der Ausgang kann passive Mikrofone mit 2,75V versorgen. |
| IN1P, IN1M, IN2P, IN2M | Zwei Audioeingänge mit Plus und Minus |

| | |
|---|---|
| | sind als Single-ended-input connection genutzt (Abbildung 27). |
| OUT1P, OUT1M, OUT2P, OUT2M | Zwei Audioausgänge, genutzt als Pseudo differential- Output Connection (Abbildung 30). |

Für den Audiocodec TAC5142 wurde ein VQFN-Gehäuse verwendet.

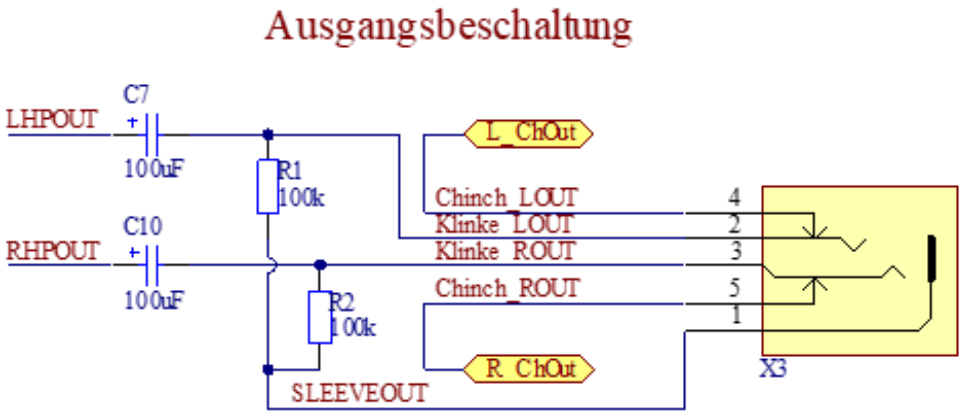



*Abbildung 29: Ausgangsbeschaltung des TAC5142*

In der Ausgangsbeschaltung der alten Diplomarbeit „GLADOS – Guided Low Altitude Detection and Observation System“ wurde ein Single-Ended-Output verwendet, welcher nur das Ausgangssignal und die Masse überträgt. Für eine bessere Rauschunterdrückung wurden zwei Änderungen vorgenommen, um die Ausgangsbeschaltung auf die Schaltung in Abbildung 30 unter dem Namen Pseudo-Differential-Output anzupassen. Dies bedeutet, dass die Masse des Klinkensteckers oder der Cinchbuchse nicht mit GND verbunden ist, sondern mit einem „Sleeveout“. Dieses „Sleeveout“ besteht aus den Minus-Kanälen von zwei der Ausgänge des Audiocodec. Differenziell bedeutet, wenn nicht nur das Signal übertragen wird, sondern auch das genau negierte Signal, wovon die Differenz dem doppelten Signal entspricht. Mit dieser Methode können Gleichtaktstörungen eliminiert werden. Ein ähnliches Konzept verfolgt ein Pseudo-Differential Output. Dabei wird nicht genau das negierte Signal übertragen, sondern zum Beispiel die Hälfte des eigentlichen Signals. Dies resultiert in einer Rauschunterdrückung und reduziert Masseschleifen. Im Gegensatz zu einem voll differentiellen Ausgang, kann er statt Single-Ended Ausgängen verwendet werden, weil kein differentieller Empfänger benötigt wird.

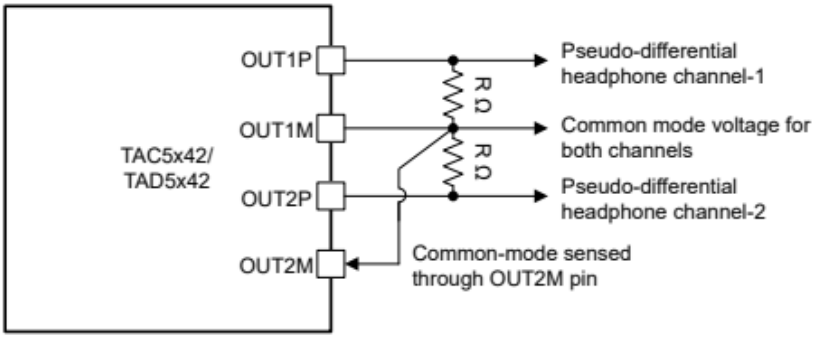



**Figure 7-12. Pseudo-differential Output Connection with External Common-Mode Sense**

*Abbildung 30: Pseudo-differential Output des TAC5142 [TI25a]*

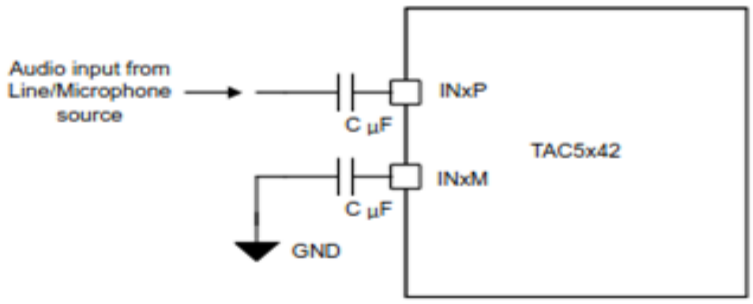



*Abbildung 31: Single-Ended Eingang des TAC5142 [TI25a]*

Der Eingang ist ein AC-gekoppelter Eingang, wie bei der alten Diplomarbeit „GLADOS – Guided Low Altitude Detection and Observation System“ und in der Abbildung 31.

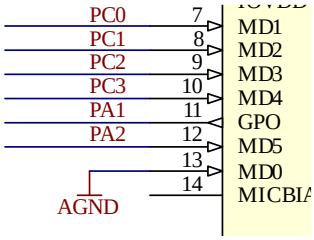



*Abbildung 32: Konfigurationspins des TAC5142*

Der Konfigurations-Pin MD0 ist fix mit GND verbunden für den I2S-Target-Mode und die Pins MD1-MD5 und der GPO-Pin sind mit GPIO-Pins der µC verbunden. Im Kapitel 5.5.1 wird die Funktion dieser Pins genauer erläutert.

Diese Platine ist für einen halbduplexen I2S-Betrieb ausgelegt, das heißt, dass eine I2S-Schnittstelle des µC empfängt und die zweite nur liest. In diesem Aufbau gibt es einen Master und zwei Slaves. Ein Slave ist der Audiocodec und der andere ist eine der I2S-Schnittstellen des µC. Das bedeutet, dass die BCLK und WS der beiden Schnittstellen des µC nicht einzeln zum Audiocodec geführt werden, sondern kurzgeschlossen werden.

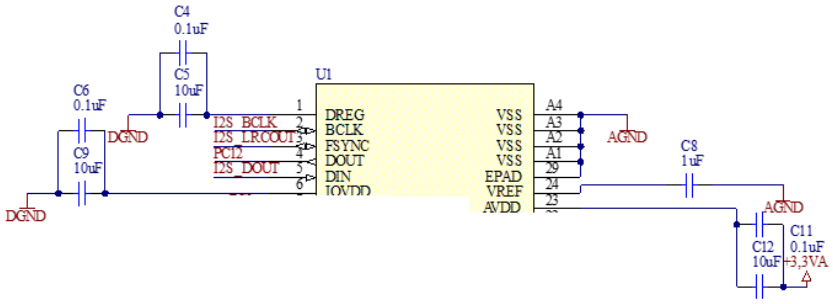



*Abbildung 33: Versorgung des TAC5142*

Die Versorgungen und Referenzspannungen sind mit Stützkondensatoren der Beispielschaltung aus Abbildung 33 verbunden. Diese Ports sind DREG, IOVDD, VREF und AVDD.

## 5.3.3 Gesamtschaltung

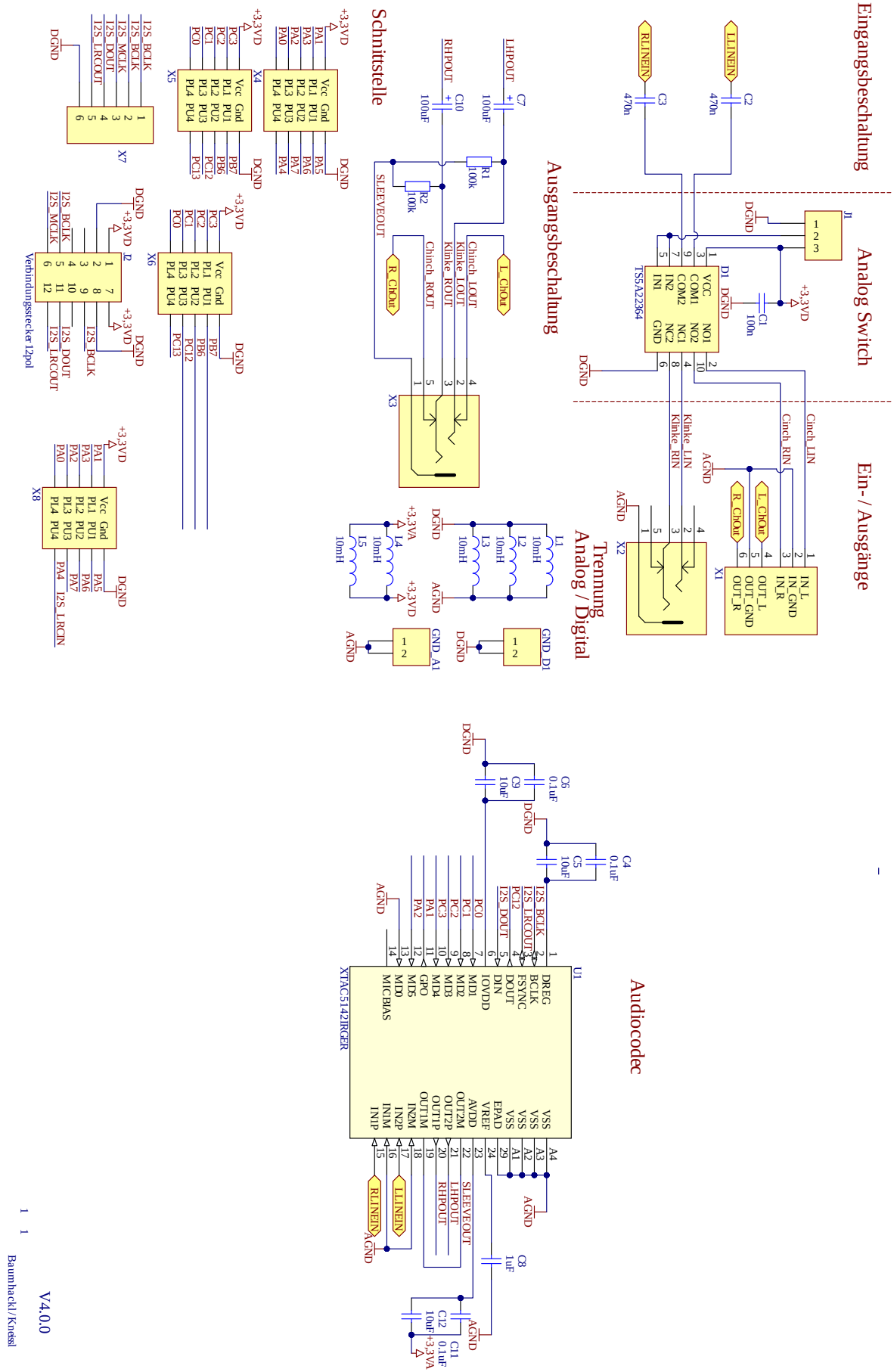



*Abbildung 34: Schaltung der Platine V5.1.1*

## 5.4 Layout

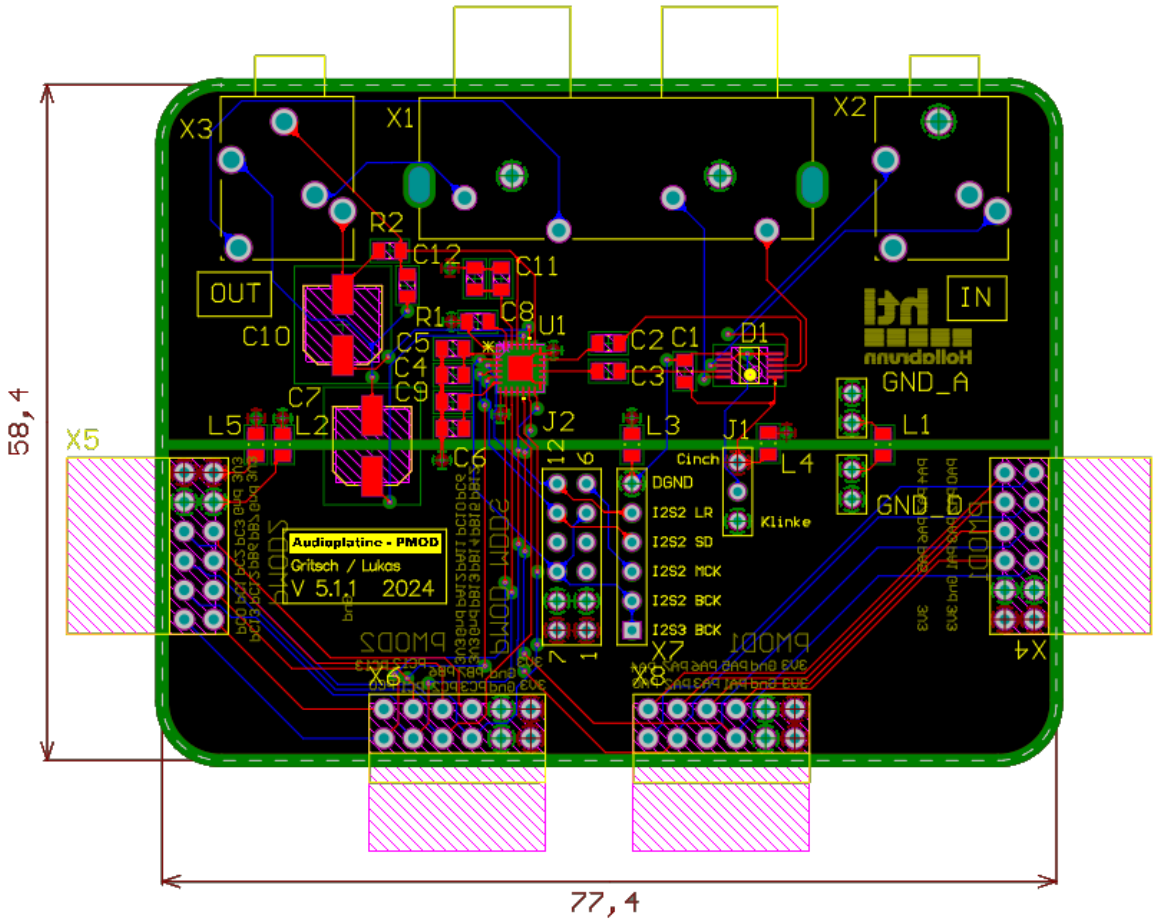



*Abbildung 35: Layout der V5.1.1*

Dieses Layout folgt demselben Schema, wie in 4.5 beschrieben.

## 5.5 Demo Software

### 5.5.1 Konfigurations-Pins

Die Pins MD0 bis MD5 sind zum Konfigurieren des Audiocodecs. Die Abbildung 36 zeigt einen Überblick über die Konfigurations-Möglichkeiten. Die Abbildungen 37,38,39 und 40 zeigen die Konfigurationsmöglichkeiten im Detail.

| PIN | TARGET MODE | CONTROLLER MODE |
|---|---|---|
| MD0 | Multi-level analog input for controller/target mode and I²S/TDM/LJ mode selection | |
| MD1 | AVDD supply, word length, and decimation/ interpolation filter type selection | Frame rate and BCLK frequency selection |
| MD2 | | |
| MD3 | I²S/LJ Mode: Digital HPF cut-off frequency and input cap quick charge selection for the ADC<br>TDM Mode: Data slot selection | Controller clock input |
| MD4 | ADC input configuration (Differential/Single-ended, AC/DC Coupled) and DAC output configuration (Differential Line-out/ Differential Receiver or Headphone/ Single-ended Line-out/ Pseudo-differential Headphone) | |
| MD5 | | |

*Abbildung 36: Konfigurationspins im Überblick[TI25a]*

| MD0 | CONTROLLER AND TARGET SELECTION |
|---|---|
| Short to Ground | Target $I^2S$ Mode |
| Short to Ground with 4.7K Ohms | Target TDM Mode |
| Short to AVDD | Controller $I^2S$ Mode |
| Short to AVDD with 4.7K Ohms | Controller TDM Mode |
| Short to AVDD with 22K Ohms | Target LJ Mode |

*Abbildung 37: Konfiguration des MD0-Pins [TI25a]*

| MD2 | MD1 | WORD LENGTH, SUPPLY MODE, AND DECIMATION/ INTERPOLATION FILTER SELECTION (Valid for Target Mode only) |
|---|---|---|
| Low | Low | AVDD=3.3V, Word Length=32, Linear-phase Filter |
| Low | High | AVDD=1.8V, Word Length=32, Linear-phase Filter |
| High | Low | AVDD=3.3V, Word Length=24, Linear-phase Filter |
| High | High | AVDD=3.3V, Word Length=32, Low-latency phase Filter |

*Abbildung 38: Konfiguration von MD1 und MD2 [TI25a]*

Für das Testen der Platine ist noch nicht entscheidend, ob der Linear-phase Filter oder der Low-latency phase Filter genutzt wird. Deswegen sind beide der markierten Zeilen valide Konfigurationen für diesen Aufbau.

**Table 7-11. Input Cap Quick-Charge and HPF HPF Cutoff Frequency Selections for ADC in Target $I^2S$/LJ Mode**

| MD3 | INPUT CAP QUICK CHARGE | HPF –3dB CUT-OFF FREQUENCY | HPF –3dB CUTT-OFF FREQUENCY @ 48 kHz SAMPLE RATE |
|---|---|---|---|
| Low | Disabled (50ms time) | 0.000021 × $f_S$ | 1Hz |
| High | Enabled (12.5ms time) | 0.00025 × $f_S$ | 12Hz |

*Abbildung 39: Konfiguration des MD3-Pins [TI25a]*

**Table 7-10. Analog Input and Output Configurations**

| MD5 | MD4 | ANALOG INPUT CONFIGURATION | ANALOG OUTPUT CONFIGURATION |
|---|---|---|---|
| Low | Low | Differential input; AC-Coupled only | Differential Output; Line-out only |
| Low | High | Differential input; AC or DC-Coupled with High Common Mode Tolerance | Differential Output; Receiver/Headphone load or Line-out |
| High | Low | Single Ended Input on INxP; AC-Coupled only | Single-ended output; Line-out only |
| High | High | Single Ended Input on INxP; AC or DC-Coupled with High Common Mode Tolerance | Pseudo-differential output with external common-mode sense; Headphone load only |

*Abbildung 40: Konfiguration von MD4 und MD5 [TI25a]*

Die markierten Konfigurationen sind die, die für die Demosoftware verwendet wurden.

Insgesamt soll diese Konfiguration den Audiocodec zu einem I2S-Slave machen, der mit einer Word Length von 32bit und 3,3V arbeitet. Zusätzlich ist ein 12Hz Hochpass konfiguriert. Dieser schneidet den Gleichanteil und die nicht hörbare Sub der Audio weg. Die Ein- und Ausgangsbeschaltung ist für einen Single Ended Input und Pseudo Differential Output konfiguriert.

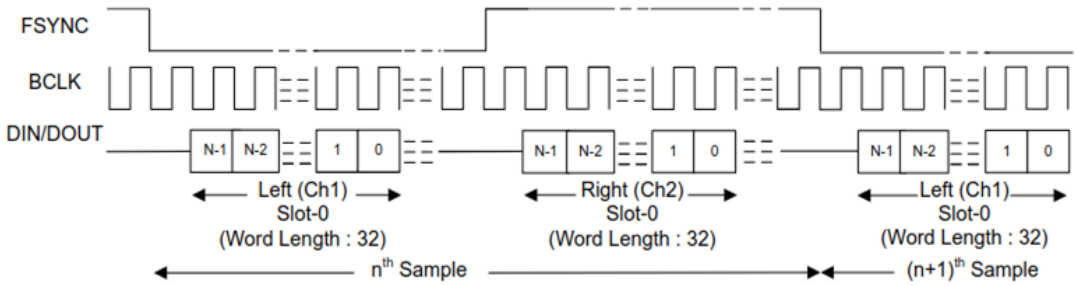



**Figure 7-3. I²S Mode Protocol Timing (MD0 shorted to ground) in Target Mode**

*Abbildung 41: I2S-Protokoll des Audiocodecs TAC5142 [TI25a]*

Hier sieht man, wie sich das I2S-Protokol des Audiocodec verhält. Dieses Verhalten entspricht genau dem des µC STM32F107T6 (siehe Kapitel 3.1).

## 5.5.2 Demosoftware

Für das Demoprogramm wird der µC STM32F107T6 benützt. Es arbeitet im Vergleich zum derzeitigen Programm mit halbduplex Master-Recieve (I2S2) und halbduplex Slave-Recieve (I2S3) anstatt mit vollduplexem I2S-Protokoll. Der Unterschied zu einem Full-Duplex-Master ist der, dass Half-Duplex nur senden oder empfangen kann, wodurch beide I2S Schnittstellen genutzt werden. Das I2S-Protokoll wird mittels DMA eingelesen und ausgegeben, was in Kapitel 7 weiter nachgelesen werden kann. Das Demoprogramm kann eine 1:1-Verbindung erzeugen, sowie verschiedene Signalformen, wie im jetzigen Programm „Audioverarbeitungssoftware V2“, welches in Kapitel 7 beschrieben wird. Weiters wird die Konfiguration der Pins MD1-MD5, wie im Kapitel 5.5.1. beschrieben, durchgeführt. Allgemein orientiert sich das Programm an dem jetzigen Programm „Audioverarbeitungssoftware V2“, welches in Kapitel 5 nachgelesen werden kann. Außerdem wird der GPO-Pin als Input definiert, hat aber keine Funktion im Programm.

### ***5.5.2.1*** *CubeMX-Konfiguration*

| | |
|---|---|
| GPO | PA1 |
| MD1 | PC0 |
| MD2 | PC1 |
| MD3 | PC2 |
| MD4 | PC3 |
| MD5 | PA2 |

Hier sieht man die Konfiguration von MD1-MD5 und dem GPO-Pin.

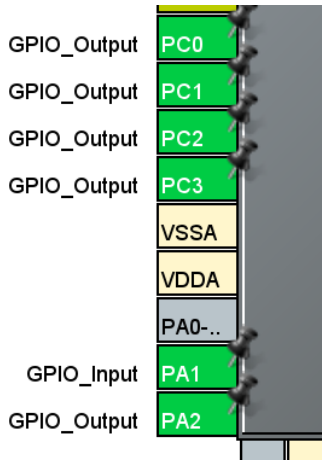



*Abbildung 42:Konfiguration der GPIO-Pins*

In CubeMX wurde I2S2 als Half-Duplex Transmit Master und I2S3 als Half-Duplex Receive Only Slave konfiguriert.

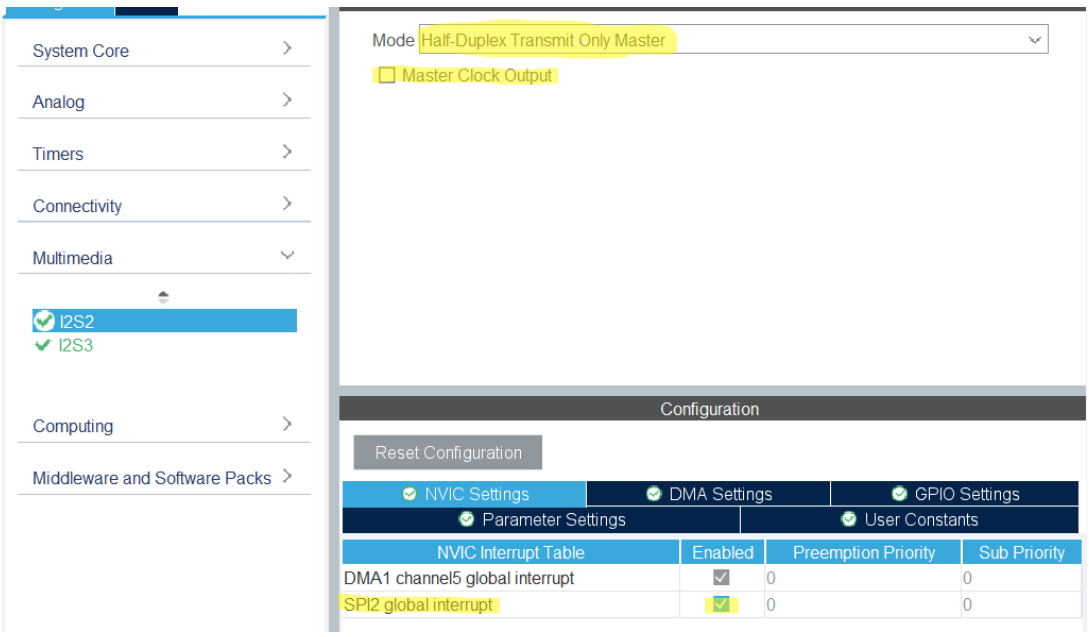



*Abbildung 43: Konfiguration von I2S2 in CubeMX*

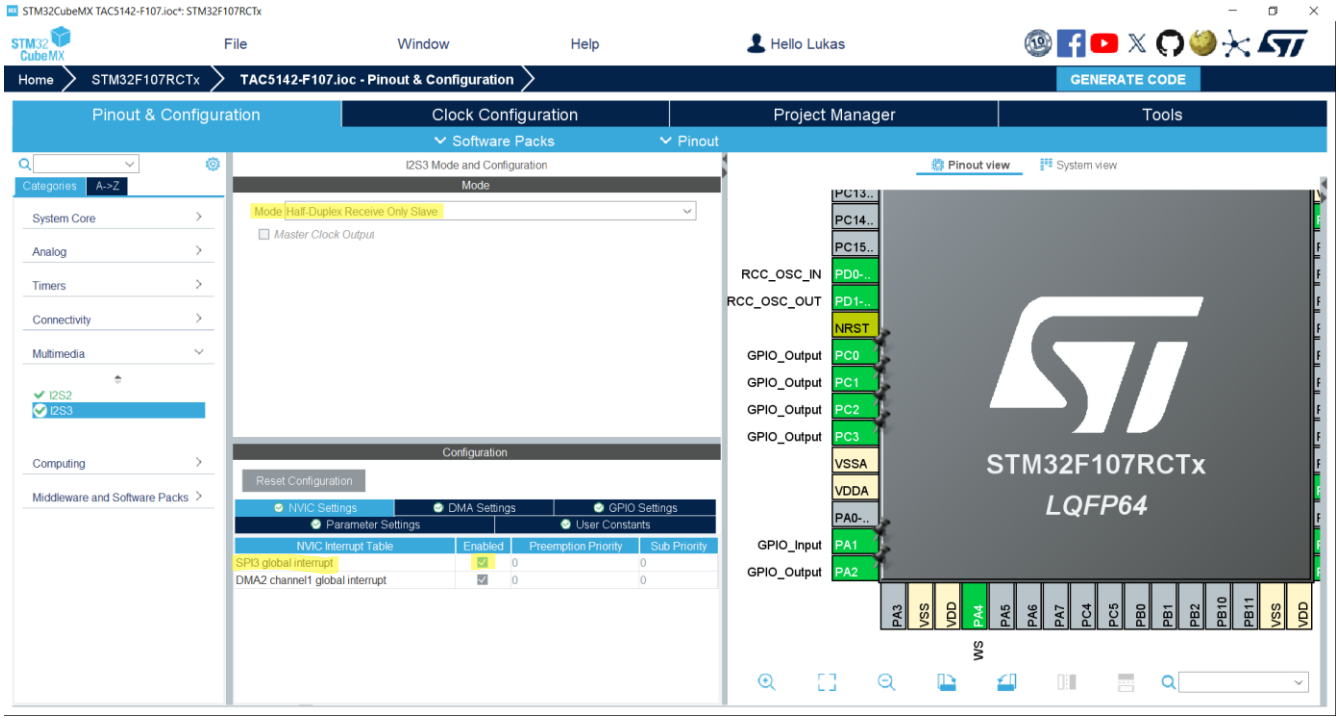



*Abbildung 44: Konfiguration von I2S3 in CubeMX*

### ***5.5.2.2*** *Programmcode*

Der Code aktiviert beide DMA's und konfiguriert die Konfigurationspins des TAC5142. In den Interrupts „`HAL_I2SEx_TxRxHalfCpltCallback`" und „`HAL_I2SEx_TxRxCpltCallback`" werden die Audiodaten verarbeitet und an das Ausgangsregister geschrieben. Genauer ist dies in Kapitel 7 nachzulesen.

```
HAL_I2S_Receive_DMA(&hi2s2, rx_buf, 4)
HAL_I2S_Transmit_DMA(&hi2s3, tx_buf, 4)
```

Dieser Code aktiviert beide DMA's.

```
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, 1);//MD1
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_1, 1);//MD2
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_2, 1);//MD3
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_3, 1);//MD4
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_2, 1);//MD5
```

Dieser Code konfiguriert die Pins MD1-MD5, so wie in Kapitel 5.5.1 beschrieben.

## 5.6 Breakoutboard

Das Breakoutboard wurde entwickelt, um andere Möglichkeiten mit dem Audiocodec TAC5142 auszutesten.

### 5.6.1 Schaltung

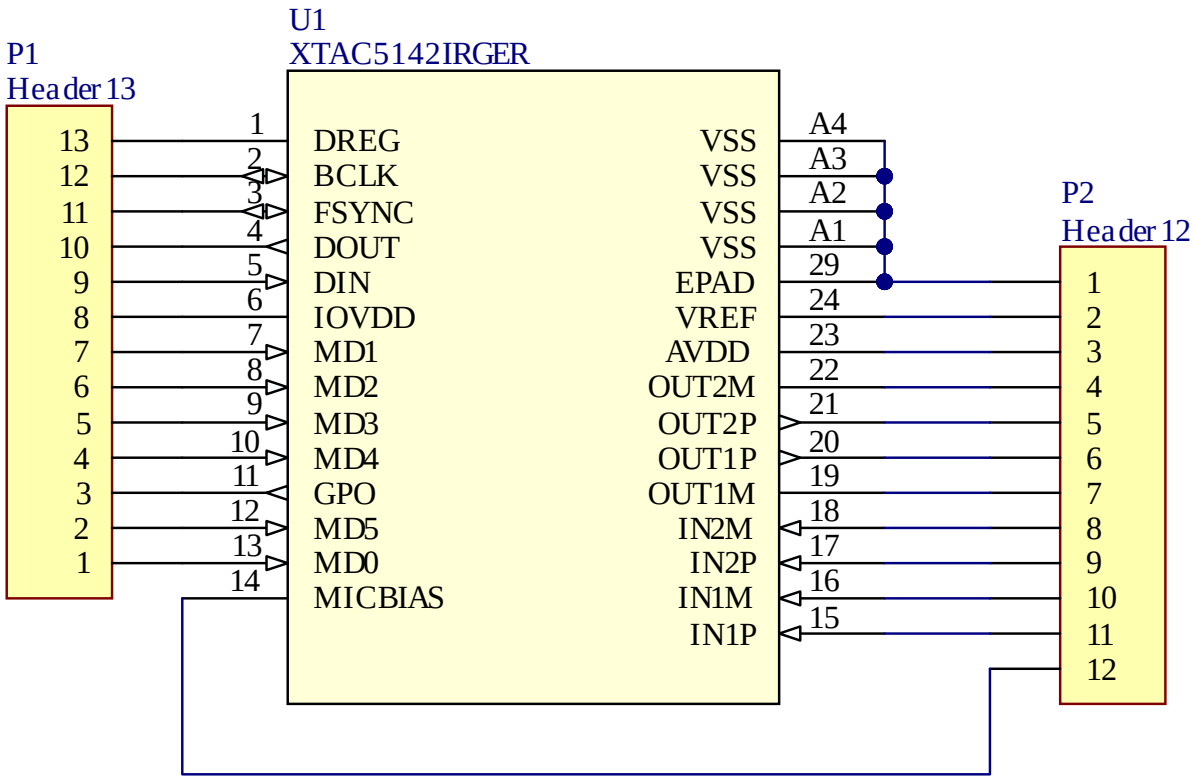



*Abbildung 45: Schaltung des Breakoutboards*

Die Schaltung verbindet alle GND-Pads auf einen Pin und alle anderen Pins auf einen individuellen Pin der 12- und 13-Pin Stiftleiste.

### 5.6.2 Layout

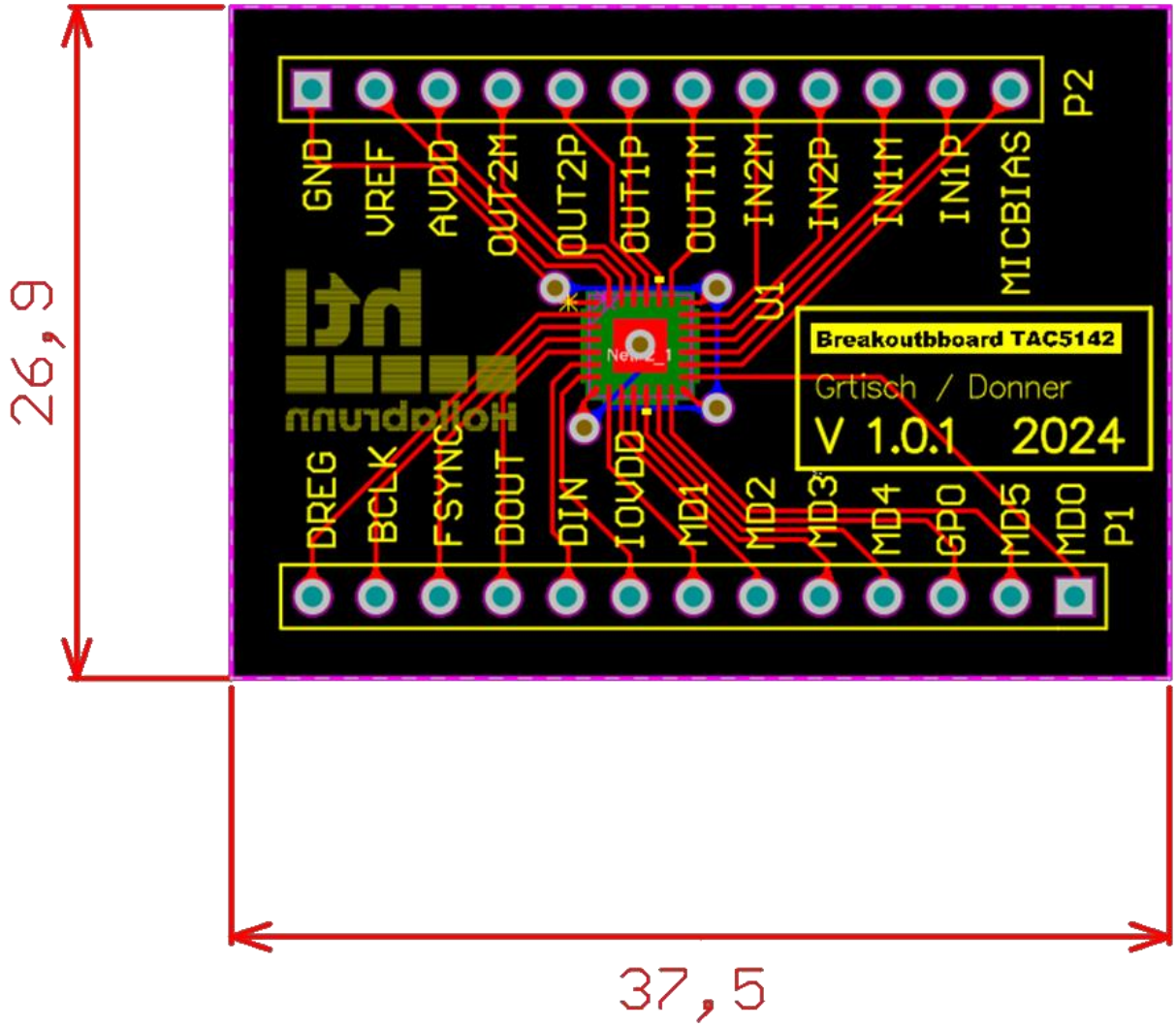



*Abbildung 46: Layout des Breakoutboards*

Auf der Platine sind Beschriftungen für die einzelnen Pins hinzugefügt worden, sowie das HTL Logo, der Platinen-Name und die Versionsnummer.

## 5.7 Fehlersuche

### 5.7.1 Messungen

Die Platine wurde, wie in Abbildung 47, an das MDDS-System angesteckt und das Demoprogramm, welches im Kapitel 5.5.2 beschrieben wurde, geladen. Diesmal war kein Kurzschluss feststellbar. Deswegen wird die BCLK, wie in Abbildung 50, das WS-Signal, wie in Abbildung 49 und die Datenleitung wie in Abbildung 51 gemessen. Außerdem werden das Eingangssignal, wie in Abbildung 52, und das Ausgangssignal gemessen. Die Abbildung 48 zeigt, wie der genaue Messaufbau umgesetzt wurde.

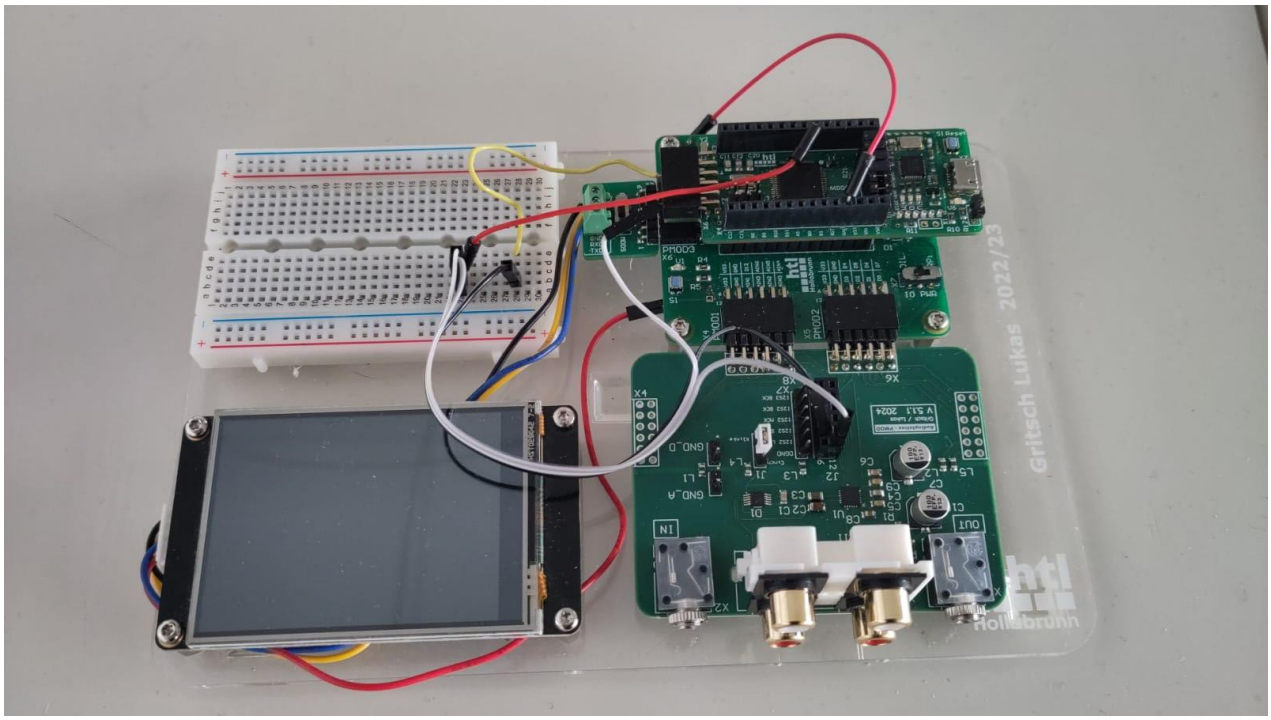



*Abbildung 47: MDDS System mit V5.1.1.*

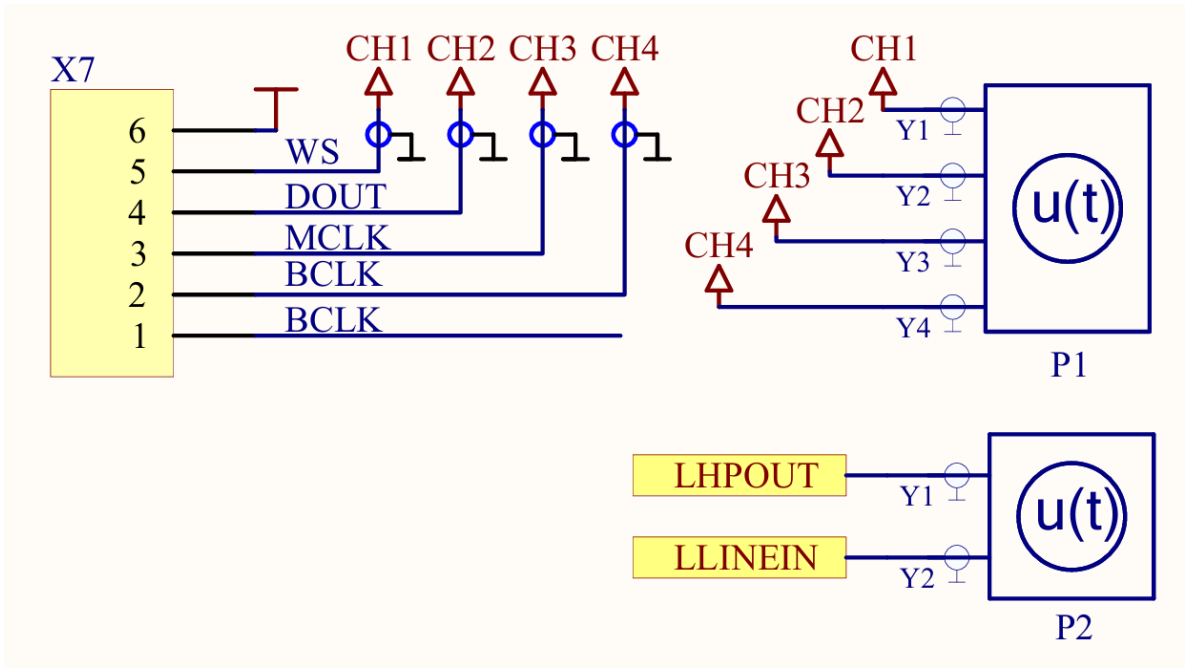



*Abbildung 48: Messchaltung für den TAC5142*

Die genutzten Bezeichnungen sind in der Gesamtschaltung im Kapitel 5.3.3. wieder zu finden.
Das ist die WS-Leitung, die vom µC STM32F107T6 erzeugt wird. Weil eine Abtastrate von 48kHz auswählt wurde, misst das Oszilloskop ein Rechtecksignal mit einer Frequenz von 48,82kHz.

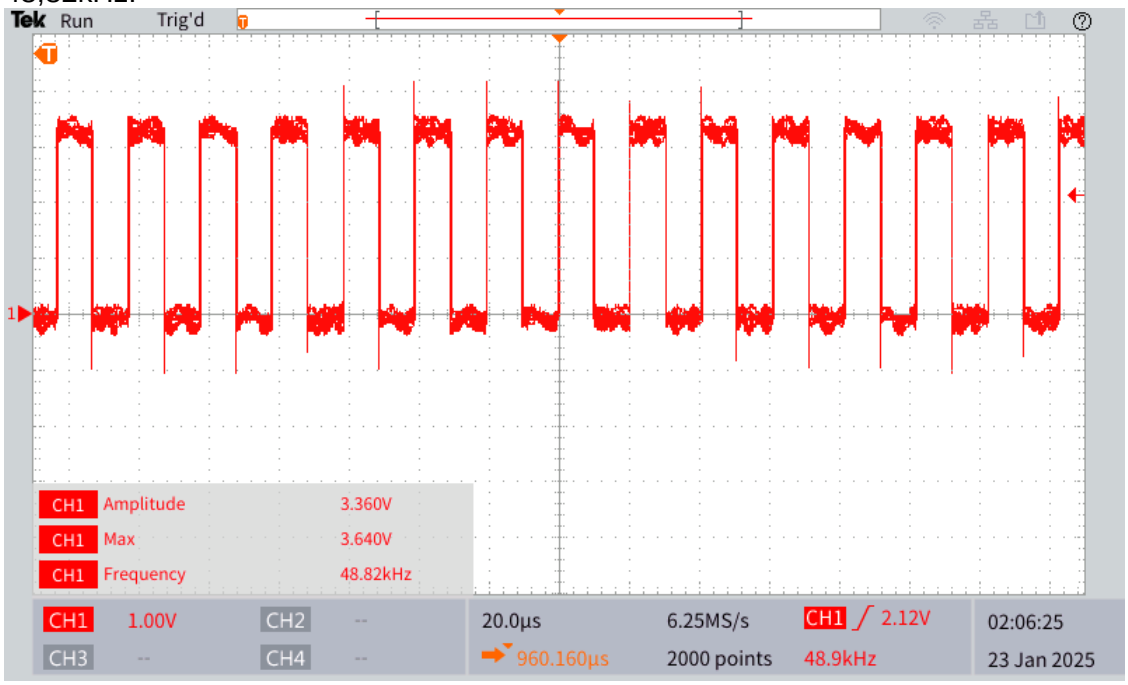



*Abbildung 49: Messung von Word Select*

Das ist die BCLK, die vom µC STM32F107T6 erzeugt wird. Mit dem Oszilloskop werden 3,129MHz gemessen, weil die BCLK immer der 64-fachen Frequenz des WS-Signals entsprechen muss.

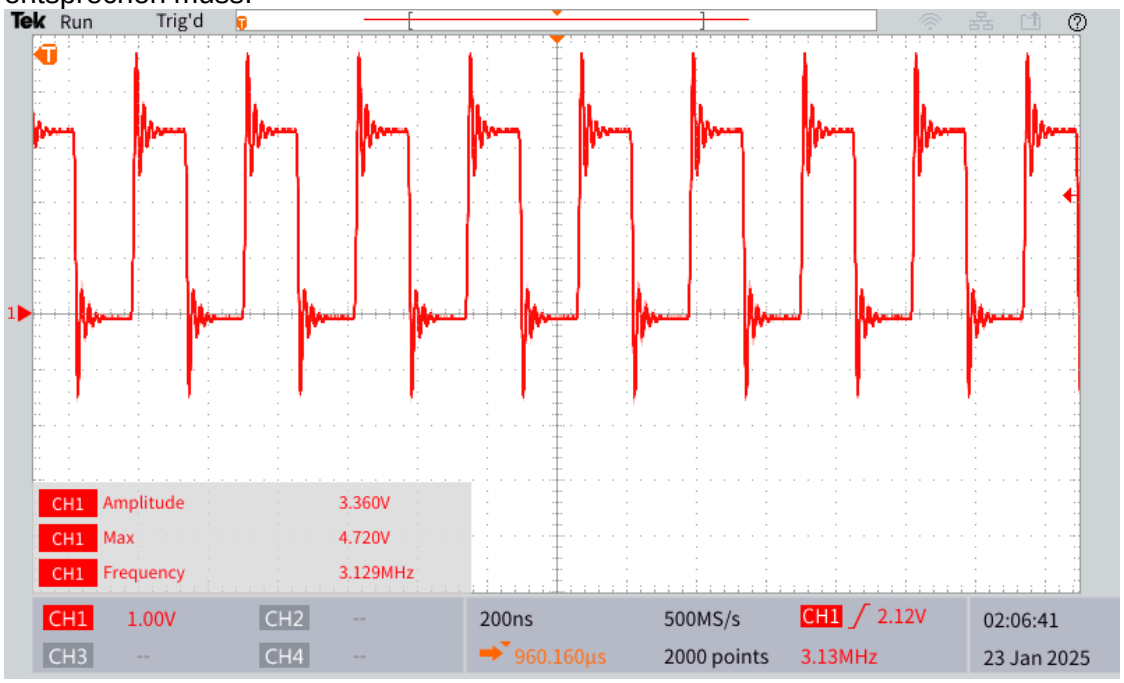



*Abbildung 50: Messung der BCLK*

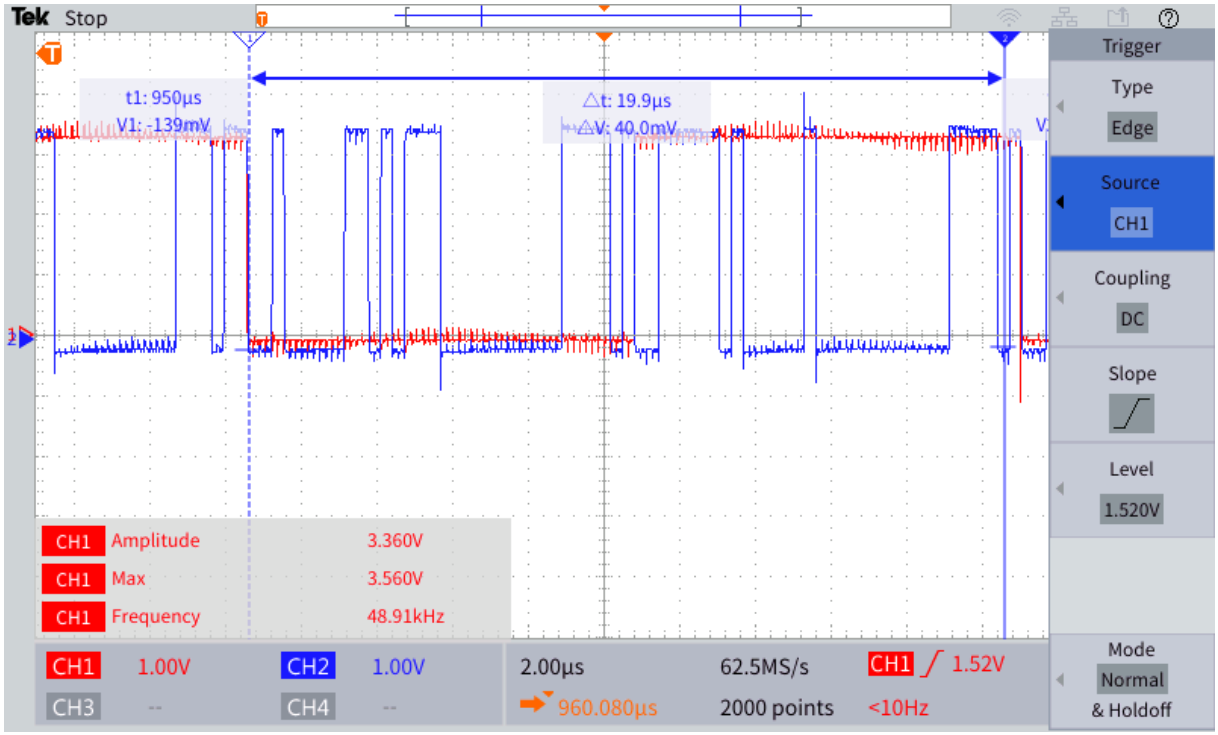



*Abbildung 51: Messung der Datenleitung*

In Rot ist die WS-Leitung dargestellt und in Blau die von I2S3 geschickten Daten. Für ein korrektes I2S-Protokoll müssen die letzten 8bit Daten immer „Low“ sein, was hier nicht der Fall ist, weil die 8bit in der Mitte des Wertes liegen. Das heißt, dass die Synchronisation zwischen I2S2 und I2S3 nicht möglich ist. Weil das 8bit Loch nicht an der richtigen Stelle ist, kann der Audiocodec das Signal nicht auswerten. Der Audiocodec kann Daten nur im Format, welches im Kapitel 5.5.1 erläutert wird, auswerten. Das bedeutet, dass dieser Aufbau nicht mit einem halbduplexen Aufbau funktionieren kann. Ein Grund für die nicht funktionierende Synchronisation ist, dass I2S2 und I2S3 intern verschiedene MCLK’s verwenden.

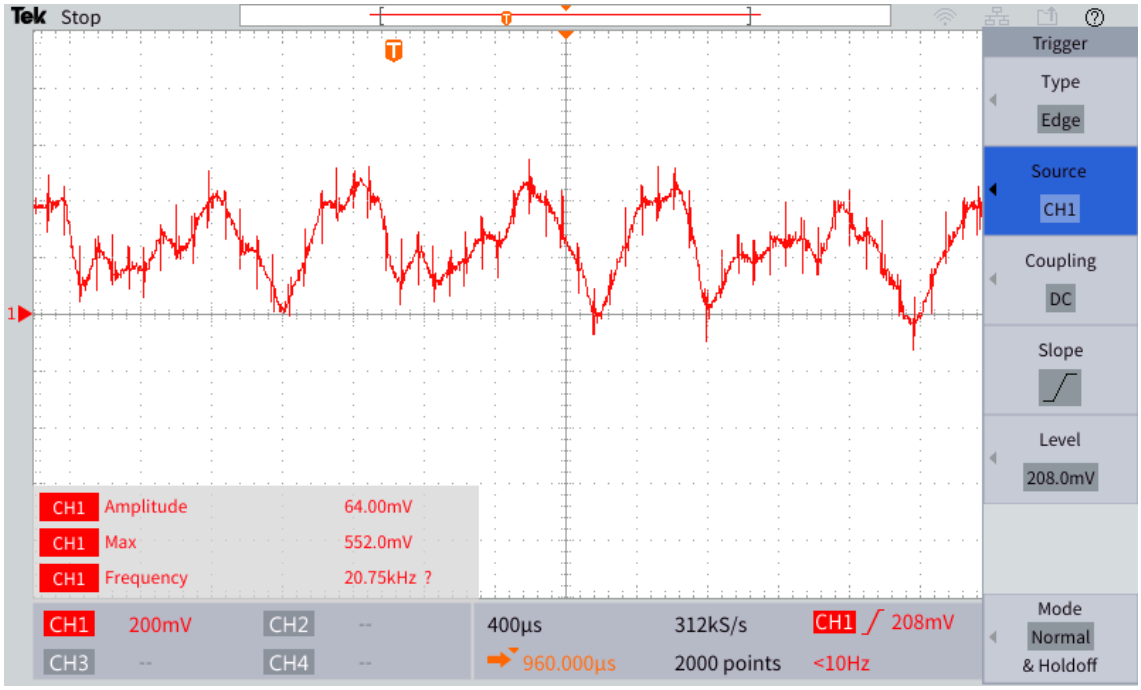



*Abbildung 52: Messung des Eingangssignals*

Hier sieht man das Audiosignal, welches von meinem Handy erzeugt wurde und direkt vor dem Pin des TAC5142 abgegriffen wurde. Ein typisches Audiosignal, wie ein vom Smartphone erzeugtes Audiosignal, hat eine maximale Spitze-Spitze-Spannung von 1V. Dies bedeutet, dass eine maximale Spitze-Spitze-Spannung von 552mV ein valides Audiosignal ist.

**Table 7-5. Supported FSYNC (Multiples or Submultiples of 48kHz) and BCLK Frequencies (IOVDD - 3.3V Operation)**

| BCLK TO FSYNC RATIO | BCLK (MHz) | | | | | | |
|---|---|---|---|---|---|---|---|
| | FSYNC (8 kHz) | FSYNC (16 kHz) | FSYNC (24 kHz) | FSYNC (32 kHz) | FSYNC (48 kHz) | FSYNC (96 kHz) | FSYNC (192 kHz) |
| 16 | **Reserved** | 0.256 | 0.384 | 0.512 | 0.768 | 1.536 | 3.072 |
| 24 | **Reserved** | 0.384 | 0.576 | 0.768 | 1.152 | 2.304 | 4.608 |
| 32 | 0.256 | 0.512 | 0.768 | 1.024 | 1.536 | 3.072 | 6.144 |
| 48 | 0.384 | 0.768 | 1.152 | 1.536 | 2.304 | 4.608 | 9.216 |
| 64 | 0.512 | 1.024 | 1.536 | 2.048 | 3.072 | 6.144 | 12.288 |
| 96 | 0.768 | 1.536 | 2.304 | 3.072 | 4.608 | 9.216 | 18.432 |
| 128 | 1.024 | 2.048 | 3.072 | 4.096 | 6.144 | 12.288 | 24.576 |
| 192 | 1.536 | 3.072 | 4.608 | 6.144 | 9.216 | 18.432 | **Reserved** |
| 256 | 2.048 | 4.096 | 6.144 | 8.192 | 12.288 | 24.576 | **Reserved** |
| 384 | 3.072 | 6.144 | 9.216 | 12.288 | 18.432 | **Reserved** | **Reserved** |
| 512 | 4.096 | 8.192 | 12.288 | 16.384 | 24.576 | **Reserved** | **Reserved** |

*Abbildung 53: Tabelle der unterstützten Frequenzen [TI25k]*

Abbildung 49 und 50 zeigen, dass eine BCLK von 3,12MHz und eine WS von 48kHz anliegt, was laut der Tabelle valide Frequenzen sind.

Dies bedeutet, dass der Audiocodec die korrekte Versorgungspannung, BCLK, WS und ein korrektes Audiosignal erhält. Aus diesem Grund ist darauf zu schließen, dass der Eingang des Audiocodec TAC5142 nicht korrekt funktioniert. Der Ausgang ist wegen dem Fehler in der Synchronisation nicht testbar.

### 5.7.2 Erkenntnisse

Es wurde ein Demoprogramm mit Master-Recieve-Mode (I2S2) und Slave-Transmit-Mode (I2S3) entwickelt und das einzige Problem stellt die Synchronisation zwischen I2S2 und I2S3 dar. Das Problem mit der Synchronisation ist im Kapitel 5.7.1 genauer erläutert. Dies könnte verursachen, dass der Audiocodec kein sinnvolles Ausgangssignal erzeugt. Doch auch wenn nur Daten empfangen werden sollen, also keine Daten vom µC geschickt werden, schickt der Audiocodec keine Daten. Dies bedeutet, dass weder Ein- noch Ausgangssignale generiert werden, was bedeutet, dass der Audiocodec nicht richtig arbeitet und die Platine dadurch nicht funktioniert. Die Audioplatine wurde im Jänner 2025 vollständig getestet. Da die aufwendige Fehlersuche viel Zeit in Anspruch genommen hätte und die Erfolgschancen sehr gering einzuschätzen waren, wurde stattdessen ein neuer Ansatz probiert. Der neue Ansatz wurde mit dem PMOD I2S Modul von der Firma Digilant als Vorlage umgesetzt.

# 6 Audioboard mit ADC CS5343 und DAC CS4344

Diese Platine ist fast eine Kopie des „Digilent Development Kit, Pmod I2S2 Stereo Audio Input and Output“, welches ein Open-Source Projekt ist. Die Platine soll so umgebaut werden, dass die relevanten Pins für das I2S-Protokoll auf einer Stiftleiste für den Logikanalysator abgreifbar sind. Weiters muss die Stiftleiste, mit der die Platine an das MDDS-System angeschlossen wird, an das MDDS-System angepasst werden.

## 6.1 Einführung in den CS5343 und CS4344

Der CS5342 ist ein I2S-fähiger ADC, der analoge Signale in ein korrektes I2S-Protokol mit 24bit Daten umwandeln kann. Außerdem unterstützt er eine Master/Slave-Auswahl und Samplerates bis zu 108kHz. [MAU25a] Der CS4344-CZZR ist ein I2S-fähiger DAC, der ein korrektes I2S Signal mit 24bit Daten in ein analoges Signal mit einer Samplerate von bis zu 192kHz umwandeln kann. [MAU25b]

## 6.2 Musterschaltung

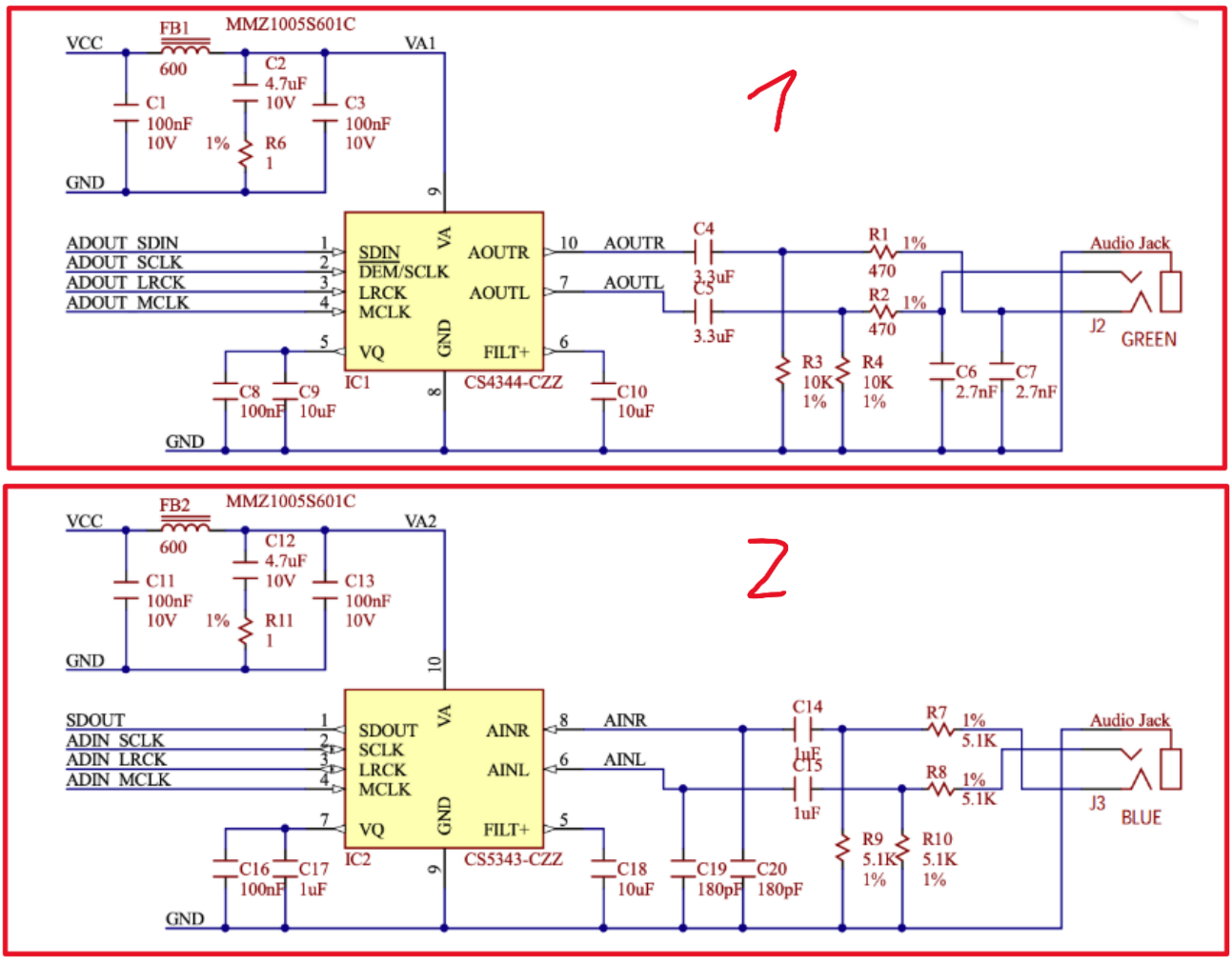



*Abbildung 54: Schaltung des Schaltmoduls von Digilent Development Kit [AUD25a]*

Diese Schaltung ist die des Digilent Development Kits. Dieses Schaltmodul wurde bereits für das Entwickeln der Software genutzt und funktioniert einwandfrei. Der Unterschied zu der in dieser Diplomarbeit entwickelten Schaltung ist, dass das Pinning der PMOD-Schnittstelle verändert wurde, sowie eine Master/Slave-Auswahl und eine Stiftleiste zum Analysieren der

I2S-Schnittstelle mit einem Logikanalysator hinzugefügt wurde. Der Schaltungsteil 1 wandelt I2S-Signale in ein Audiosignal um. Teil 2 wandelt ein Audiosignal in I2S-Signale um.

## 6.3 Schaltung

### 6.3.1 Master/Slave-Auswahl

Diesen Schaltungsteil kann man in Abbildung 57 unter der Nummer 3 finden. Beim ADC CS5343-CZZ kann man zwischen Master und Slave-Mode umschalten. Um zwischen Master und Slave umzuschalten, ist ein 10kΩ auf der SD-Leitung notwendig, was in Abbildung 45 dargestellt wird.

| Mode | Selection |
|---|---|
| Master Mode | 10 kΩ pull-up resistor from SDOUT to VA |
| Slave Mode | 10 kΩ pull-down resistor from SDOUT to GND |

*Abbildung 55: Master/Slave-Auswahl [MOU25a]*

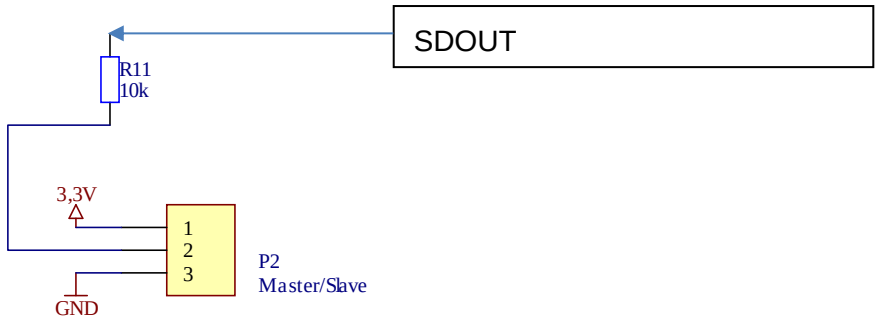



*Abbildung 56: Schaltungsumsetzung der Master/Slave-Auswahl*

Beim Master-Modus werden die Clock-Signale vom ADC CS5343 erzeugt. Im Slave-Modus braucht er die Signale vom µC.

## 6.3.2 Gesamtschaltung

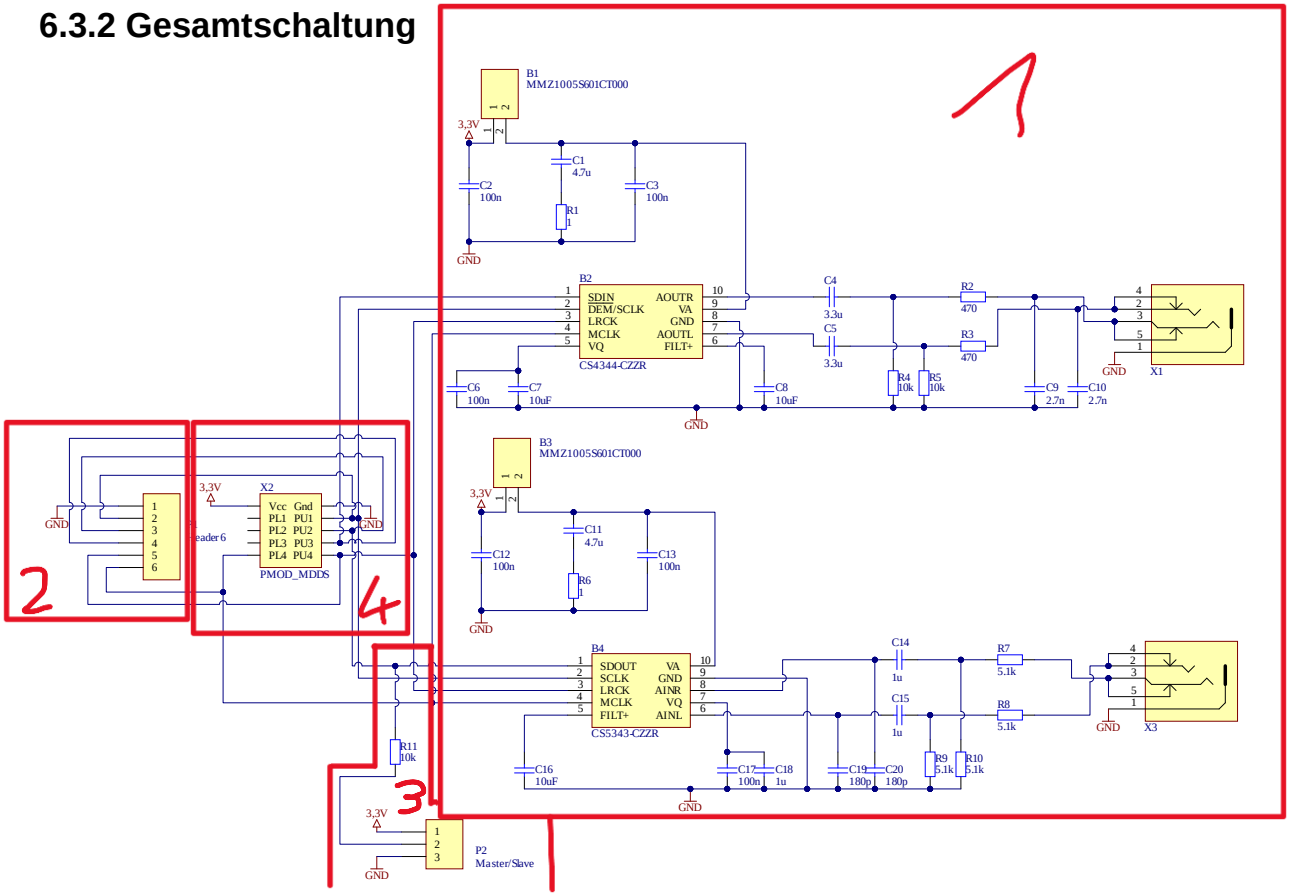



*Abbildung 57: Schaltung des V5.2.1*

Der Schaltungsteil 1 ist eine Kopie der Musterschaltung im Kapitel 6.2. Zusätzlich verbaut ist eine Stiftleiste mit GND, BCLK, DIN, DOUT, WS und MCLK, welche die relevanten Signale des I2S-Protokolls darstellen und leicht mit einem Logikanalysator analysiert kann. Dieser Teil wird mit 2 bezeichnet. Der Schaltungsteil 4 ist die PMOD-Schnittstelle, welche an den µC STM32F404T6 angepasst wurde.

Diese Schaltung wird nicht mit einem Audiocodec aufgebaut, sondern hat getrennten ADC und DAC. Außerdem wurden nicht die exakt gleichen Bauteile verwendet, weil diese nicht mehr hergestellt werden. Es wurden nicht CS4344-CZZ und CS5343-CZZ sondern CS4344-CZZR und CS5343-CZZR verwendet, welche aber ident zum Vorgänger sind.

## 6.4 Layout

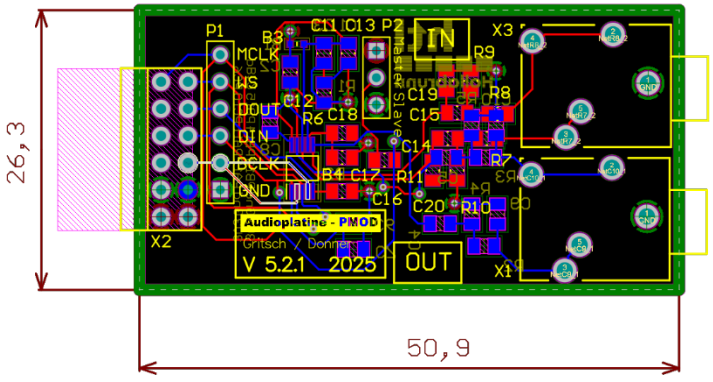



*Abbildung 58: Layout der V5.2.1*

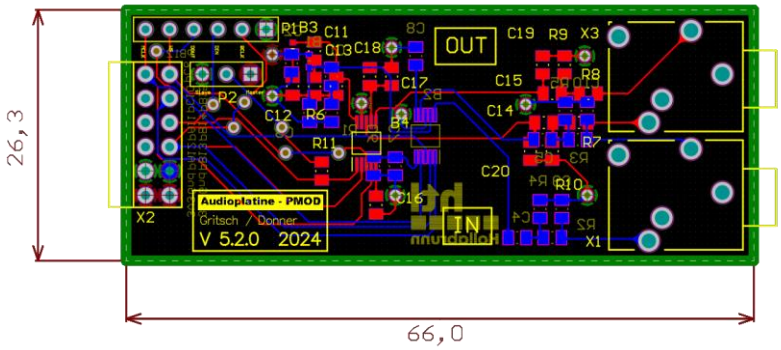



*Abbildung 59: Layout der V.5.2.0*

Die Platine wurde so designt, dass sie an einer einzelnen PMOD-Schnittstelle angesteckt werden kann. Die Platine V5.2.0 ist schaltungstechnisch bis auf die fehlende Masseverbindung der Klinkenbuchsen, welche mit einem Draht nachgelötet wurde, ident zu der Platine V5.2.1. Jedoch wurde die V5.2.1 noch nicht bestückt, weswegen alle Tests mit V5.2.0 durchgeführt wurden. Weiters wurden die Beschriftungen der V5.2.1 angepasst, die effizienter geroutet wurden, wodurch sie kürzer ist.

## 6.5 Ergebnisse

Die Platine wurde mit der Software Audioverarbeitungssoftware V2 getestet, welche im Kapitel 7 genau erläutert wird.

### 6.5.1 Messungen

Beim Anstecken der Platine konnte kein Kurzschluss festgestellt werden, deswegen wurde als Erstes das I2S-Protokoll kontrolliert. Die Kontrolle des I2S-Protokoll wurde mit dem Logikanalysator durchgeführt, was in Abbildung 61 sichtbar ist. Als nächstes wird der Ausgang mit dem Sägezahnsignal, dem Rechtecksignal, dem Dreiecksignal und dem Sinussignal getestet. Dafür wird in der Software der dazugehörige Modus ausgewählt. Der linke oder rechte Kanal des Klinkensteckers wird an das Oszilloskop angesteckt und die Masse an den Schirm des Klinkensteckers. Die Ergebnisse sind in Abbildung 62-65 zu sehen. Für den Test des Eingangs wird der Talkthrough ausgewählt. Am Eingang wird mit einem Smartphone ein Lied abgespielt und am Ausgang wird ein Lautsprecher angeschlossen.

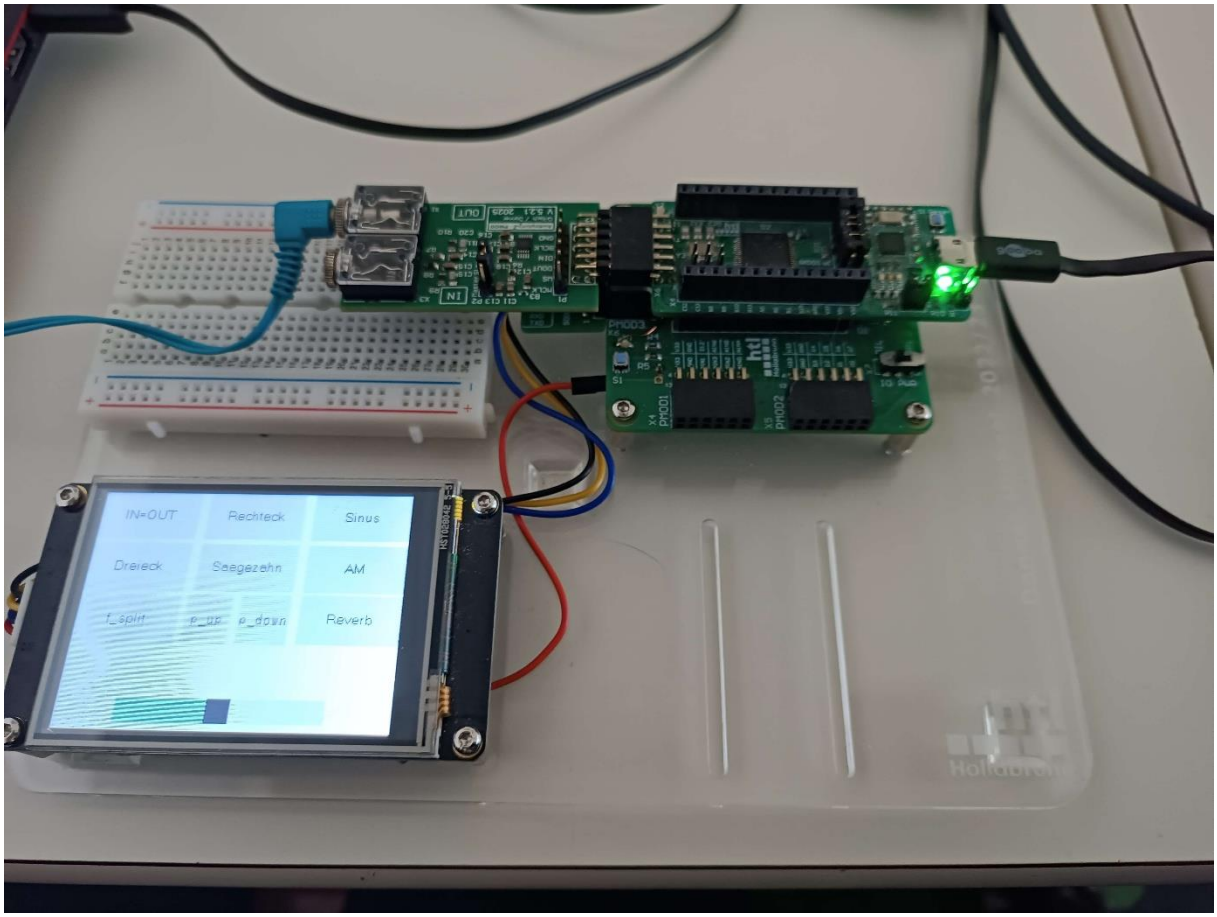



*Abbildung 60: Audioplatine V5.2.1 am MDDS-System angesteckt*




*Abbildung 61: I2S-Protkoll mit Logikanalysator gemessen*

In diesem Aufbau ist der Cortex Master und erzeugt ein Rechtecksignal. An der Audioplatine liegt kein Signal an, wodurch es nur „Rauschen“ misst. Dieses Rauschen sind kleine negative Werte, wie z.B.: 0xFFD226.

Wenn die WS-Leitung von High auf Low bzw. von Low auf High geht, dann lässt der Audiocodec einen BCLK-Cycle frei und sendet dann MSB bis LSB auf der fallenden Taktflanke. Am Ende bleiben noch einige Bits frei, weil nur 24bit Daten auf einen 32bit Frame gesendet werden.

Anschließend wurden verschiedene Signalformen erzeugt, um die Korrektheit des DAC der Audioplatine zu überprüfen.

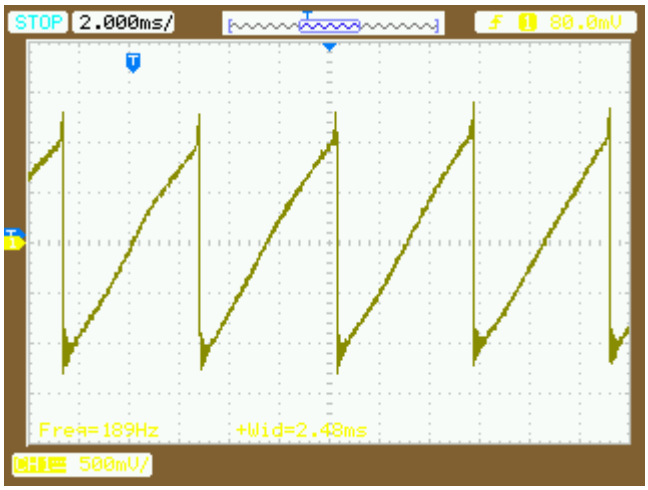



*Abbildung 62: Sägezahnsignal von der Audioplatine erzeugt*

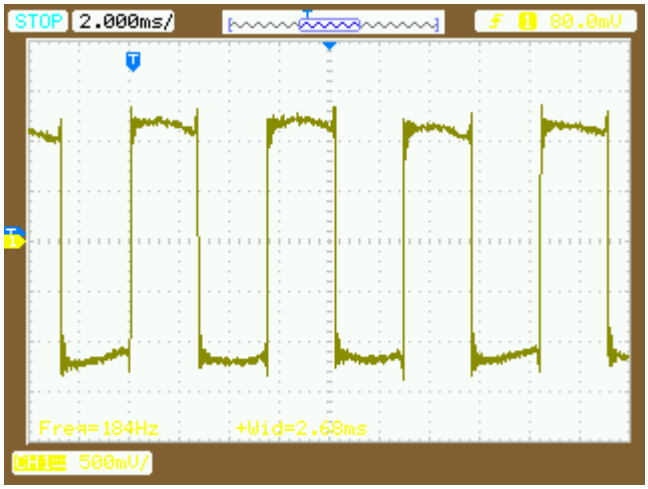



*Abbildung 63: Rechtecksignal von der Audioplatine erzeugt*

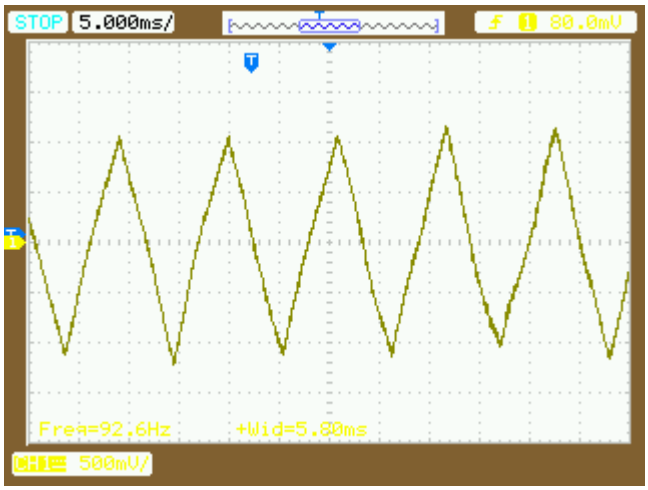



*Abbildung 64: Dreiecksignal von der Audioplatine erzeugt*

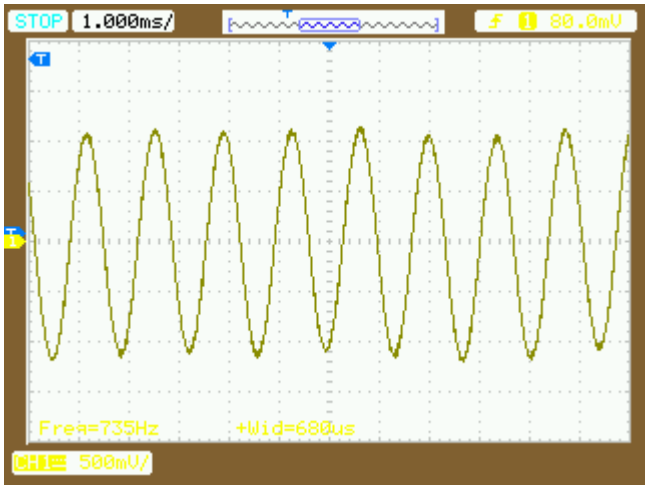



*Abbildung 65: Sinussignal von der Audioplatine erzeugt*

Die Signalformen zeigen, dass der DAC korrekt arbeitet.

Außerdem wurde mit einer 1:1 Verbindung die Übertragung eines Songs überprüft, was erfolgreich durchgeführt werden konnte.

### 6.5.2 Ergebnisse

Die Platine wurde mit Signalformen, einer 1:1-Verbindung und mit dem Logikanalysator überprüft und zeigt keine Fehler auf.

Die Version 5.2.0 hat die Schwäche, dass die Masseverbindung der Klinkenbuchsen mit einem Draht nachgelötet sind, die IN und OUT Beschriftungen vertauscht sind und die Platine etwas zu lang ist. Deswegen wurden die Beschriftungen, die Länge der Platine und die Masseverbindung in der Version 5.2.1 angepasst.

# 7 Software Audio Effekt Box

## 7.1 Software-Architektur

### 7.1.1 Einleitung

In diesem Kapitel wird die Software-Architektur der Audio Effektbox beschrieben. Dazu zählt Software für den Mikrocontroller, das Nextion-Display und die Kommunikation mit dem Audio-Codec. Das folgende Blockschaltbild beschreibt den allgemeinen Aufbau der Software-Architektur.

### 7.1.2 Blockschaltbild

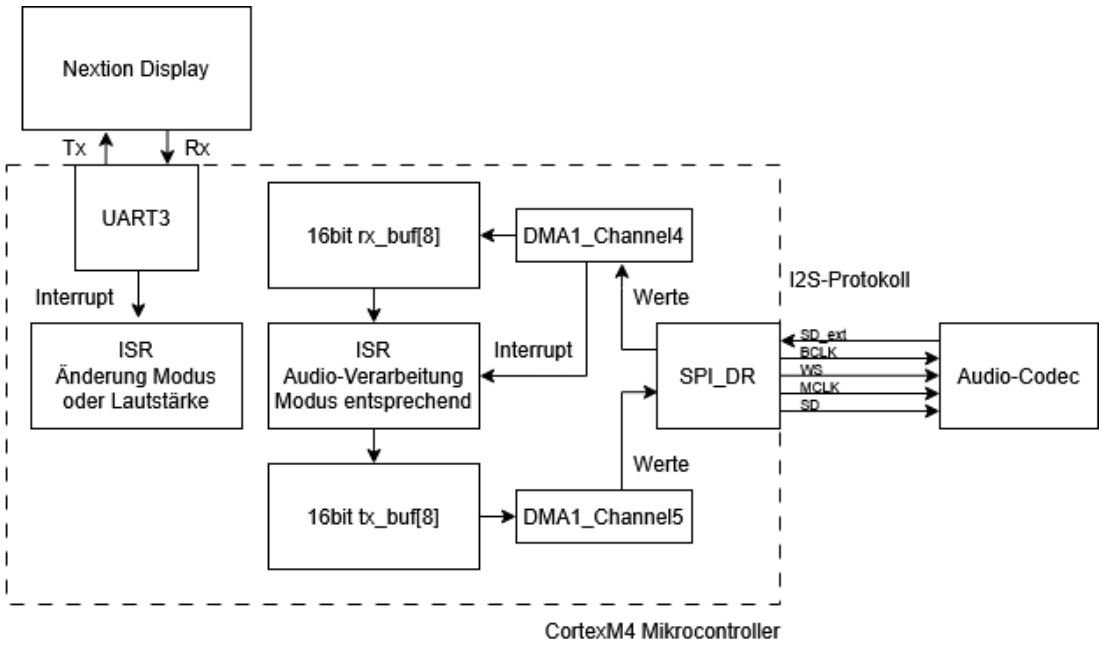



*Abbildung 66: Allgemeines Blockschaltbild der Software-Architektur*

### 7.1.3 Beschreibung

Neue Audio-Samples werden über das I2S-Protokoll an das Peripherie-Datenregister SPI_DR gesendet. Von dort aus kopiert der DMA1_Channel4 die neuen Audio-Samples in den internen RAM-Speicher rx_buf[8], wo eingehende Audio-Samples abgespeichert werden. Bei jedem Interrupt des DMA1_Channel4 werden die neuen Audio-Samples verarbeitet und in den Ausgangs-Buffer tx_buf[8] gespeichert. Von dort aus kopiert der DMA1_Channel5 die ausgehenden Werte an das SPI_DR-Register, welches Speicher für jeweils eingehende und ausgehende Audio-Samples hat. Danach werden die Audio-Samples über das I2S-Protokoll wieder an den Audio-Codec gesendet.

Es könnte die Frage aufkommen, warum ein Element des Buffers für die Audio-Samples 16bit groß ist, obwohl ein einzelnes Audio-Sample 24bit groß ist. Der Grund dafür liegt im Peripherie-Datenregister (SPI-DR), welches lediglich 16bit breit ist. Damit ein Buffer-Element direkt in dieses Register geschrieben werden kann, ist es sinnvoll, die Buffer-Struktur ebenfalls 16 Bit groß zu gestalten. Aus diesem Grund wird auch ein 24bit-Audio-Sample auf zwei aufeinanderfolgende 16bit-Buffer-Elemente verteilt. Ein Audio-Sample wird also beispielsweise in tx_buf[0] und tx_buf[0] gespeichert.

Wie im Kapitel **I2S** schon beschrieben, ist die Abtastrate 48kHz. Das heißt alle 1/48000s wird ein Left- und ein Right-Audio-Sample vom Mikrocontroller eingelesen und auch ausgegeben.

Die Auswahl welche Audio-Verarbeitung stattfindet und die Anpassung der Lautstärke der Audio wird über das Nextion-Display gesteuert. Dieses löst bei Änderung ein Interrupt im Hauptprogramm aus, wodurch der Modus der Audio-Verarbeitung bzw. die Lautstärke geändert wird.

Eine genauere Beschreibung wie die einzelnen Komponenten funktionieren ist in späteren Kapiteln beschrieben.

## 7.2 Entwicklungs-Hardware

### 7.2.1 Allgemein

Um während der Entwicklungszeit der neuen Audio-Platine die Software bereits testen zu können, wurde ein Audio-Codec und eine Platine zur Verfügung gestellt, auf der die Software getestet werden konnte.

### 7.2.2 Audio-Codec

Als Audio-Codec wurde der „Digilent PMOD I2S2“-Audio-Codec verwendet. Dieser ist von den benötigten I2S-Signalen gleich und benutzt auch einen 24bit ADC und 24bit DAC.




*Abbildung 67: Test Audio-Codec*

### 7.2.3 Platine

Die im Bild zu sehende Platine wurde zu Testzeiten verwendet, um die I2S-Signale vom Mikrocontroller zum Audio-Codec zu verbinden. Die PMOD-Schnittstellen am Base-Board wurden zusätzlich für die Übertragung von I2S-Signalen verwendet, bevor alle I2S-Signale über die Mikrocontroller PMOD-Schnittstelle übertragen wurden. Dadurch, dass die I2S-Signale einen längeren Weg bis zu dem Audio-Codec nehmen mussten tauchten öfter Störungen bzw. Rauschen auf.

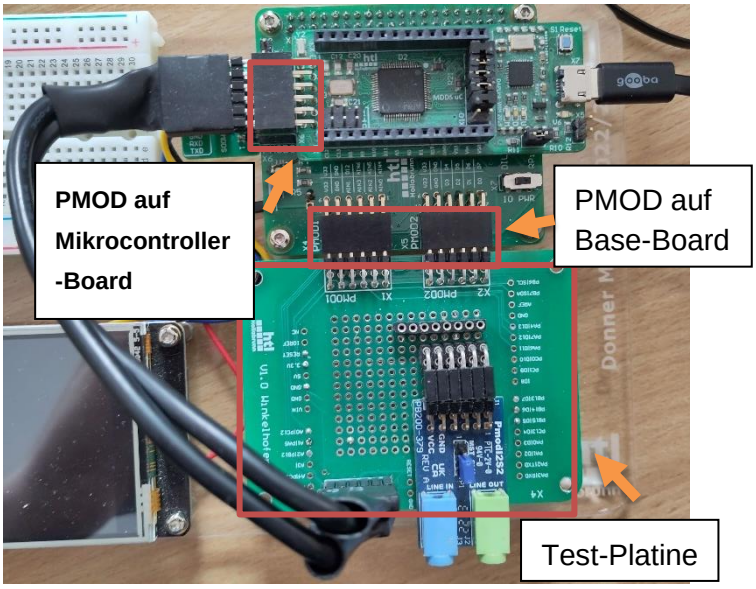



*Abbildung 68: Test-Platine für Test-Audio-Codec*

### 7.2.4 Base Board Ports

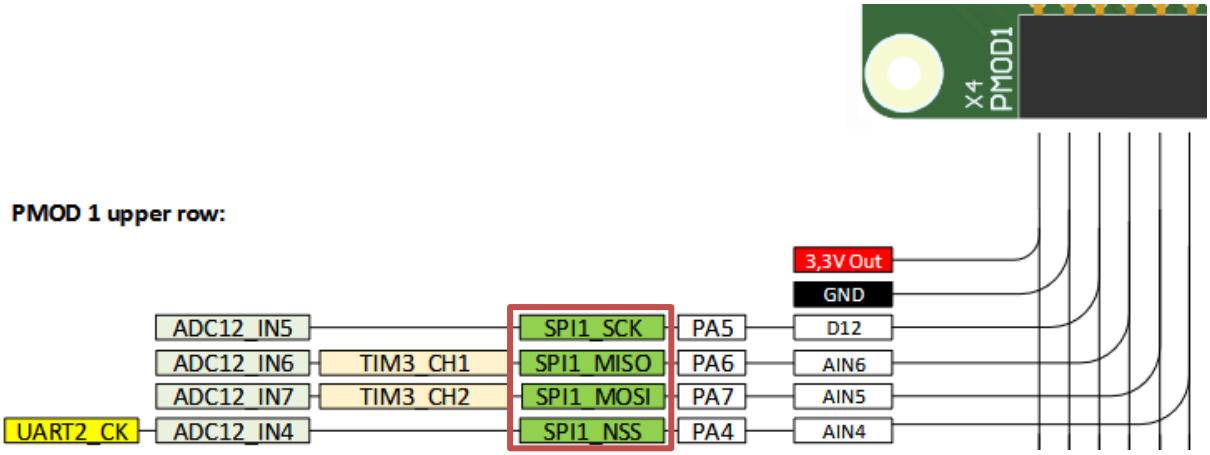



*Abbildung 69: PMOD Base Board Platine*

Auf der Base-Board Platine gibt es auf der linken PMOD-Schnittstelle ein weiteres SPI-Interface, welches als I2S-Schnittstelle umkonfiguriert wurde. Diese I2S-Schnittstelle wurde zusätzlich zu Testzeiten verwendet, da der Mikrocontroller STM32F107RBT6 I2S nur im Half-Duplex Betrieb verwenden konnte.

## 7.3 Auswahl des Mikrocontrollers

### 7.3.1 Allgemein

Da der mit dem MDDS-System standardmäßig verwendete Mikrocontroller STM32F103RBT6 kein I2S-Protokoll unterstützt, musste ein anderer Mikrocontroller ausgewählt werden, der das I2S-Protokoll unterstützt.

### 7.3.2 STM32F107RBT6

Dieser Mikrocontroller war der erste der benutzt wurde, da dieser das I2S-Protokoll unterstützt.
Das Problem an diesem lag daran, dass dieser kein Full-Duplex-Betrieb unterstützt und deshalb zwei I2S-Schnittstellen angelegt werden mussten. Dafür mussten auch alle Clock-Signale zwei Mal existieren, weil beide I2S-Schnittstellen unabhängig voneinander waren. Das wäre prinzipiell kein Problem gewesen, jedoch sind nicht beide Pins, auf denen MCLK ausgegeben werden kann, aus dem Cortex herausgeführt. Dadurch konnte die MCLK der zweiten I2S-Schnittstelle nicht versorgt werden.

### 7.3.3 STM32F407G-DISC1

Das Discovery-Board „*STM32F407G-DISC1*“ diente nur für Testzwecke, ob der Test-Audio-Codec mit diesem funktioniert und damit sollte herausgefunden werden, ob der Test-Audio-Codec überhaupt Audio ausgeben und aufnehmen kann.




*Abbildung 70: STM32407G-DISC1*

### 7.3.4 STM32F303RET6

Letztendlich wurde der Mikrocontroller STM32F303RET6 gewählt, da dieser nicht nur das I2S-Protokoll im Full-Duplex Modus unterstützt, sondern auch PIN-Kompatibel ist und somit mit dem MDDS-System eingesetzt werden kann.

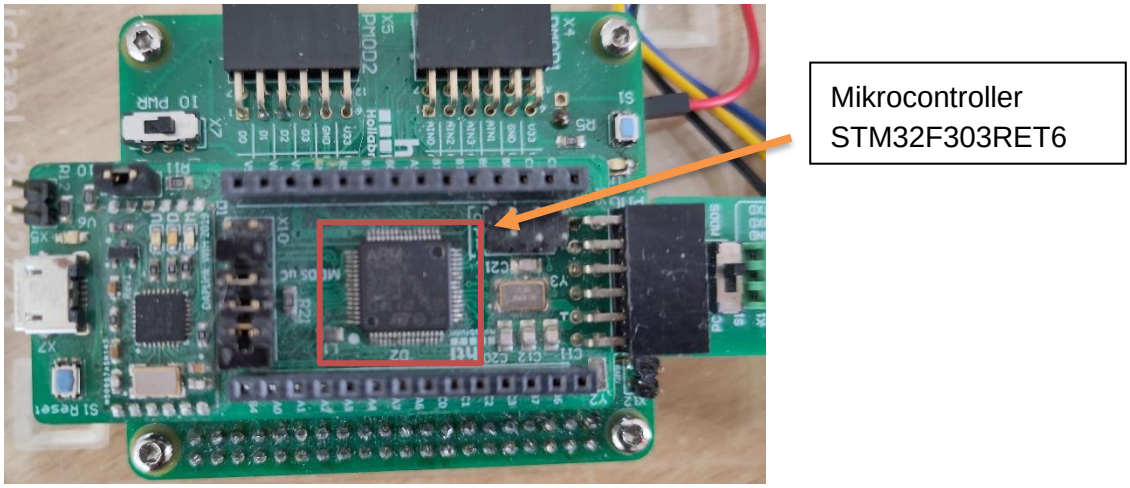



*Abbildung 71: Mikrocontroller STM32F303RET6 auf MDDS-System*

## 7.4 Konfiguration Mikrocontroller STM32F303RET6

### 7.4.1 Allgemein

Um GPIO's, DMA (Direct Memory Access), I2S-Protokoll, Clocks und USART3 zu Konfigurieren wurde STM32CubeMX verwendet.

### 7.4.2 STM32CubeMX

In STM32CubeMX kann der verwendete Cortex-Mikrocontroller ausgewählt werden und auf graphischer Oberfläche können verschiedene Konfigurationen und Initialisierungen getätigt werden. Ist die Konfiguration fertig kann STM32CubeMX Source-Code ausgehend von den Einstellungen generieren. Dieser erstellte Code kann im Anschluss bearbeitet und erweitert werden, wofür in unserem Fall Keil uVision verwendet wurde, da es schon aus der Schulzeit bekannt war.




*Abbildung 72: STM32CubeMX Logo [WIK25]*

### 7.4.3 Nicht verwendete Projekte

#### *7.4.3.1 Allgemein*

Im Laufe der Diplomarbeit wurden mehrere Ansatzmöglichkeiten probiert, um eine funktionierende Audio-Übertragung zu realisieren. Dafür wurden mehrere STM32CubeMX Projekte erstellt, welche entweder einen anderen Cortex verwendet haben oder eine unterschiedliche I2S-Konfiguration.

### *7.4.3.2 STM32F107RCT6 Master-Master Version*

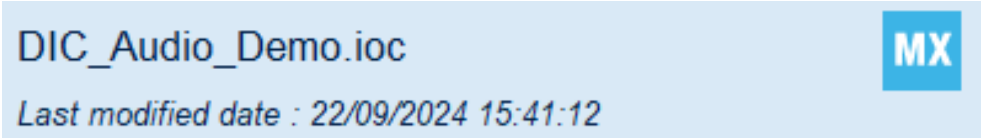



*Abbildung 73: Demo-Projekt STM32CubeMX*

Blockschaltbild:

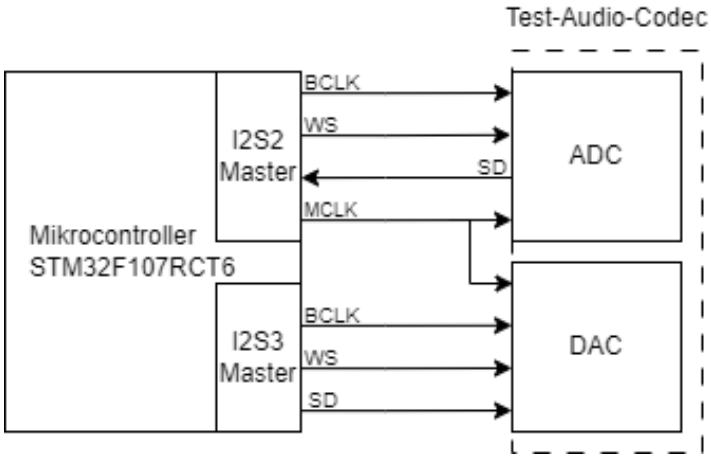



*Abbildung 74: STM32F107RCT6 Master-Master Blockschaltbild*

Dieses Projekt war das Erste und sollte mit Cortex STM32F107RCT6 und zwei Half-Duplex-Master I2S Schnittstellen realisiert werden. Problem: Es kann nur eine Master-Clock erzeugt werden, da die Master-Clock der zweiten Schnittstelle nicht als Port aus dem Cortex herausgeführt ist. Die eine Master-Clock musste also für beide I2S-Protokolle (I2S2 und I2S3) verwendet werden. Durch interne Taktteilungen kann WS bzw. die Abtastrate nicht genau 48kHz sein. Die Abtastraten sind jedoch für beide I2S-Protokolle leicht unterschiedlich mit 46.875kHz und 48.913kHz. Dadurch kann die MCLK der einen Schnittstelle nicht auch für die andere verwendet werden, da das Verhältnis von MCLK = 256 * Abtastrate nicht mehr stimmt.

Messung mit Logikanalysator der tatsächlichen WS-Frequenzen:




*Abbildung 75: WS_I2S2 und WS_I2S3 Diagramm*

WS I2S2:

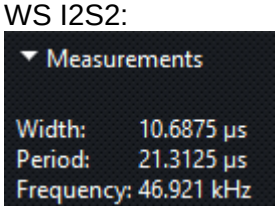



*Abbildung 77: I2S2 Frequenz*

WS I2S3:

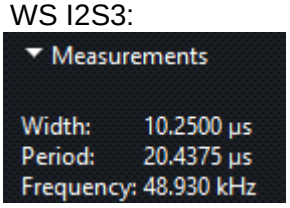



*Abbildung 76: I2S3 Frequenz*

**Ergebnis:** Da die MCLK von der I2S-Schnittstelle ausging, die für das Einlesen von Audio (Receive) zuständig war, konnten nur Daten empfangen werden. Bei dem Ausgeben von Audio konnte immer nur ein Rauschen war genommen werden.

### *7.4.3.3 STM32F107RCT6 Master-Slave Version*

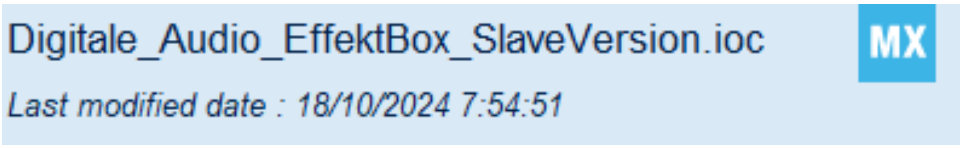



*Abbildung 78: Digitale Audio Effektbox Slave-Version*

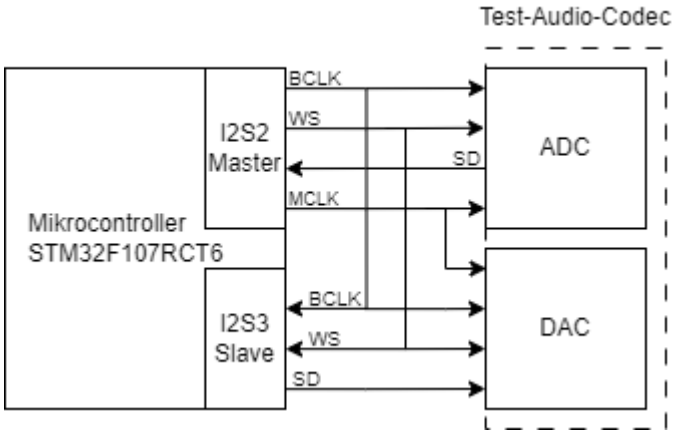



*Abbildung 79: STM32F107RCT6 Master-Slave Blockschaltbild*

Dieses Projekt wurde als Lösung für die Probleme des ersten Projekts erstellt und arbeitet auch noch mit dem Cortex STM32F107RCT6. Der Unterschied liegt darin, dass die Transmit-Schnittstelle jetzt Slave ist. Das heißt, dass WS und BCLK extern an der Transmit-Schnittstelle angelegt werden müssen, damit der Cortex Transmit-Daten erzeugt.

**Idee:** Dadurch reicht eine MCLK die den Audio-Codec versorgt und die I2S3-Schnittstelle (Transmit) erzeugt Daten basierend auf demselben Frame wie die I2S2-Schnittstelle (Receive). Diese Lösung hat im Endeffekt funktioniert.

**Problem**: Es war eine sogenannte „Pfusch-Lösung“, da der Cortex Takt-Signale generiert, die er an Ports ausgibt, nur um diese wieder an eigene Ports zurück-zu-führen.

### *7.4.3.4 Discovery-Board STM32F407G-DISC1*

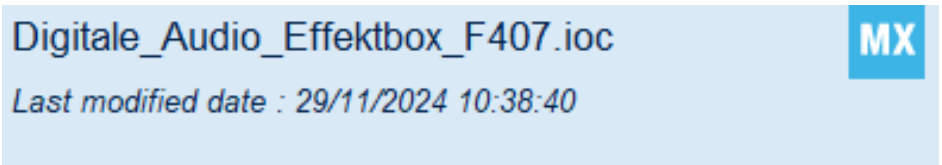



*Abbildung 80: Projekt mit Cortex STM32F407G*

Dieses Projekt wurde mit Cortex STM32F407G erstellt, welcher nicht mit dem MDDS-System verwendet wird, sondern mit einer eigenen Platine. Dieses Projekt wurde nur für Test-Zwecke eines Full-Duplex I2S-Protokoll verwendet und ob der Test-Audio-Codec überhaupt Audio ausgeben und aufnehmen kann.

**Ergebnis:** Das System mit dem Discovery-Board „*STM32F407G-DISC1*“ wurde nur für eine Woche verwendet und nie funktionsfähig gemacht. Stattdessen wurde mit dem Mikrocontroller STM32F303RET6 weitergearbeitet.

### 7.4.4 Projekt Erstellung mit STM32F303RET6 Mikrocontroller

Auf der Startseite kann unter „*New Project*“ ein neues Projekt auf 3 verschiedene Arten erstellt werden.

Für unser Projekt starten wir mit der Auswahl des Mikrocontroller. Dafür wird zuerst auf „*ACCESS TO MCU SELECTOR*“ unter „*Start My project from MCU*“ gewählt.

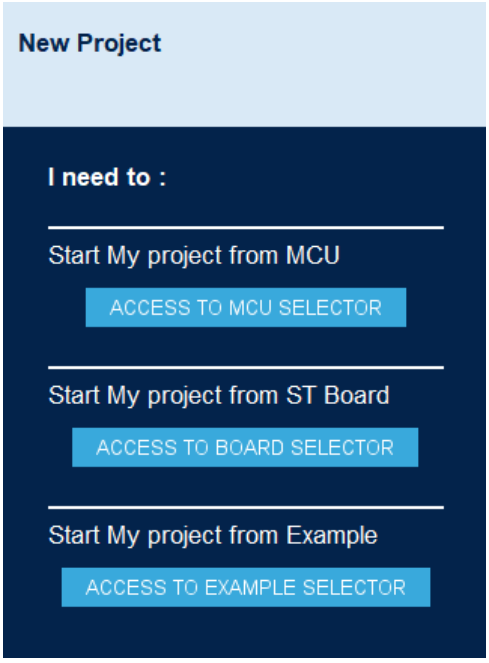



*Abbildung 81: STM32CubeMX neues Projekt*




*Abbildung 82: Mikrocontroller Suche*

Unter dem neu geöffneten Fenster kann links oben unter MCU/MPU Filters nach dem gewünschten Mikrocontroller gesucht werden. In unserem Fall: STM32F303RCT6

Wenn der Mikrocontroller in der Suchleiste eingegeben wurde, erscheinen unten rechts Mikrocontroller, die der Suche entsprechen. Jetzt muss das Feld für den gewünschten Mikrocontroller: STM32F303RCT6 nur noch ge-doppel-klickt werden.




| * | Commercial Part No | Part No | Reference | Marketing ... | Unit Price for 10k... | Board | Package | Flash | RAM | I/O | Frequ... |
|---|---|---|---|---|---|---|---|---|---|---|---|
| ☆ | STM32F303RCT6 | STM32F... | STM32F303R... | Active | 3.7488 | | LQFP 64 10x10x1.4 mm | 256 kB... | 48 kBytes | 52 | 72 MHz |
| ☆ | STM32F303RCT6TR | | STM32F303R... | Active | 3.7488 | | LQFP 64 10x10x1.4 mm | 256 kB... | 48 kBytes | 52 | 72 MHz |

*Abbildung 83: Mikrocontroller Auswahl*

### 7.4.5 Debug

Es sollte unter „*SYS*“ → „*Debug*“ der Punkt Serial Wire eingestellt werden, ansonsten kann der kompilierte Code nicht geflasht werden.

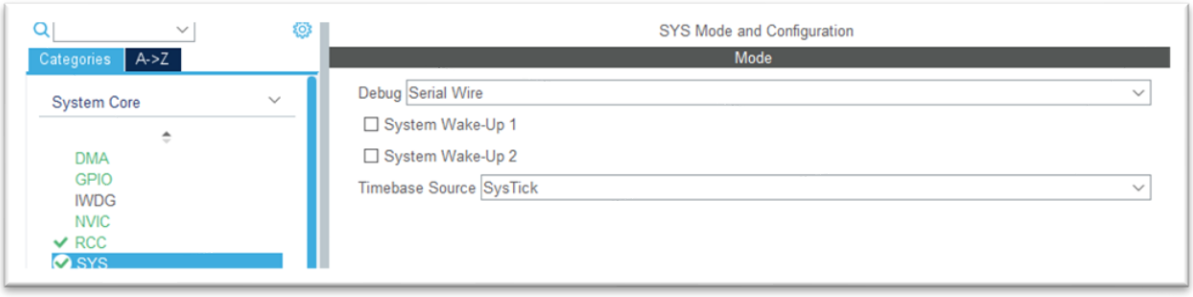



*Abbildung 84: System Konfiguration STM32CubeMX*

Die Pins PA13 und PA14 werden damit als Takt und zur Datenübertragung für Debugging und Flashing verwendet.

### 7.4.6 Clock Konfiguration

Unter „*System Core*“ → „*RCC*“ sollte als Taktversorgung unter „*High Speed Clock*“ Crystal/Ceramic Resonator eingestellt werden.

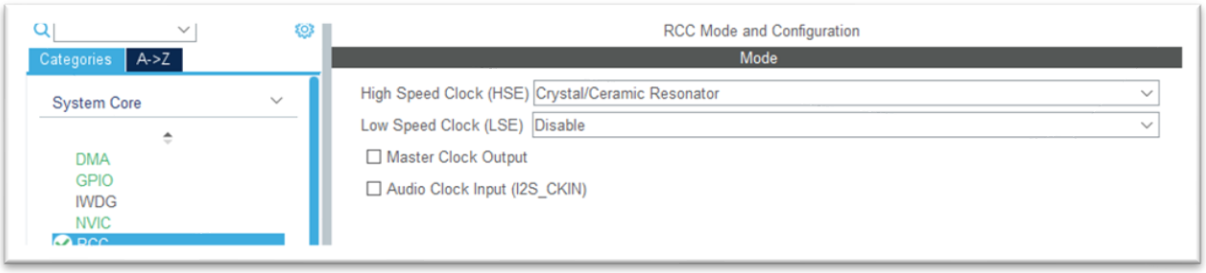



*Abbildung 85: Quarz-Oszillator für Clock-Versorgung STM32CubeMX*

Diese Einstellung ermöglicht es den externen Quarz-Oszillator als Taktversorgung zu verwenden, welcher auf folgenden PINs auf der Mikrocontroller-Platine angelötet ist:

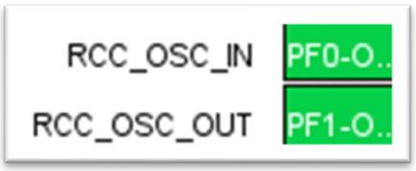



*Abbildung 86: Quarz-Oszillator Pins*

Unter dem Punkt „*Clock Configuration*“ wurden folgende Einstellungen getätigt. Es ist zu sehen, dass am Anfang die HSE-Clock benutzt wurde welche durch PLL (Phase Locked Loop) auf 72Mhz erhöht wurde und diese 72MHz Takt versorgt unter anderem I2S oder USART3, wobei die Frequenz bei USART3 noch halbiert wurde.

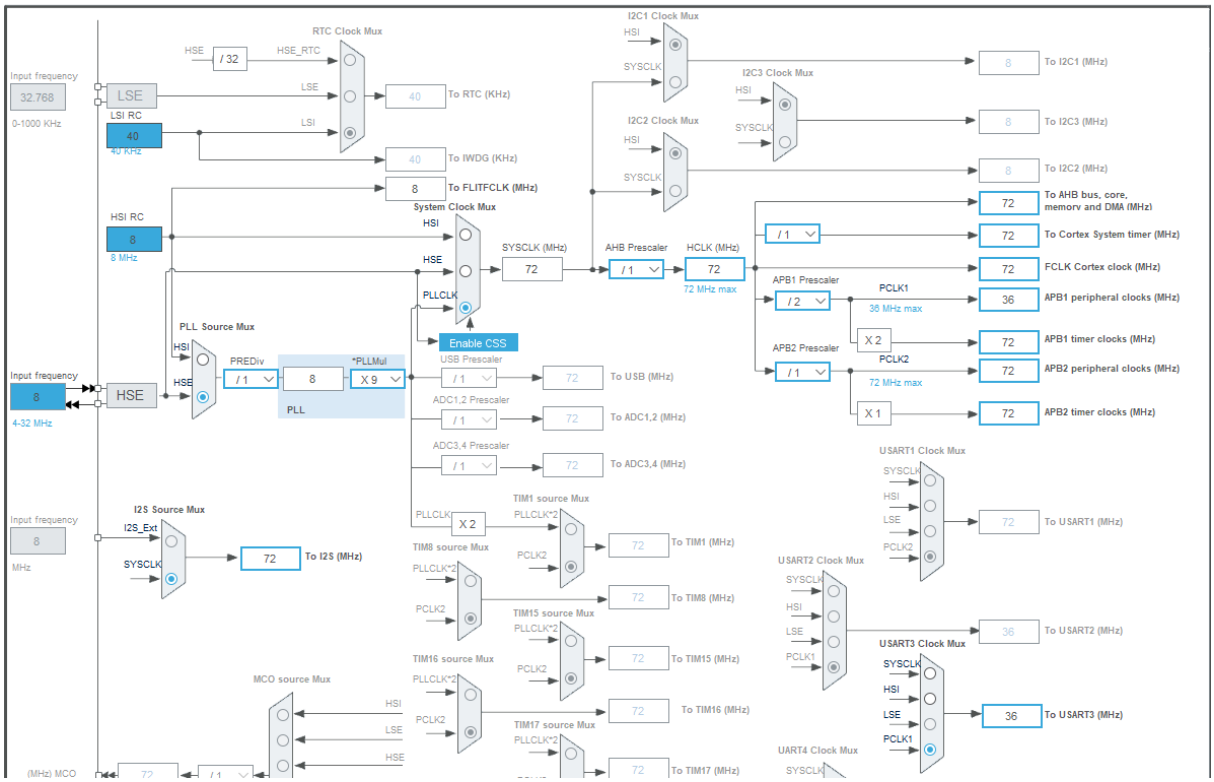



*Abbildung 87: Clock Configuration STM32CubeMX*

## 7.4.7 I2S2 Audio-Protokoll Konfiguration

Mit den folgenden Konfigurationen wird das Audioprotokoll I2S für die Ein- und Ausgabe beschrieben.

Unter „Multimedia“ können zwei I2S-Schnittstellen eingestellt werden. In unserem Fall reicht eine, da diese mit dem Mikrocontroller STM32F303RET6 Full-Duplex sein kann.

Ob eine Master-Clock notwendig ist, kommt auf den verwendeten Audio-Codec an. In unserem Fall ist die Master-Clock nötig.

Die generischen Parameter des I2S-Protokoll wurden, wie im Kapitel 3.1 beschrieben ist, gewählt.

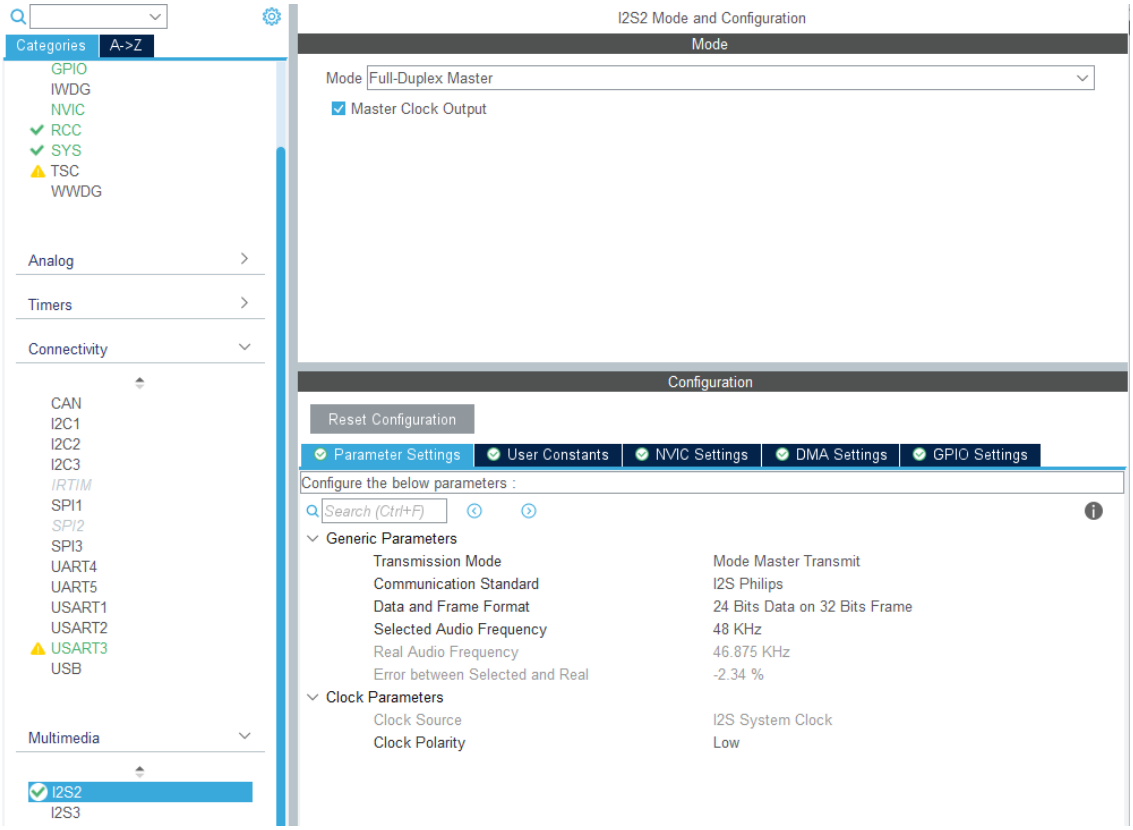



*Abbildung 88: I2S Parameter Settings*

Unter „*DMA Settings*“ werden zwei DMA’s konfiguriert, welche jeweils Eingangs- und Ausgangs-Audio-Samples kopieren. Wie genau das funktioniert wird im Kapitel 7.6.1 beschrieben.

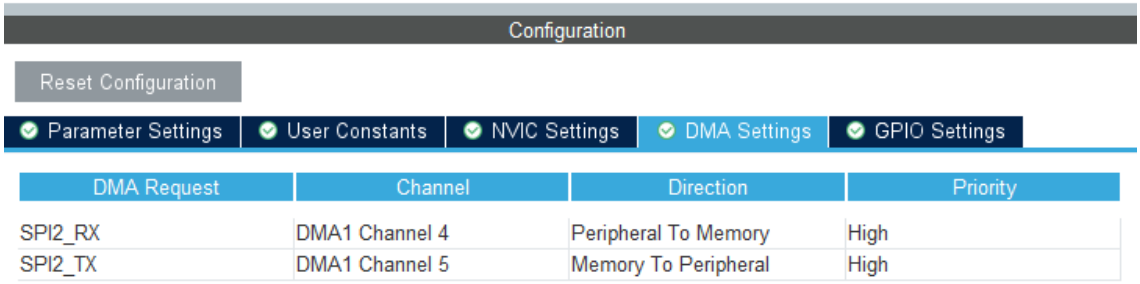



*Abbildung 89: I2S DMA Konfiguration*

Unter „NVIC Settings“ wird für den Receive-DMA der Interrupt aktiviert und für den Transmit-DMA deaktiviert. Wie und wofür der Interrupt verwendet wird, wird im Kapitel 7.6.2 beschrieben.

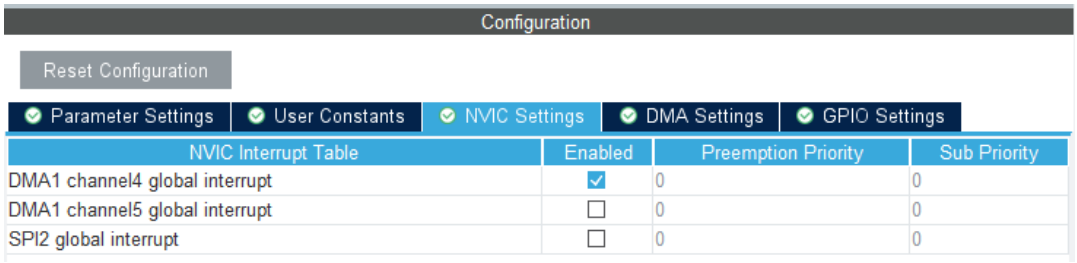



*Abbildung 90: I2S NVIC Konfiguration*

Beide DMA's müssen noch unter „*System Core*“ → „*DMA*“ konfiguriert werden. Dabei haben beide dieselben Einstellungen wie hier beschrieben:

Der Modus muss auf „Circular“ gestellt werden, damit die DMA's wieder von vorne beginnen, wenn das Kopieren des Buffer fertig ist. Dadurch werden kontinuierlich Audio-Daten kopiert.

Das Datenregister SPI_DR ist nur 16bit groß, aber unsere Wortbreite hat 24bit Auflösung. Es wird mit einer Kopier-Aktion also nur die Hälfte des Audio-Samples kopiert und deshalb muss „Half Word“ ausgewählt werden.

Bei Increment Address muss Memory markiert werden und Peripheral nicht. Das liegt daran, dass im RAM mehrere Speicherzellen für Audio-Samples hintereinander stehen im Gegensatz zum Periphere-Register, wo nur 16bit Platz ist.

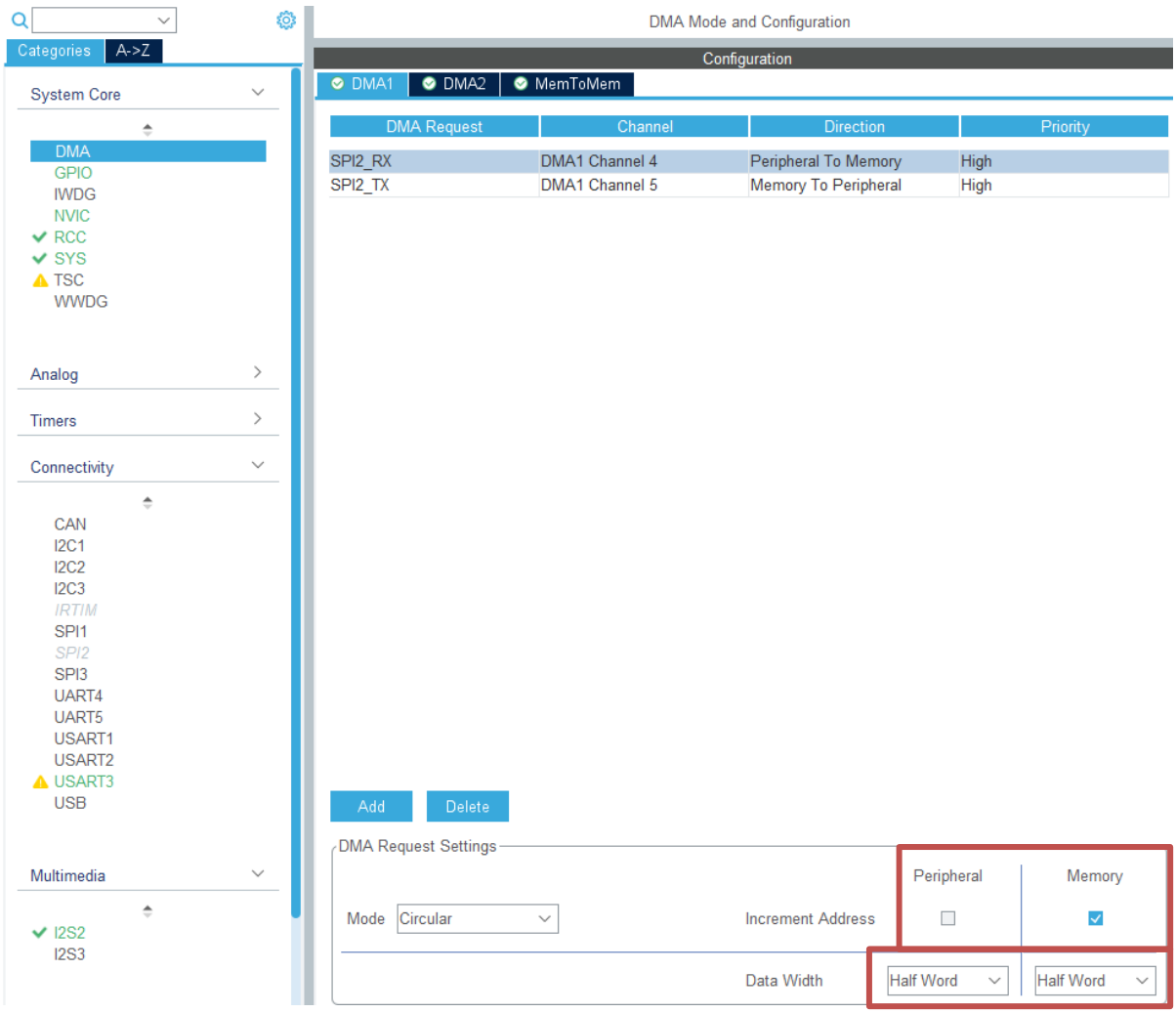



*Abbildung 91: DMA Konfiguration*

Unter „*GPIO*“ → „*I2S*“ werden die verwendeten PINs für die I2S-Schnittstelle angezeigt.

I2S2_ext_SD ist der Port, der Eingangs-Audio Daten einliest und wurde als No pull-up definiert, da der Audio-Codec diese Daten generiert. Alle anderen PINs sind als Pull-down definiert, um jederzeit einen definierten Zustand zu haben.

Unter „Maximum Output Speed“ wird noch die maximale Änderungsgeschwindigkeit des Ports beschrieben. MCLK hat zum Beispiel mit 12MHz Takt die höchste Frequenz und es wurde deshalb „High“ ausgewählt.

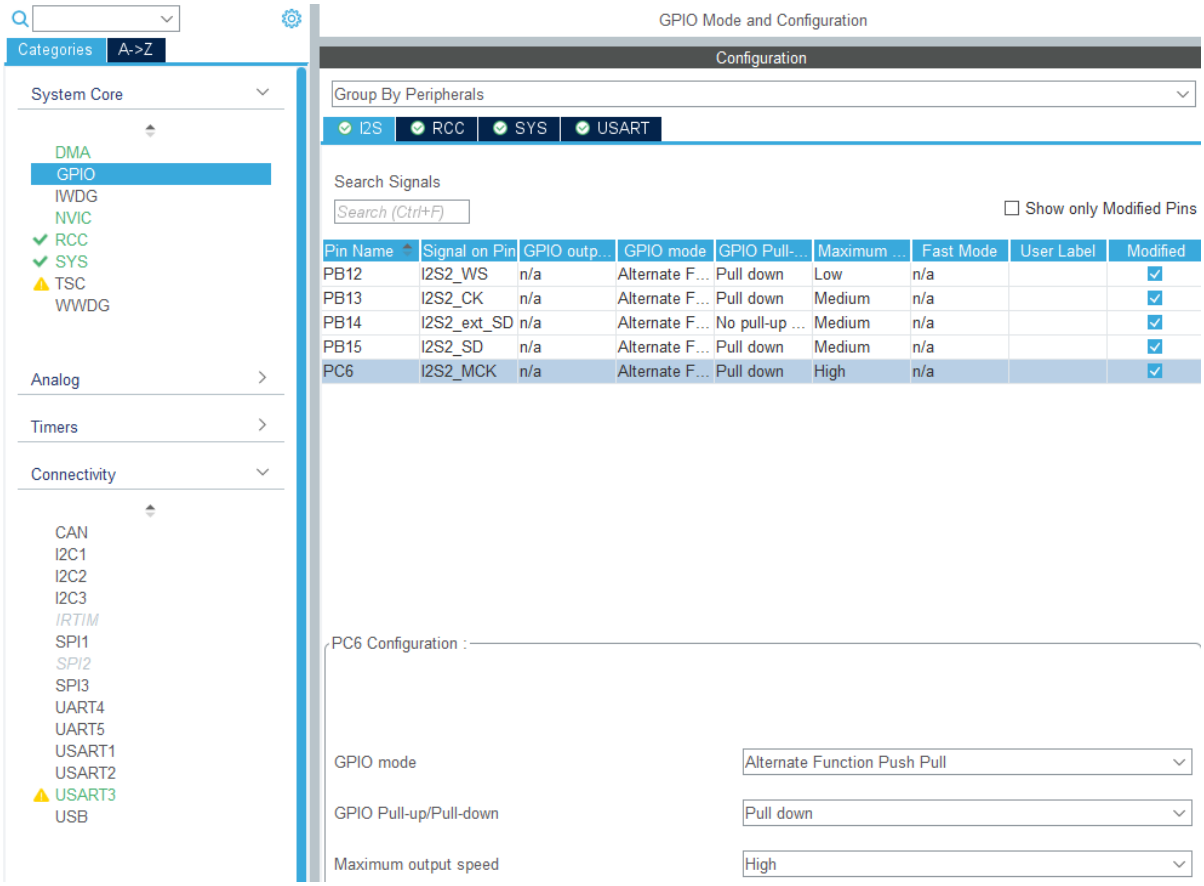



*Abbildung 92: GPIO Konfiguration*

### 7.4.8 USART3 Konfiguration

Um mit dem Nextion-Display zu kommunizieren, muss USART3 konfiguriert werden.

Der Modus ist „Asynchronous“. Das bedeutet, dass kein synchronisierender Takt existiert, sondern nur eine Transmit- und Receive-Leitung.

„*Hardware Flow Control*“ kann nicht verwendet werden, da das I2S-Protokoll PINs verwendet, die von „Hardware Flow Control“ auch benutzt werden.

Unter Parameter Settings werden noch Basic Parameter eingestellt, wie im Kapitel 7.8.3 USART3 Kommunikation beschrieben ist.

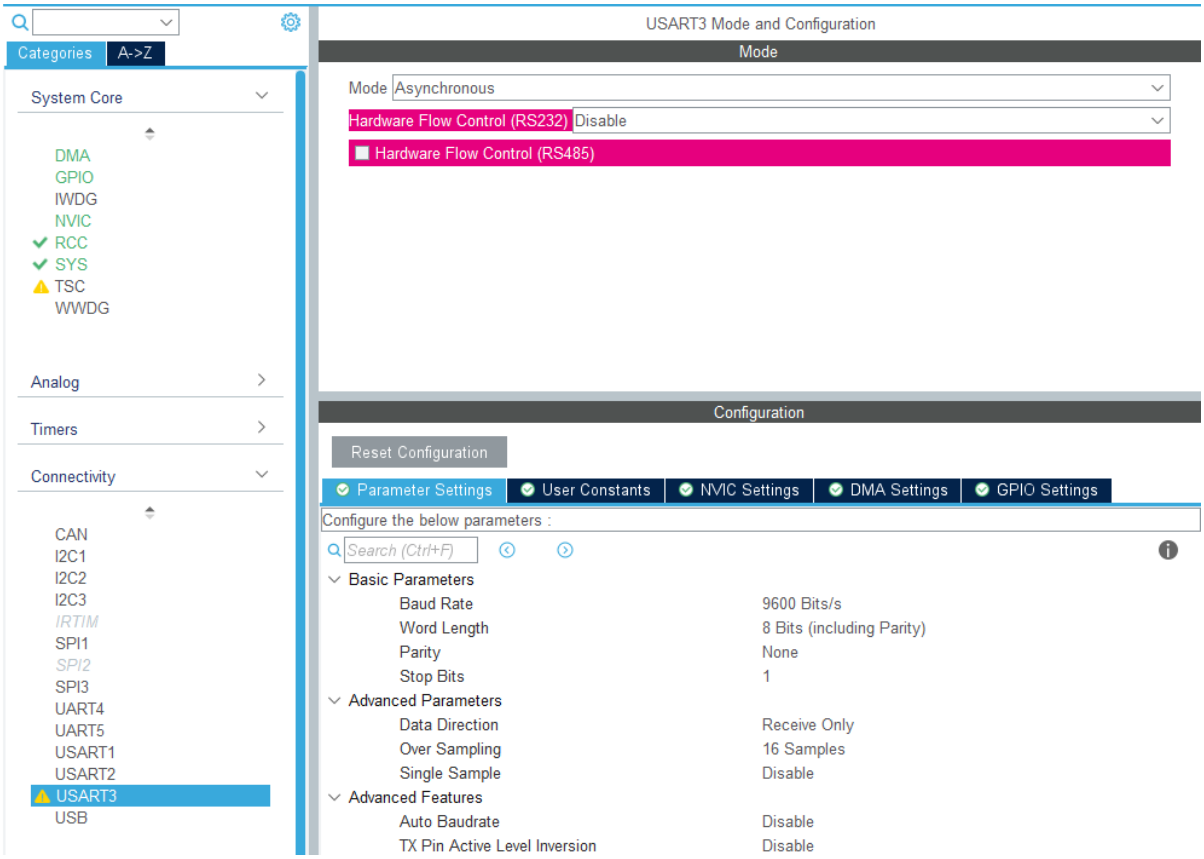



*Abbildung 93: USART3 Konfiguration*

### 7.4.9 Pinbelegung des Cortex

Hier ist noch ein allgemeiner Überblick über alle benutzten Pins des Cortex STM32F303RET6:

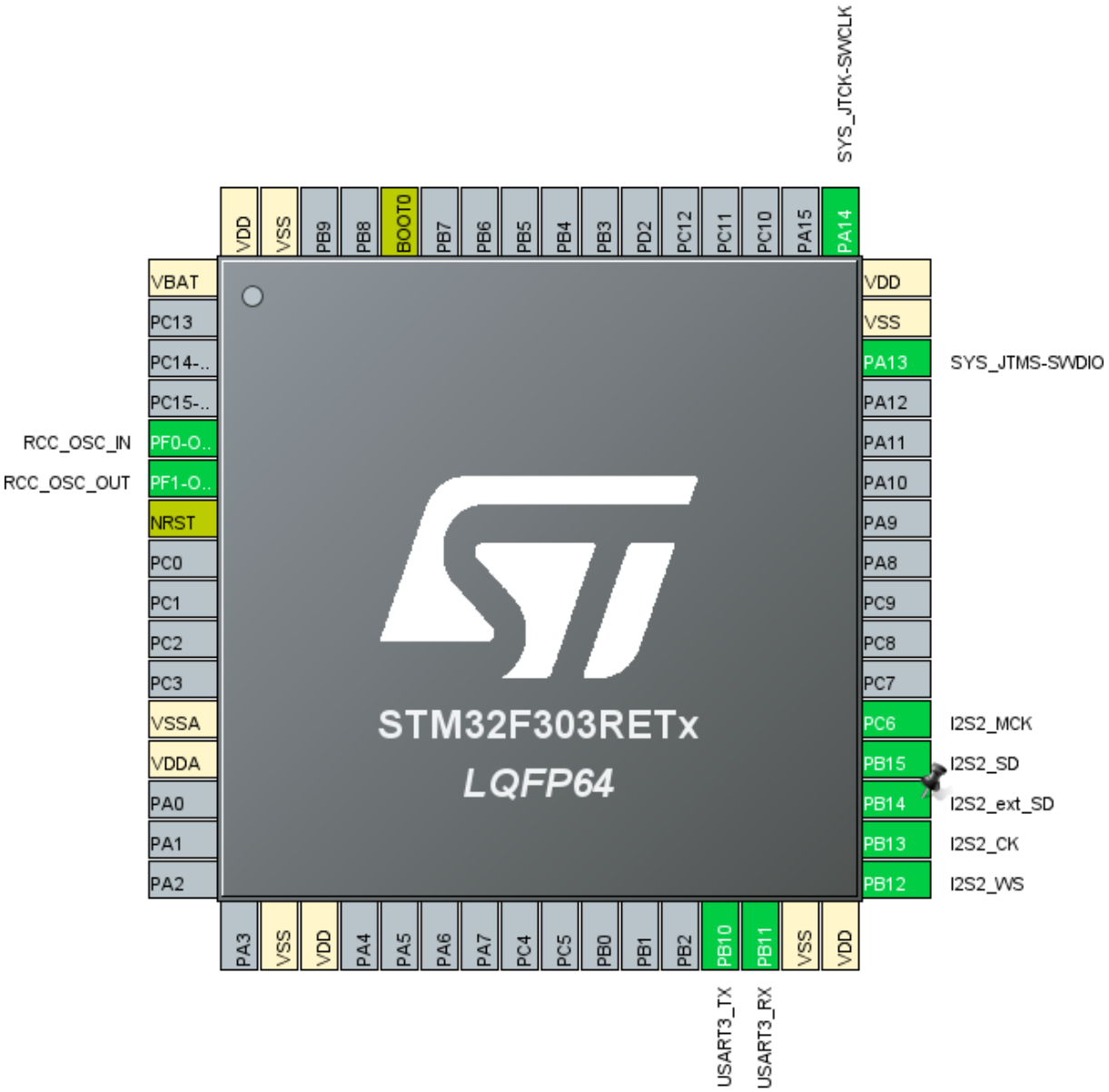



*Abbildung 94: Pinout*

## 7.4.10 Code Generation

Unter „*Project Manager*“ werden noch generische Einstellungen wie der Projekt-Name oder der Projekt-Speicherort angegeben. Wichtig ist noch unter „*Toolchain / IDE*“ MDK-ARM einzustellen.

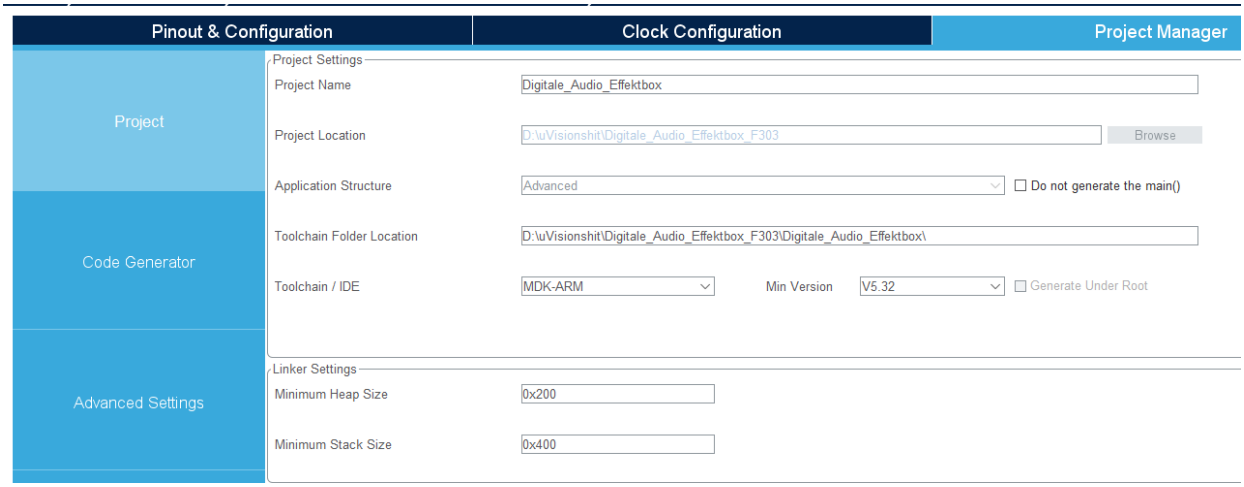



*Abbildung 95: Project Manager Menü*

Danach muss nur noch auf „Generate Code“ rechts oben geklickt werden. Dadurch wird Code generiert, der auf Grundlage der getätigten Einstellungen basiert.




*Abbildung 96: Generate Code Button*

## 7.6 Framework Audioverarbeitung

### 7.6.1 DMA

Bevor die Audioverarbeitung genauer erläutert werden kann, muss der oft verwendete Begriff DMA beschrieben werden. Dieser steht für „*Direct Memory Access*“ und ermöglicht es Daten zwischen RAM und Peripherie zu kopieren, ohne dass die CPU dabei eingreifen muss. Außerdem kann der DMA-Controller interrupts zu bestimmten Zeitpunkten auslösen. z.B.: Wenn die Kopier-Aktion abgeschlossen ist.

### 7.6.2 Interrupt

Ein weiterer Begriff, der oft verwendet wird, ist „Interrupt“. Dieser beschreibt das Unterbrechen der CPU, bei was auch immer sie gerade rechnet. Dazu wird eine sogenannte „*ISR*“ oder „*Interrupt-Service-Routine*“ aufgerufen, die das aktuelle Programm ersetzt und wenn diese abgearbeitet ist, springt die CPU wieder zurück, wo sie aufgehört hat. Interrupts haben auch eine Priority, je nachdem welcher Interrupt wichtiger ist. Wenn ein neuer Interrupt eine niedrigere Priority hat, unterbricht dieser den aktuellen Interrupt.

### 7.6.3 Grundprinzip

Um eine möglichst effiziente Audio-Verarbeitung zu realisieren, wurde das Konzept des Ping-Pong-Buffer-Audio-Stream entwickelt. Dabei kann der Mikrocontroller permanent seinen Prozessor damit beschäftigen Audio zu verarbeiten, da der Prozessor unabhängige DMA-Controller die neuen Audio-Samples von der Peripherie in den RAM und verarbeitete Audio-Samples aus dem RAM zur Peripherie kopiert.

Da das Datenregister SPI_DR für eingehende und ausgehende Audio nur 16bit groß ist, muss, um ein Audio-Sample zu übertragen, zwei Mal in das Register geschrieben werden. Deshalb ist der Receive- und Transmit-Buffer auch ein Vielfaches von 16bit groß, um auf das Register schreiben zu können. Da der ADC und DAC 24bit Auflösung haben, werden 2 x 16bit übertragen, um an ein Audio-Sample zu kommen. Da jetzt aber auch noch zwischen linkem und rechtem Ohr unterschieden wird, brauchen wir 4 x 16bit Speicher. Der DMA-Controller darf aber nicht auf den Speicherbereich zugreifen, mit dem die CPU arbeitet, deshalb wird insgesamt für Receive und Transmit ein Buffer von 8x16bit angelegt. Die CPU und der DMA-Controller wechseln sich ab auf welche Hälfte sie zugreifen dürfen.

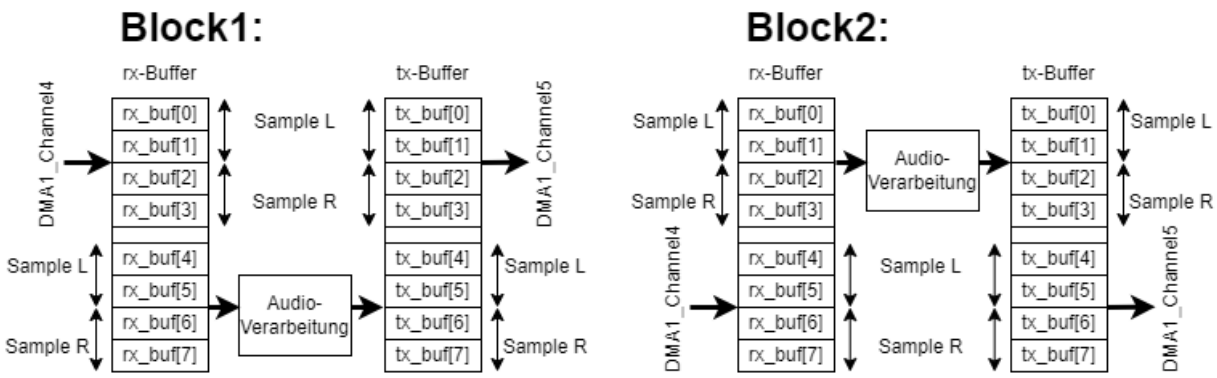



*Abbildung 97: Ping-Pong buffer-audio-stream*

Sample L und Sample R sind die Audio-Samples, die entweder für das linke oder das rechte Ohr gesendet werden. Wie zu sehen ist wechselt sich das Kopieren ab zwischen Left-Sample und Right-Sample.

In **Block1** ist zu sehen, dass der DMA Daten in den Lower-Buffer (0-3) von tx kopiert und aus dem Lower-Buffer von rx kopiert, während die CPU Audio-Samples des Higher-Buffer von rx verarbeitet und in den Higher-Buffer von tx speichert.
In **Block2** passiert dasselbe, nur dass der DMA Higher-Buffer-Daten kopiert und die CPU Lower-Buffer-Daten verarbeitet.

Mit dieser Konfiguration kann der Prozessor seine Leistung vollkommen auf das Verarbeiten von Audio-Samples richten. Er darf aber für eine Hälfte des Buffer nicht länger brauchen, als der DMA seine Hälfte des Buffer kopiert, was 2 * 1/48kHz = 41.66us sind.

Visualisierung des Kopierens des DMA-Controller:

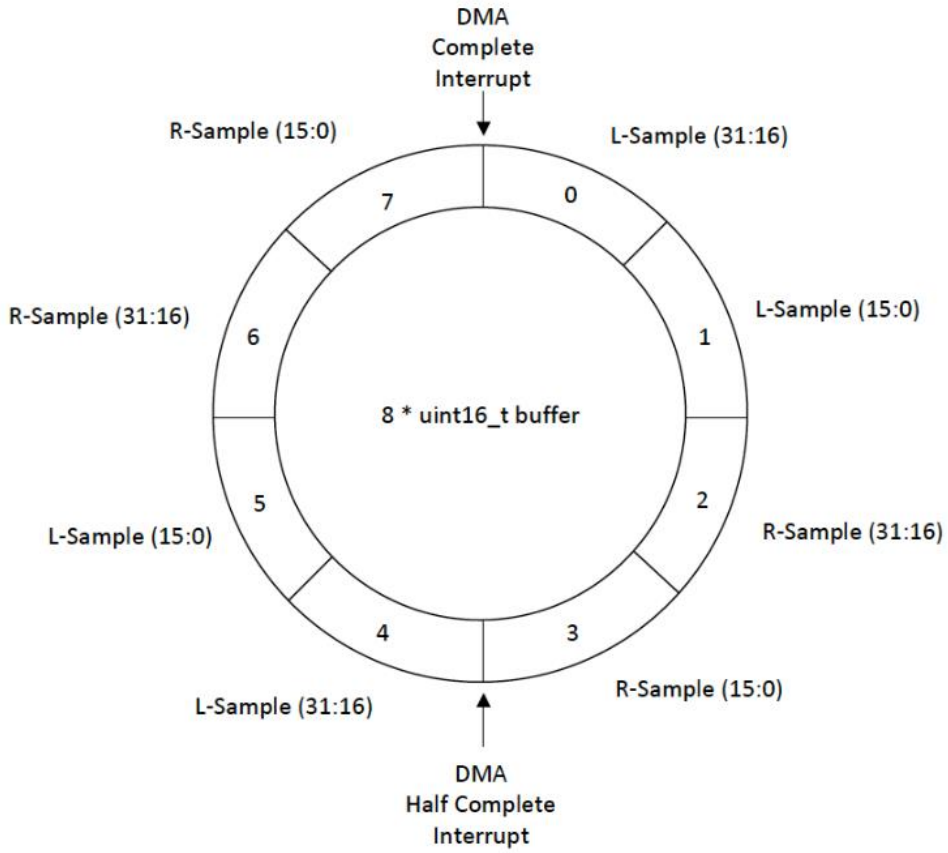



*Abbildung 98: Ping-Pong buffer Kreis-Diagramm [AUD25]*

Dieses Diagramm beschreibt einen 8*uint16_t buffer, welcher entweder rx_buf oder tx_buf sein kann. Für beide gilt dasselbe Prinzip. Der Ring beschreibt das kontinuierliche Kopieren der Audio-Samples des DMA im Uhrzeigersinn. Jedes Audio-Sample besteht wie beschrieben aus zwei 16bit Buffern. Dabei werden die höher-wertigen 16bit (msb) zuerst kopiert. Wenn der DMA den Buffer von 0-3 kopiert hat bzw. 1 Left- und Right-Sample kopiert hat, wird der „*DMA Half Complete Interrupt*“ ausgelöst. Durch diesen Interrupt weiß die CPU, dass die untere Hälfte des Buffer gerade kopiert worden ist und dass diese Daten frei zum Verarbeiten sind. Während sich die CPU also um den unteren Teil des Buffer kümmert (0-3) kopiert der DMA weiterhin 16bit Blöcke (4-7) bis der „*DMA Complete Interrupt*“ ausgelöst wird. Dadurch verarbeitet die CPU die Daten der oberen Hälfte des Buffer und der DMA fängt das Kopieren wieder von vorne an. Dadurch wechseln sich CPU und DMA wie beschrieben kontinuierlich ab auf welchen Bereich des Speichers sie zugreifen, damit keine Datenkohärenzprobleme auftreten.

### 7.6.4 Allgemeiner Ablauf

Wie in diesem Fluss-Diagramm zu sehen ist, wird der Prozess ausschließlich durch Interrupts gesteuert, da das Programm nach Initialisierungen in einer Endlosschleife bleibt.

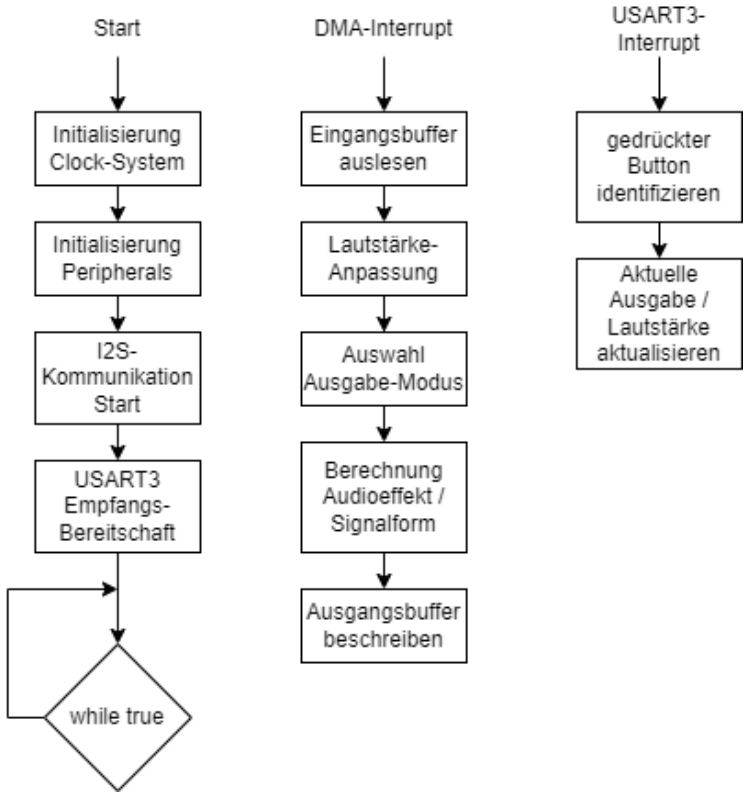



*Abbildung 99:Allgemeiner Software Ablauf Flussdiagramm*

### 7.6.5 Interrupt Auslösung

Hier ist zu sehen, wie welche Interrupts des Programms ausgelöst werden.

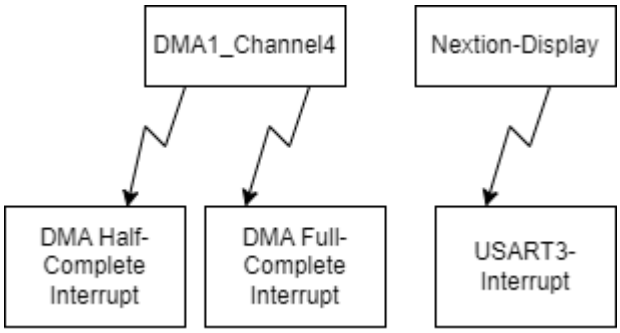



*Abbildung 100: Interrupt Auslösung*

Die Audioverarbeitung beginnt also ausschließlich dann, wenn der DMA1_Channel4 (zuständig für die eingehenden Audio-Samples) zur Hälfte und ganz fertig ist. Wenn der Half-Complete-Interrupt ausgelöst ist, weiß der Prozessor, dass der Lower-Buffer jetzt bereit ist zu bearbeiten, genauso wie bei dem Full-Complete-Interrupt der Higher-Buffer.

### 7.6.6 Erstelltes Projekt

Wie im Kapitel Konfiguration-Mikrocontroller schon beschrieben, hat STM32CubeMX ein Projekt erstellt in dem die Konfigurationen für die I2S-Schnittstelle, DMA, Clock-Versorgung und UART3-Kommunikation schon definiert sind. Für alle Konfigurationen werden Funktionen aus der HAL-Library benutzt, welche diverse Register setzen auf Basis von den gewünschten, angegebenen Eigenschaften.

### 7.6.7 Main.c

Jeder folgende Code ist aus dem Main.c-file des erstellten STM32CubeMX-Projekts.

Dieser Ausschnitt beschreibt den wichtigsten Teil des `int main(void)`. Es sind einige Kommentare, die den Code-Beginn / das Code-Ende darstellen und von STM32CubeMX erstellt worden sind, weggelassen worden. Des Weiteren werden noch Werte von ein paar Variablen gesetzt, die aber gerade nicht relevant sind. Diese Maßnahme dient zur Übersichtlichkeit des gezeigten Codes.

```
299. int main(void)
300. {
308.   /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
309.   HAL_Init();
310.
315.   /* Configure the system clock */
316.   SystemClock_Config();
317.
322.   /* Initialize all configured peripherals */
323.   MX_GPIO_Init();
324.   MX_DMA_Init();
325.   MX_I2S2_Init();
326.   MX_USART1_UART_Init();
327.   MX_USART3_UART_Init();
328.   // start I2S and DMA Transmission
329.   HAL_I2SEx_TransmitReceive_DMA(&hi2s2, tx_buf, rx_buf, 4);
330.
331.   HAL_UART_Receive_IT(&huart3, rxData, 7); // start waiting for UART3 messages
332.
350.   /* Infinite loop */
352.   while (1)
353.   {
355.
357.   }
359. }
```

Es werden HandleTypeDef-Strukturen definiert in denen die Konfigurationen der beiden DMA's und der I2S-Schnittstelle gespeichert werden. Im `int main(void)` werden zuerst HAL und Clocks initialisiert. Danach werden auch schon die gewünschten Peripherien initialisiert. Diese Funktionen wurden von STM32CubeMX erstellt auf Basis von den gewünschten Eigenschaften der Peripherie.

`HAL_I2SEx_TransmitReceive_DMA(&hi2s2, tx_buf, rx_buf, 4);`
dieser Aufruf beginnt die I2S-Übertragung und aktiviert die DMA's welche für einen Kopier-Kreislauf 4 Audio-Samples senden, wie mit der 4 angegeben ist. Dadurch ist der Half-Complete-Callback nach einem Left- und Right-Audio-Sample und der Full-Complete-Callback nach zwei, wie vorher beschrieben.

```
HAL_UART_Receive_IT(&huart3, rxData, 7);
```

Ein weiterer wichtiger Aufruf, wobei auf 7Byte UART3-Daten gewartet werden und wenn diese 7 Byte eingetroffen sind, wird ein Interrupt ausgelöst.

## 7.6.8 Interrupt-Service-Routinen

### *7.6.8.1 DMA-Interrupt*

Diese Funktion wird aufgerufen, wenn der DMA die untere Hälfte des Receive-Buffer befüllt hat.

```
715. void HAL_I2SEx_TxRxHalfCpltCallback(I2S_HandleTypeDef *hi2s)
716. {
717.     left_in = (((int)rx_buf[0]<<16)|rx_buf[1])>>8;  // Bit-Shift the value of one Sample
718.     right_in = (((int)rx_buf[2]<<16)|rx_buf[3])>>8; // from 2*16bit to one 32bit integer
719.
720.     calculate_Volume();
721.     calculate_Output();
722.
723.     // put out the values
724.     tx_buf[0] = (left_out>>8)&0xFFFF;  // Bit-Shift the value of one Sample back from
725.     tx_buf[1] = (left_out<<8)&0xFF00;  // 32bit to 2*16bit
726.     tx_buf[2] = (right_out>>8)&0xFFFF;
727.     tx_buf[3] = (right_out<<8)&0xFF00;
728. }
```

Diese Funktion wird aufgerufen, wenn der DMA die obere Hälfte des Receive-Buffer befüllt hat.

```
730. void HAL_I2SEx_TxRxCpltCallback(I2S_HandleTypeDef *hi2s)
731. {
732.     left_in = (((int)rx_buf[4]<<16)|rx_buf[5])>>8;  // Bit-Shift the value of one Sample
733.     right_in = (((int)rx_buf[6]<<16)|rx_buf[7])>>8; // from 2*16bit to one 32bit integer
734.
735.     calculate_Volume();
736.     calculate_Output();
737.
738.     // put out the values
739.     tx_buf[4] = (left_out>>8)&0xFFFF;  // Bit-Shift the value of one Sample back from
740.     tx_buf[5] = (left_out<<8)&0xFF00;  // 32bit to 2*16bit
741.     tx_buf[6] = (right_out>>8)&0xFFFF;
742.     tx_buf[7] = (right_out<<8)&0xFF00;
743. }
```

Beide Funktion sind fast identisch und funktionieren auch gleich außer, dass entweder die obere Hälfte oder die untere Hälfte des Receive-Buffer verarbeitet wird.
Die Audio-Samples sind noch in zwei 16bit Buffer unterteilt. Um die Werte leichter weiter-zu-verarbeiten zu können, werden die beiden Buffer durch Bit-Shifting zu einem 32bit integer zusammengefügt (left_in und right_in).

Jetzt werden die Audio-Samples mit der eingestellten Lautstärke skaliert. Dafür wird die Funktion `calculate_Volume();` aufgerufen.

Die Funktion `calculate_Output();` berechnet dann die Ausgangs-Werte je nach eingestelltem Modus.

### *7.6.8.2 UART3-Interrupt*

Wenn der Benutzer einen Modus auswählt bzw. die Lautstärke angepasst hat, wird ein UART3-Interrupt ausgelöst. Bei diesem werden die empfangenen Daten je nach gesendeter Component-ID ausgewertet, um zu wissen, was der Benutzer gedrückt hat.

Der Aufruf `memcmp(rxData, SaegezahnID, 4)` vergleicht die empfangenen Daten mit der Component-ID von zum Beispiel dem Sägezahn-Signalgenerator. Ist diese gleich setzt das Programm die aktuelle Ausgabe auf Sägezahn.

Bei dem Audio-Effekt „frequency_split“ wird zusätzlich noch unterschieden, ob der Button zwei Mal gedrückt wurde und wechselt auf welchem Ohr tiefe bzw. hohe Frequenzen ausgegeben werden.

Wenn erkannt wird, dass die Lautstärke geregelt wird, ist es nicht genug zu wissen, dass sie geändert wird. Es muss auch übermittelt werden, welche neue Lautstärke gewünscht ist. Deshalb wird der Befehl, auf eine weitere UART3-Nachricht zu warten, aufgerufen und da diese dann nur eine Zahl und keine Komponente-ID ist, weiß das Programm, dass die Lautstärke auf diese Daten angepasst werden soll.

```
551. void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
552.     if (huart->Instance == USART3) {  // Check if it's USART3
553.               HAL_UART_Transmit(&huart1, rxData, 7, HAL_MAX_DELAY);
554.               if (memcmp(rxData, SaegezahnID, 4) == 0)
555.               {
556.                         aktuelle_Ausgabe = Saegezahn;
557.               }
558.               else if (memcmp(rxData, DreieckID, 4) == 0)
559.               {
560.                                   aktuelle_Ausgabe = Dreieck;
561.               }
574.               else if (memcmp(rxData, frequency_splitID, 4) == 0)
575.               {
576.                         if(aktuelle_Ausgabe == frequency_split)
577.                         {
578.                                   if(isLeftHigh == 't')
579.                                   {
580.                                             isLeftHigh = 'f';
581.                                   }
582.                                   else
583.                                   {
584.                                             isLeftHigh = 't';
585.                                   }
586.                         }
587.                         else
588.                         {
589.                                   aktuelle_Ausgabe = frequency_split;
590.                         }
591.               }
701.               else if (memcmp(rxData, SlideBarID, 4) == 0)
702.               {
703.                         HAL_UART_Receive_IT(&huart3, rxData, 1);
704.               }
705.               else                // must be be the value of slide bar
706.               {
707.                         SlideBarValue = rxData[0];
708.                         SlideBarValueFraction = (float)SlideBarValue / (float)0x64;
709.                         volumeAdjustmentFactor = 2 * SlideBarValueFraction;
710.               }
711.         HAL_UART_Receive_IT(&huart3, rxData, 7);  // Restart interrupt reception
712.     }
713. }
```

**Bemerkung:** In Zeile 553 wurde die empfangene UART3-Nachricht über UART1 gesendet, um die UART3-Nachricht für Testzwecke am PC anzeigen zu können. In dem UART3-Interrupt Code wurde ein Teil an Code weggelassen, um die Dokumentation übersichtlicher zu gestalten und Stellen, die gleich funktionieren, nicht zu zeigen.

### 7.6.9 Ausgabe-Modus

Wie in vorherigem Kapitel zu sehen war, gibt es eine Variable, die den aktuellen Modus steuert und diese wird über den UART-Interrupt geändert. In der Funktion `calculate_Output();` wird nun anhand dieser Variable, der relevante Code-Abschnitt für den Audio-Effekt oder den Signalgenerator ausgewählt. Code der Funktion `calculate_Output();` wird über mehrere Kapitel beschrieben, da darin alle Signalgeneratoren codiert sind. Pitch-Shift und Reverb haben ihre eigenen Unterprogramme die aber auch in `calculate_Output();` aufgerufen werden.

### 7.6.10 Lautsärkenregelung

Die Funktion `calculate_Volume();` skaliert die Eingangs-Audio-Samples je nach `volumeAdjustmentFactor` der vom Benutzer durch den Regler eingestellt wird. Der Faktor kann von 0 – 2, also von kein Signal bis zur Verdoppelung der Amplitude des Signals gehen.

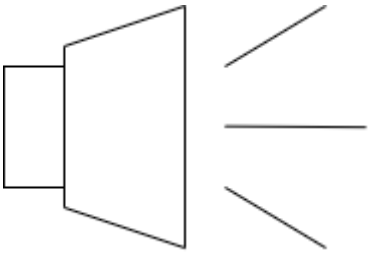


*Abbildung 101: Lautstärken-Symbol*

```
745. void calculate_Volume(void)
746. {
747.     left_in = (int32_t)(left_in * volumeAdjustmentFactor);
748.     right_in = (int32_t)(right_in * volumeAdjustmentFactor);
749.
750.     if (left_in > 0x007FFFFF)
751.     {
752.               left_in = 0x007FFFFF; // Clamp to max positive value
753.     }
754.     else if (left_in < (int32_t)0xFF800000)
755.     {
756.               left_in = (int32_t)0xFF800000; // Clamp to neg positive value
757.     }
758.
759.     if (right_in > 0x007FFFFF)
760.     {
761.               right_in = 0x007FFFFF; // Clamp to max positive value
762.     }
763.     else if (right_in < (int32_t)0xFF800000)
764.     {
765.               right_in = (int32_t)0xFF800000; // Clamp to neg positive value
766.     }
767. }
```

In Zeile 747 und 748 werden jeweils das Left- und Right-Audio-Sample mit dem Faktor skaliert.

Danach wird noch überprüft, ob die maximale positive Zahl überschritten bzw. die maximale negative Zahl unterschritten wird. Wenn ja, wird der Wert auf den maximalen positiven bzw. maximalen negativen Wert begrenzt.

# 7.7 Signalgeneratoren

## 7.7.1 Allgemein

Signalgeneratoren verarbeiten im Gegensatz zu Audio-Effekten keine Eingangs-Audio, sondern generieren diverse Signale, die von einem Oszilloskop gemessen werden können. Diese sind nicht wirklich dazu gedacht sich anzuhören.

**Bemerkung:** Bei den generierten Signalen gibt es im Spannungsverlauf Rauschen. Dieses kann software-technisch schwer optimiert werden.

## 7.7.2 Sägezahn

Das Sägezahn-Signal ist dadurch charakterisiert, dass der Spannungswert linear ansteigt, bis er an den maximalen Punkt gelangt, wonach er von dem negativsten Wert wieder linear ansteigt.

Code für Sägezahn-Signal:

```
773.    if(j<0x00FF0000)
774.    {
775.            j=j+0x10000;
776.    }
777.    else
778.    {
779.            j = 0x00000000;
780.    }
781.    left_out = j;
782.    right_out = j;
```

Die Zähler-Variable j zählt von 0 bis 0x00FF0000 und danach wieder von vorne. Der positive Bereich geht von 0-0x007F0000 und der negative Bereich von 0x00800000 bis 0x00FF0000 Der negative Bereich beginnt ab 0x00800000, da die Daten in der Zweierkomplementen-Darstellung übertragen werden.

Das gemessene Sägezahn-Signal mit Oszilloskop:

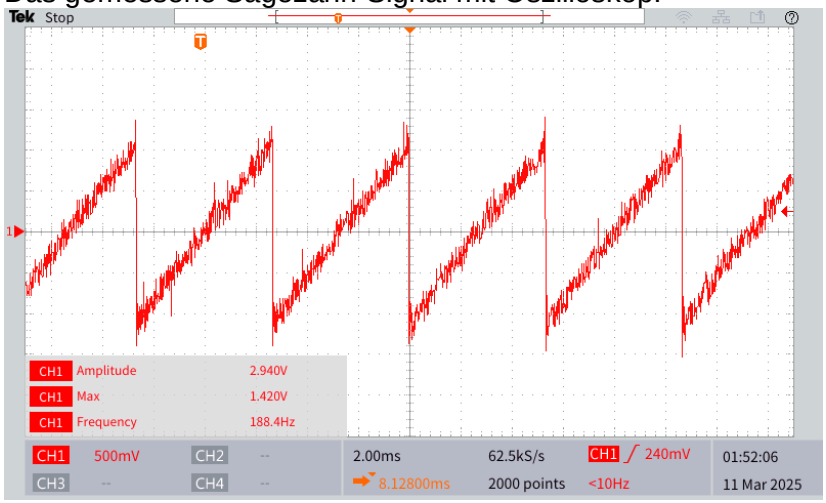



*Abbildung 102: Sägezahn-Signal*

Frequenz-Formel: Abtastrate / 255 = 48kHz / 255 = 188,235Hz.
Frequenz gemessen: 188,4Hz.

### 7.7.3 Dreieck

Das Dreieck-Signal ist dadurch charakterisiert, dass der Spannungswert linear ansteigt, bis er an den maximalen Punkt gelangt, wonach er wieder linear absteigt, bis er den negativsten Wert erreicht hat. Danach steigt der Spannungswert wieder linear an.

Code für Dreieck-Signal:

```
785.     if(dreieckUpCount == 't')
786.     {
787.               if(j==0x007F0000)
788.               {
789.                         dreieckUpCount = 'f';
790.                         j=j-0x10000;
791.               }
792.               else if(j==0x00FF0000)
793.               {
794.                         j=0x00000000;
795.               }
796.               else
797.               {
798.                         j=j+0x10000;
799.               }
800.     }
801.     else
802.     {
803.               if(j==0x00800000)
804.               {
805.                         dreieckUpCount = 't';
806.                         j=j+0x10000;
807.               }
808.               else if(j==0x00000000)
809.               {
810.                         j=0x00FF0000;
811.               }
812.               else
813.               {
814.                         j=j-0x10000;
815.               }
816.     }
817.     left_out = j;
818.     right_out = j;
```

Hier wird durch die Variable `dreieckUpCount` unterschieden, ob der Spannungswert gerade steigt oder fällt. Danach wird überprüft ob das Maxima erreicht ist, um die Steigung umzudrehen. Wenn nicht, wird der Zählerstand vergrößert bzw. verkleinert.

Das gemessene Dreieck-Signal mit Oszilloskop:

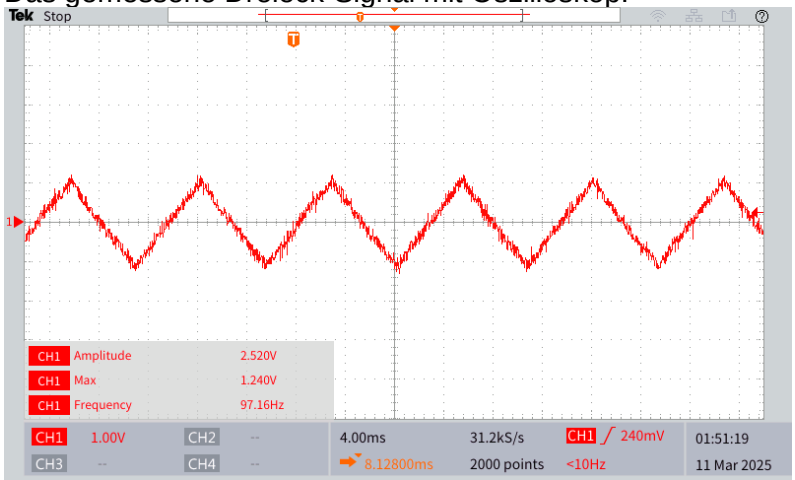



*Abbildung 103: Dreieck-Signal*

Frequenz-Formel: Abtastrate / 510 = 48kHz / 510 = 94,117Hz

Frequenz gemessen: 97,16Hz

### 7.7.4 Rechteck

Das Rechteck-Signal ist dadurch charakterisiert, dass der Spannungswert 50% der Zeit maximal positiv ist und 50% der Zeit maximal negativ ist.

Code für Rechteck-Signal:

```
821.    if(j<0x00FF0000)
822.    {
823.            j=j+0x10000;
824.    }
825.    else
826.    {
827.            j = 0x00000000;
828.    }
829.    if(j < 0x007F0000)
830.    {
831.            left_out = 0x007FFFFF;
832.            right_out = 0x007FFFFF;
833.    }
834.    else
835.    {
836.            left_out = 0x00800000;
837.            right_out = 0x00800000;
838.    }
```

Hier wird wieder eine Zählervariable benutzt. Über der Hälfte des maximalen Zählerwertes ist die Spannung maximal negativ und unter der Hälfte des maximalen Zählerwertes ist die Spannung maximal positiv. Um das Verhältnis von maximal negativ und maximal positiv zu verändern, muss nur der Schwellenwert in Zeile 829 verändert werden.

Das gemessene Rechteck-Signal mit Oszilloskop:

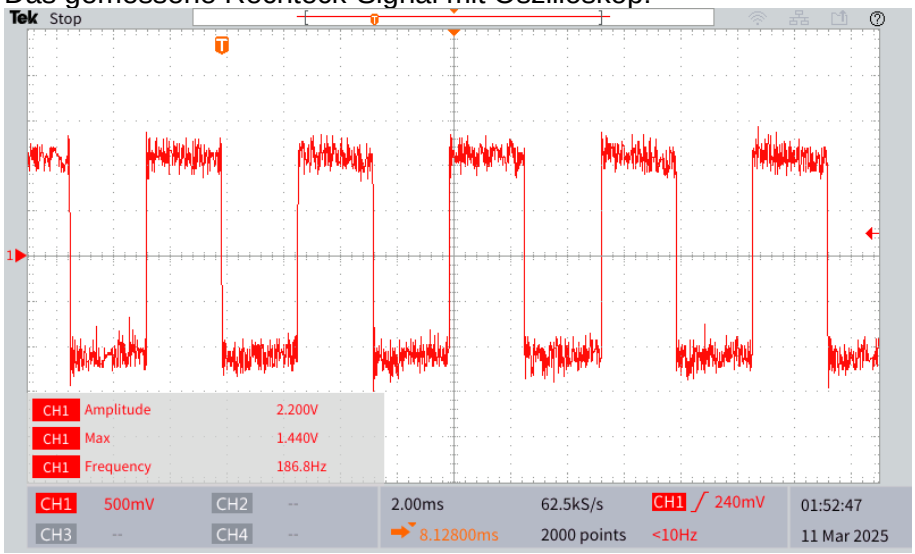



*Abbildung 104: Rechteck-Signal*

Frequenz-Formel: Abtastrate / 255 = 48kHz / 255 = 188,235Hz.

Frequenz gemessen: 186,8Hz.

### 7.7.5 Sinus

Das Sinus-Signal ist ähnlich wie das Dreieck-Signal, nur dass es keine scharfen Kanten gibt und die Steigung bei 0 Volt am stärksten ist und dann gegen das Maxima immer schwächer wird.

Code für Sinus-Signal:

```
841.    if(i<63)
842.    {
843.            i++;
844.    }
845.    else
846.    {
847.            i = 0;
848.    }
849.    left_out = sinValues[i];
850.    right_out = sinValues[i];
```

Hierfür wurde ein Lookup-table `sinValues[]` angelegt worin die Werte von einer Periodendauer eines Sinus gespeichert sind. Diese Werte werden sequenziell und fortlaufend ausgegeben und somit entsteht ein kontinuierliches Sinus-Signal.

Das gemessene Sinus-Signal mit Oszilloskop:

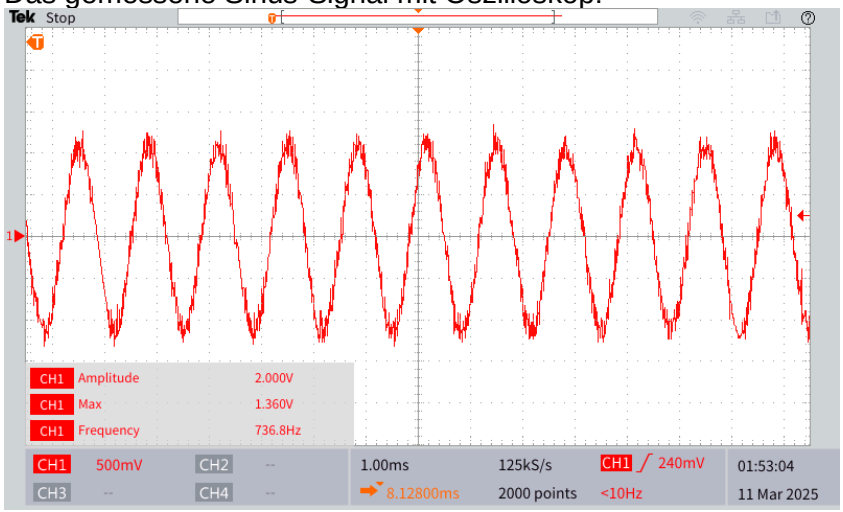



*Abbildung 105: Sinus-Signal*

Frequenz-Formel: Abtastrate / 64 = 48kHz / 255 = 750Hz.

Frequenz gemessen: 736,8Hz.

### 7.7.6 AM

Die Amplitudenmodulation hat im Vergleich zu den anderen bisher gezeigten Signalen keine feste Wellenform. Bei der Amplitudenmodulation gibt es ein hochfrequentes Sinus-Signal, welches der sogenannte Träger ist. Weiters gibt es noch ein Signal, welches Daten enthält, die übertragen werden sollen. Dieses Signal wird dann auf dem Träger-Signal codiert, indem die Amplitude des Träger-Signals dynamisch größer oder kleiner wird, je nach der Amplitude des Daten-Signals. Das modulierte Signal wird dann klassischerweise übertragen. In unserem Fall ist das erzeugte Signal zu niederfrequent, um dieses über Funk übertragen zu können. Dasselbe Prinzip ist aber trotzdem gegeben.

Code für Amplitudenmodulation:

```
853.    if(i<31)
854.    {
855.            i++;
856.    }
857.    else
858.    {
859.            i = 0;
860.    }
861.
862.    dynamicAmplitudeLeftAM = baseAmplitudeAM + left_in;
863.    dynamicAmplitudeRightAM = baseAmplitudeAM + right_in;
864.
865.    left_out = (sinValues32[i] * dynamicAmplitudeLeftAM) / baseAmplitudeAM;
866.    right_out = (sinValues32[i] * dynamicAmplitudeRightAM) / baseAmplitudeAM;
```

Zuerst wurde wieder ein Lookup-table `sinValues32[]` erstellt, welcher das Träger-Signal enthält. Wenn kein Signal anliegt, ist die Amplitude des Träger die Hälfte des maximalen Spannungspegel, welcher in `baseAmplitudeAM` gespeichert ist. Um die veränderte Amplitude des modulierten Signals zu erhalten, wird also in Zeile 862 & 863 das Eingangssignal mit der Basis-Amplitude addiert. Die neue Amplitude wird dann mit dem aktuellen Sinus-Wert multipliziert und durch `baseAmplitudeAM` dividiert, da die `dynamicAmplitudeLeftAM` kein Faktor ist, sondern ein Audio-Wert. Jetzt ist der Träger mit den Daten der eingehenden Audio codiert.

Das gemessene AM-Signal mit Oszilloskop:

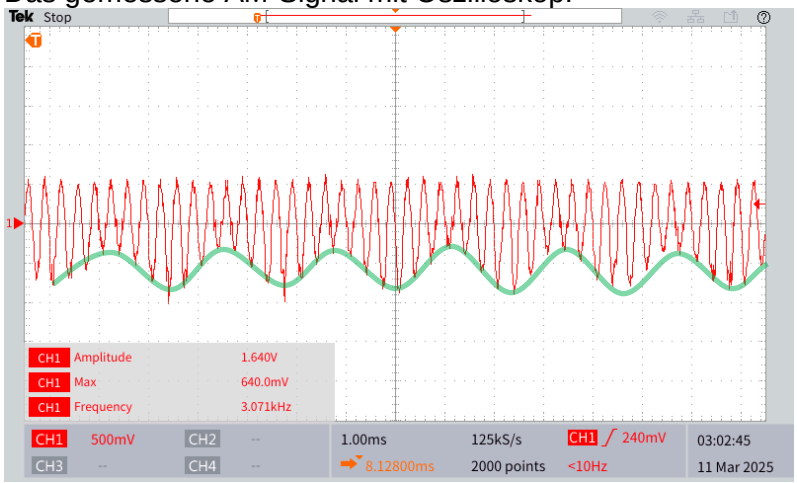



*Abbildung 106: AM-Signal*

Bei dem gezeigten AM-Signal wurde am Eingang ein 440Hz Sinus eingespeist. Dieser kann entlang der grün eingezeichneten Kurve erkannt werden. Das kann auch daran abgelesen werden, dass die Periodendauer des grünen Sinus ungefähr 2.2ms ist, was einer Frequenz von ungefähr 440Hz entspricht.

**Bemerkung:** Bei dem Code gibt es noch einen unentdeckten Fehler, wodurch die positive Seite des modulierten Träger immer dieselbe Amplitude beibehält.

Frequenz: Nicht konstant, hängt von Eingangssignal ab.

## 7.8 Modus Auswahl mit Nextion Display

### 7.8.1 Allgemein

Nach der Programmierung der Signalgeneratoren und Audio-Effekte entstand die Frage, wie diese außerhalb des Programms gesteuert werden können. Aus diesem Grund wurde das Nextion-Display verwendet, welches schon vom schuleigenen MDDS-System zur Verfügung stand. Während der Konfiguration des Nextion-Display und der UART3-Kommunikation kam auch die Idee einen Lautstärken-Regler auf dem Display hinzuzufügen. Damit können jetzt also Signalgeneratoren, Audio-Effekte und die Lautstärke mittels Buttons und Regler vom Benutzer eingestellt werden.

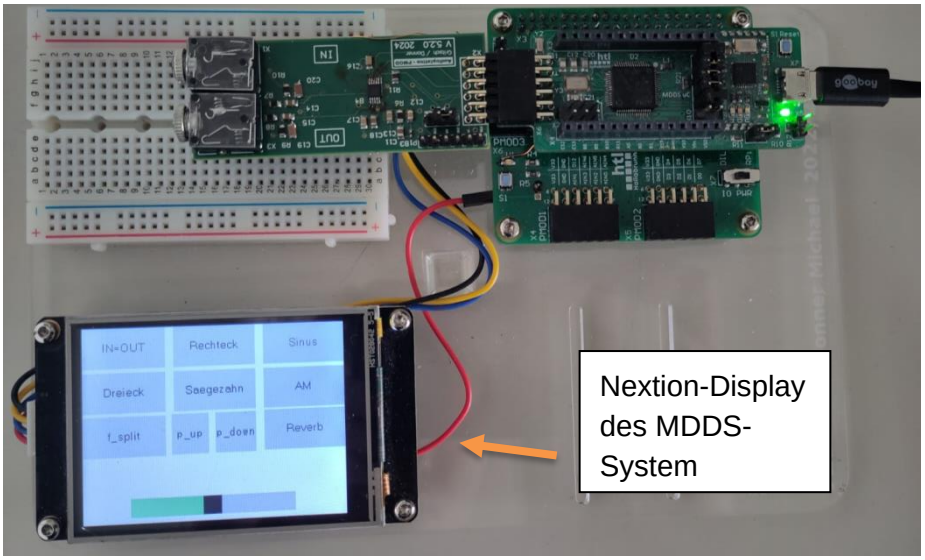



*Abbildung 107: Nextion-Display im MDDS-System*

### 7.8.2 Nextion-Editor

Die Konfiguration für das Nextion-Display wurde in dem Programm „Nextion Editor“ getätigt.

#### *7.8.2.1 Projekt Erstellung*

Nachdem das Programm geöffnet wurde, muss zuerst ein neues File angelegt werden. Unter „*File*“ → „*New*“ kann ein neues File angelegt werden, welches an einem beliebigen Speicherplatz gespeichert wird.

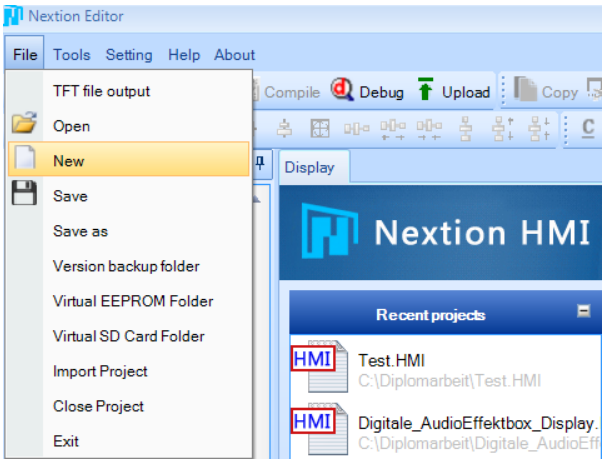



*Abbildung 108: Nextion-Editor New File*

Als Nächstes muss ausgewählt werden, welche Art von Display verwendet wird. Das kann auf der Unterseite des Nextion-Displays überprüft werden, wie in dem rechten Bild zu sehen ist. Unser Display kann unter „*Enhanced*“ → „*NX3224K028_011*“ gefunden werden. Darauf klicken und „*OK*“ drücken.

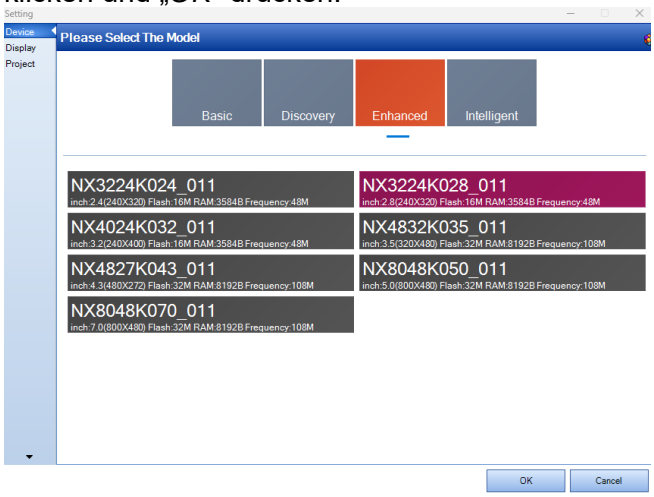



*Abbildung 110: Nextion-Display Auswahl*

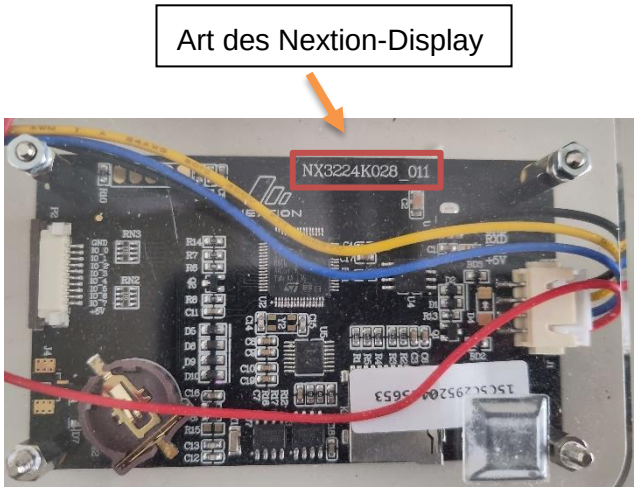



*Abbildung 109: Nextion-Display Art*

Danach muss noch konfiguriert werden, dass das Nextion-Display Horizontal verwendet werden soll.

Unter „*Character Encoding*“ sollte noch „*ASCII*“ eingestellt werden. Eine Alternative dazu wäre „*iso-8859-1*“ wobei bei dieser Codierung auch ländereigene Zeichen wie z.B.: „*ä,ö,ü,ß*“ aus dem Deutschen verwendbar sind. Für unsere Anwendung wurde sich dazu entschieden, nur lateinische Schriftzeichen am Display zu nutzen. Danach wieder auf „*OK*“ drücken.

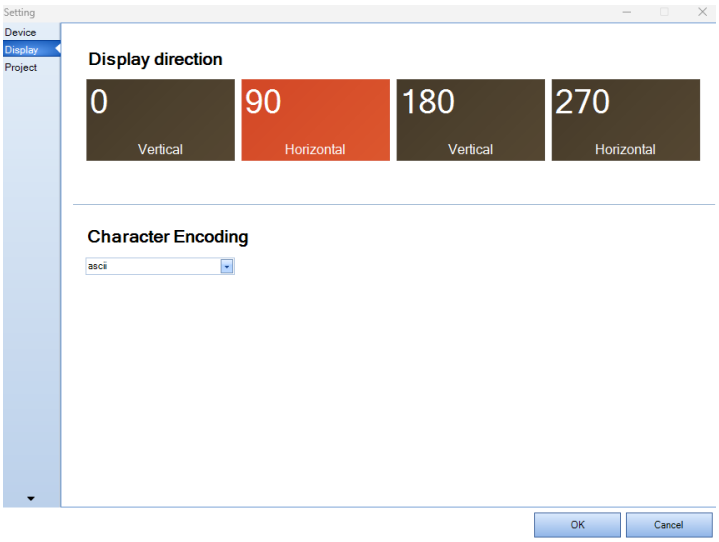



*Abbildung 111: Display Ausrichtung & Character Encoding*

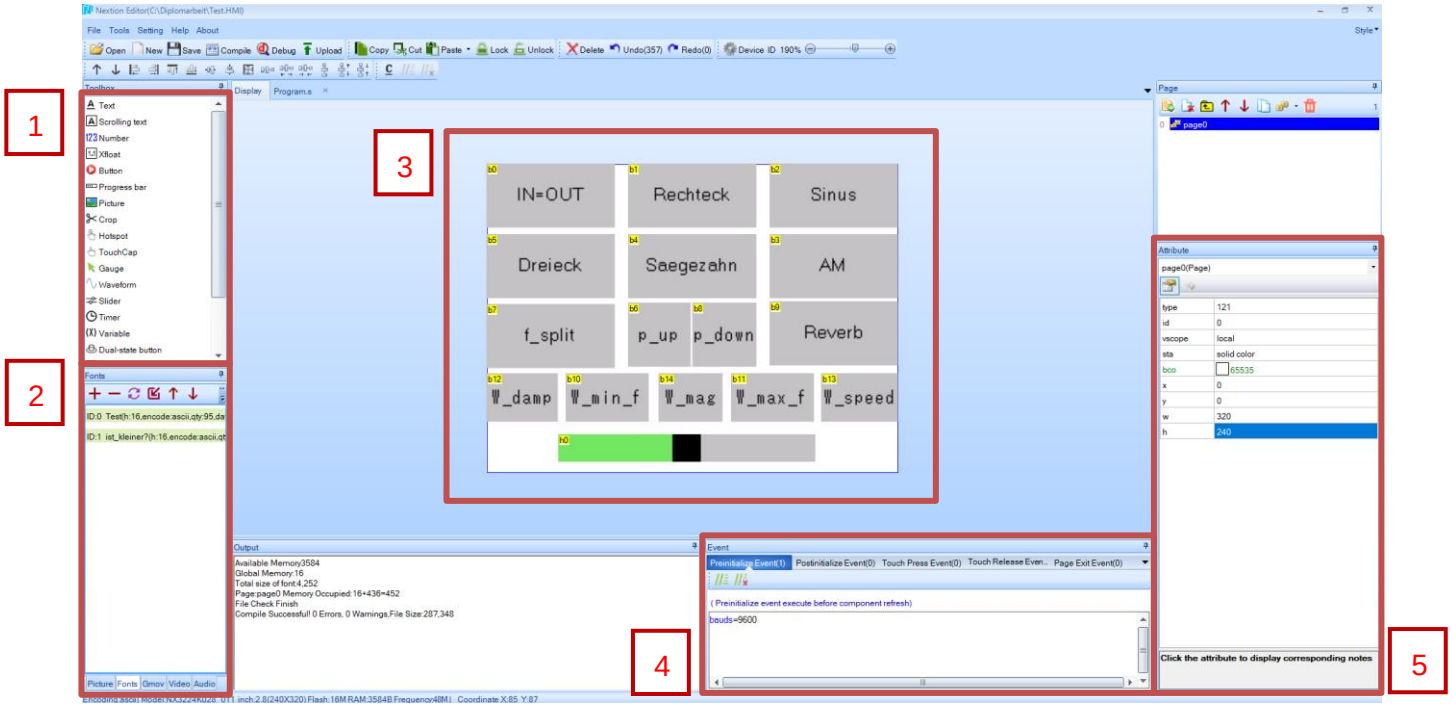



*Abbildung 112: Nextion-Editor Haupt-Editor-Bereich*

Die obige Abbildung zeigt den Haupt-Editor-Bereich vom Nextion-Editor wo alle Komponenten, die auf dem Display zu sehen sind, konfiguriert werden können.

1) Hier sind Komponenten vorhanden die am Bildschirm angezeigt werden können. Für den Modus haben wir nur Buttons verwendet und der Regler für die Lautstärke ist ein Slider. Diese Komponenten können einfach angedrückt werden und dann erscheinen sie am Display.
2) Fonts: Hier sind Fonts zu sehen die erstellt wurden und welche für die Textanzeige wie zum Beispiel der Text der Buttons verwendet werden können.
   Unter „*Tools*“ → „*Font Generator*“ kann ein neuer Schriftzug erstellt werden.

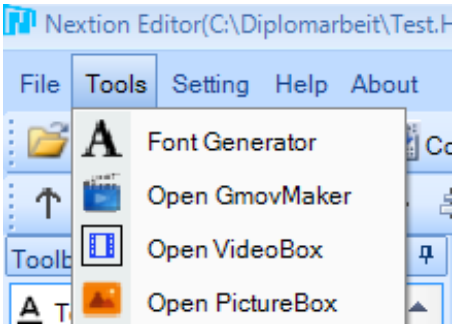



*Abbildung 114: Nextion-Editor Font Generator*

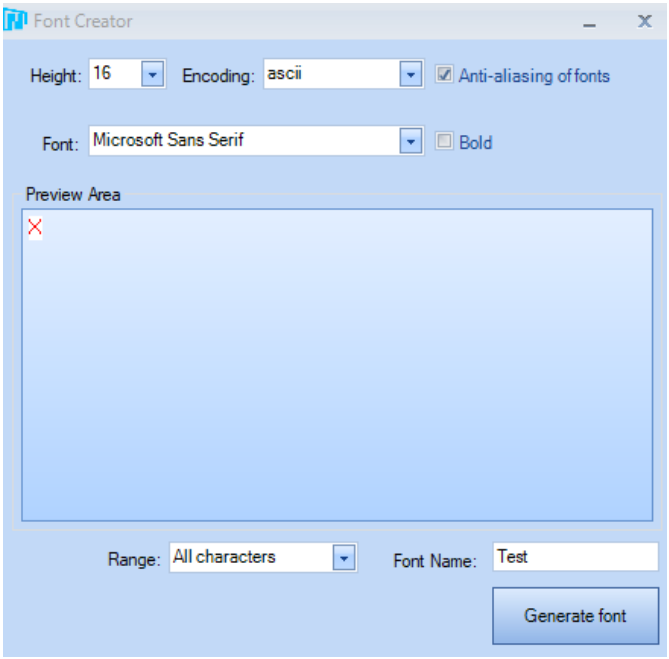



*Abbildung 113: Nextion-Editor Font Creator*

Unter „*Font Creator*“ kann jetzt unser Schriftzug definiert werden. Dazu wird die Höhe, die Codierung, und die Font ausgewählt. Danach muss dem Schriftzug noch ein Name gegeben werden und dann kann er mit „*Generate font*“ erstellt werden.

3) Hier ist das Vorschaubild des Bildschirm zu sehen. Die Komponenten, die unter „*Tools*“ ausgewählt worden sind, können hier ausgewählt, herumgeschoben und skaliert werden. In dem Bild ist zu sehen, dass der Button „*IN=OUT*“ ausgewählt ist.
4) Hier sind die Events zu sehen die mit dem Interagieren des ausgewählten Komponenten in Verbindung stehen. Jeder Button löst in unserer Konfiguration ein Event aus, wenn er gedrückt wird. Wie in dem Fall von dem Button „*IN=OUT*“ zu sehen ist, sendet dieser, wenn er gedrückt ist, seine „*Component-ID*“. Das passiert bei jedem Button / Regler, wenn dieser gedrückt ist und mit dieser ID weiß unser Mikrocontroller, welche Komponente gedrückt worden ist.
5) Hier sind die Attribute unserer ausgewählten Komponente zu sehen. Die wichtigste Eigenschaft hier ist unter „*txt*“, wo der angezeigte Text angegeben werden kann.

Wenn alle Konfigurationen abgeschlossen sind, kann unter „File“ → „*TFT file output*“ eine Datei erstellt werden, die die Informationen des erstellten Nextion-Display enthält.
Diese Datei muss auf einer SD-Karte gespeichert werden, welche danach die Daten auf das Nextion-Display flasht.
Um das File auf dem verwendeten Computer auf der SD-Karte speichern zu können wurde ein SD-Karten-Adapter verwendet.

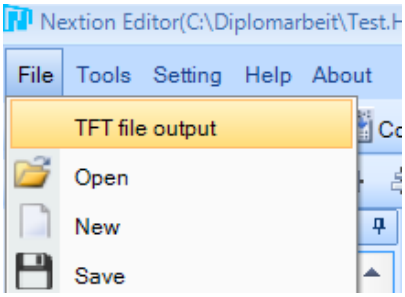



*Abbildung 115: Nextion-Editor File Output*

Wenn das File auf der SD-Karte gespeichert ist, wird diese In den SD-Karten-Slot auf der Seite des Nextion-Display mit der Unterseite nach oben gesteckt.
Wenn die Stromversorgung gegeben ist, flasht die SD-Karte das Programm auf das Nextion-Display. Danach wird die SD-Karte entfernt und bei dem nächsten Reset ist die neue Konfiguration vorhanden.




*Abbildung 116: SD-Karte & Adapter*

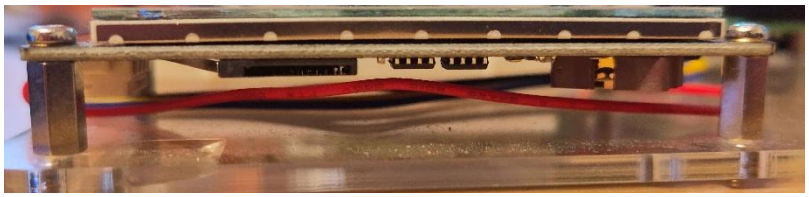


*Abbildung 117: SD-Karten-Slot*

### *7.8.2.2 Debug*

Ein weiterer wichtiger Aspekt der Konfiguration des Nextion-Display ist der Debugger. Hier kann gesehen werden, welche Zeichenkette die Komponente bei dem angegebenen Event schickt.

„*Debug*“ befindet sich auf der Tool-Leiste.

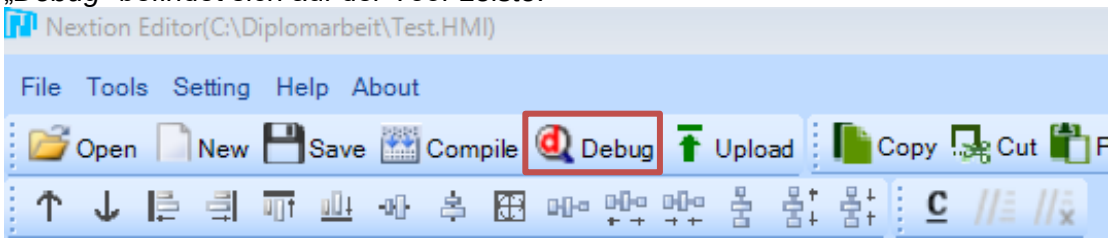



*Abbildung 118: Debug auf Tool-Leiste*

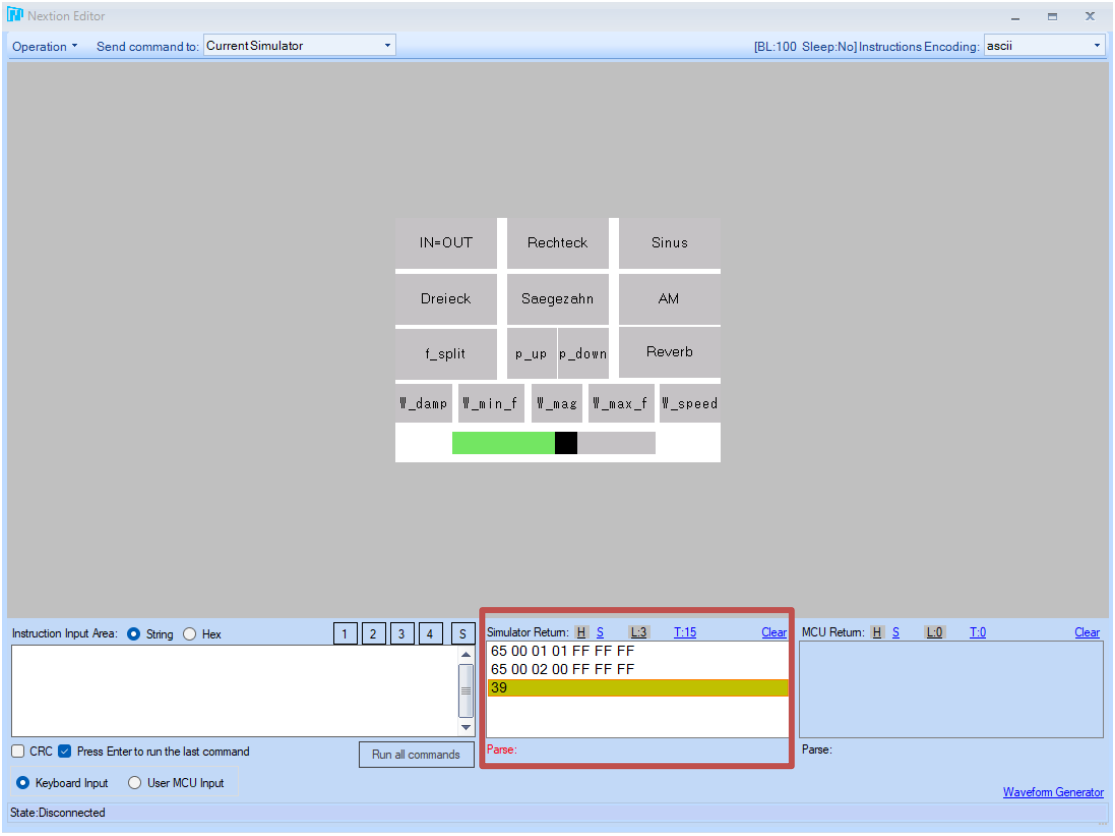



*Abbildung 119: Nextion-Display Simulator*

In der Mitte des Fensters ist der simulierte Bildschirm des Nextion-Display. Wenn auf einen Button gedrückt wird, bzw. der Regler geschoben wird, werden bei „*Simulator Return*“ die Daten angezeigt, die über UART3 an den Mikrocontroller geschickt werden würden. Im Falle des Regler‘s schickt dieser noch zusätzlich eine Zahl, die von 0 bis 0x64 bzw.100 geht.

## 7.8.3 UART3-Kommunikation

### *7.8.3.1 Allgemein*

Wie schon im Debug-Modus des Nextion-Display zu sehen ist, sendet das Nextion-Display Daten über die UART3-Schnittstelle, wenn ein Button oder der Regler gedrückt bzw. verschoben wird.

### *7.8.3.2 Port*

Auf dem Base-Board befinden sich die Ports PB11 und PB10 welche jeweils als UART3 Receive-Leitung oder UART3 Transmit-Leitung konfiguriert werden können. In unserem Fall empfangen wir nur Daten vom Nextion-Display, also wird auch nur der UART3_RX-Port verwendet.

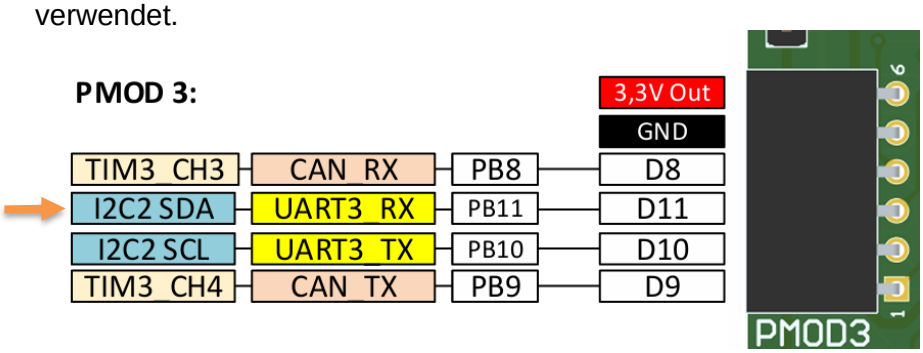



*Abbildung 120: UART3 Port-Besetzung*

### *7.8.3.3 Protokoll*

Die USART3-Kommunikation kann sowohl synchron (also mit zusätzlicher Clock) als auch asynchron verlaufen. In unserem Fall funktioniert die UART3-Kommunikation asynchron, da die Clock nicht benötigt wird.
Die Datenleitung ist prinzipiell über einen Pull-Up Widerstand auf einem Versorgungspegel von 3,3V. Wenn eine „0“ anliegen soll wird die Leitung auf GND gezogen.
Um die Kommunikation zu beginnen, gibt es ein Start-Bit (Low) und um sie zu beenden ein Stopp-Bit (High). Außerdem wird nach jedem übertragenen Byte auch ein Stopp und ein anschließendes Start-Bit übertragen.
Zwischen jedem Start- und Stopp- Bit wird 1 Byte an Daten übertragen, welche alle lsb-first übertragen werden. Das bedeutet, dass die niederwertigeren Bits zuerst und die höherwertigeren Bits zuletzt übertragen werden.

Folgende Abbildung zeigt das Übertragungs-Protokoll graphisch für 1 Byte.

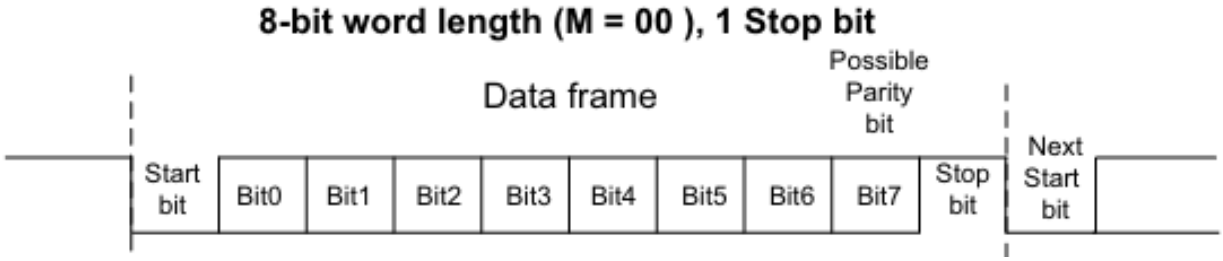



*Abbildung 121: UART3 Protokoll [STC25]*

### *7.8.3.4 Protokoll Diagramme*

In den folgenden Diagrammen wird die tatsächliche Übertragung der USART3-Kommunikation gezeigt.

Übertragung Komponente-ID von Button IN=OUT




*Abbildung 123: Button IN_OUT gedrückt*

Zusätzlicher Wert bei Lautstärken-Regler Übertragung

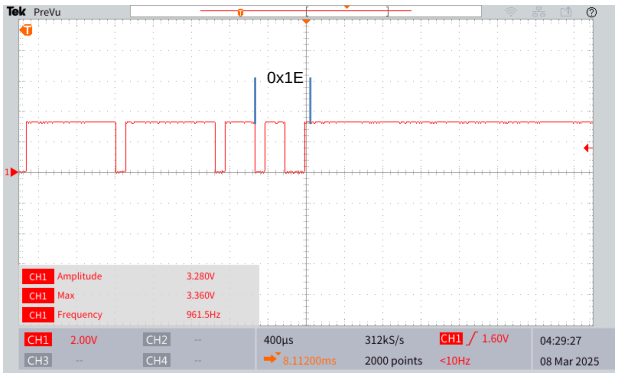



*Abbildung 122: Lautstärkeregler verschoben*

In diesem Diagramm wird das erste übertragene Byte bei dem Drücken von dem Button „IN=OUT“ noch genauer analysiert. Dieses Byte sollte 0x65 betragen.

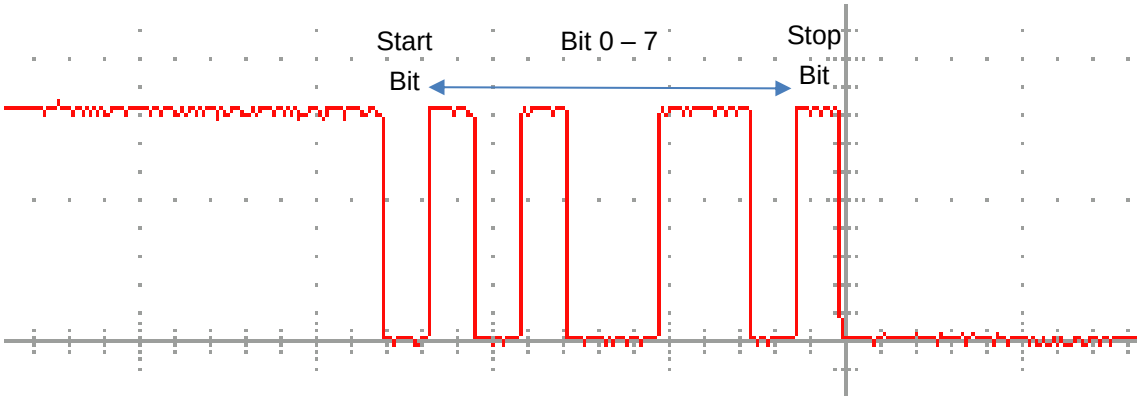



*Abbildung 124: 1 Byte UART-Übertragung*

Die Hexadezimale Zahl 0x65 ist in binär umgewandelt 0b01100101. Da die Übertragung lsb-first ist, muss bei der Überprüfung eine Bit-Reihenfolge von hinten mit der anderen von vorne weg verglichen werden. Dadurch kann gesehen werden, dass die Simulation mit der realen Übertragung übereinstimmt.

# 7.9 Audio-Effekte

## 7.9.1 Allgemein

In diesem Kapitel geht es um die Audio-Effekte. Diese bearbeiten die eingehende Audio so, dass ein gewisser Effekt der Audio am Ausgang zu hören ist. Um zu erfahren, wie diese Audio-Effekte funktionieren wurden die Quellen [YOU25a], [YOU25b] und [YOU25c] verwendet.

## 7.9.2 Frequency Split

### *7.9.2.1 Allgemein*

Bei diesem Audio-Effekt geht es darum, dass hohe Frequenzen und tiefe Frequenzen jeweils auf einem Ohr ausgegeben werden. Des Weiteren soll die Möglichkeit bestehen den Ausgang der hohen bzw. tiefen Frequenzen zu tauschen. Die Abschnittfrequenz, bei welcher das Signal geteilt wird, wurde auf 600Hz bestimmt. Das heißt Frequenzen unter 600Hz sollen auf einem Ohr und Frequenzen über 600Hz auf dem anderen Ohr ausgegeben werden.

### *7.9.2.2 IIR-Filter*

Ein IIR-Filter erlaubt es, durch eine gewisse Berechnungsweise, einen digitalen Filter wie Tiefpass, Hochpass oder Bandpass zu realisieren. Dabei wird der aktuelle Eingangswert dem Filter übergeben und dieser berechnet dann mit verschiedenen Koeffizienten und vergangenen Werten den Ausgangswert.

Die gewählte Filterstruktur heißt Kaskaden-Struktur und sieht so aus:

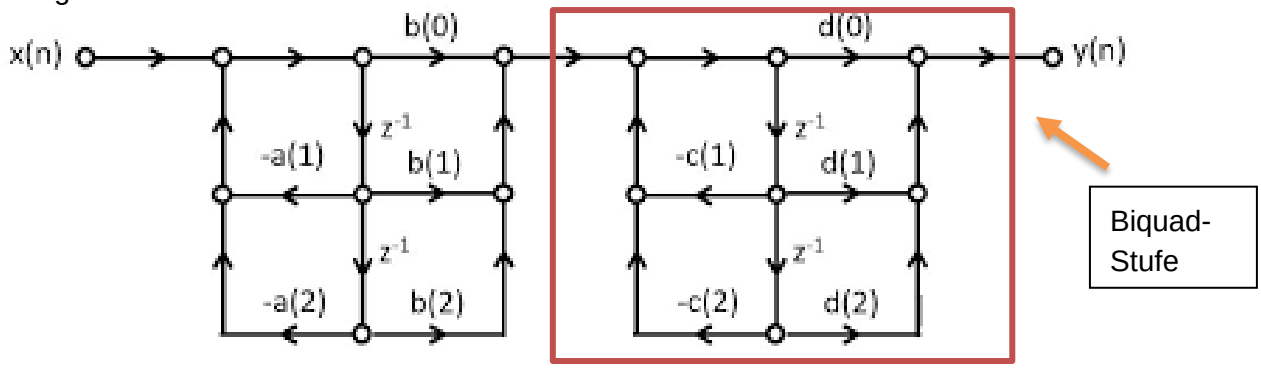



*Abbildung 125: IIR-Filter Kaskaden-Struktur [SCH25]*

**a, b, c, d** sind Koeffizienten, mit denen Der Wert davor multipliziert wird.

**$z^{-1}$** ist eine Zeitverzögerung. Der Wert davor wird für den nächsten Durchgang abgespeichert und nach dem **$z^{-1}$** genutzt.

**o:** Hier werden Werte, die mit einem Pfeil auf o zeigen zusammengezählt bzw. der aktuelle Wert teilt sich auf mehrere Pfade auf.

Ein „Block“ der Kaskadenstruktur wird „Biquad-Stufe“ genannt.

### *7.9.2.3 Matlab Filter Designer*

Um den Filter zu programmieren, fehlen uns noch die Koeffizienten. Diese können von Matlab Filter Designer berechnet werden. Für unsere Anwendung wird jeweils ein Tiefpass und ein Hochpass mit der Grenzfrequenz von 600Hz benötigt.

Dazu muss der Filter-Designer installiert sein und im Workspace das Keyword „filterDesigner“ eingegeben werden. Dann erscheint diese graphische Oberfläche:

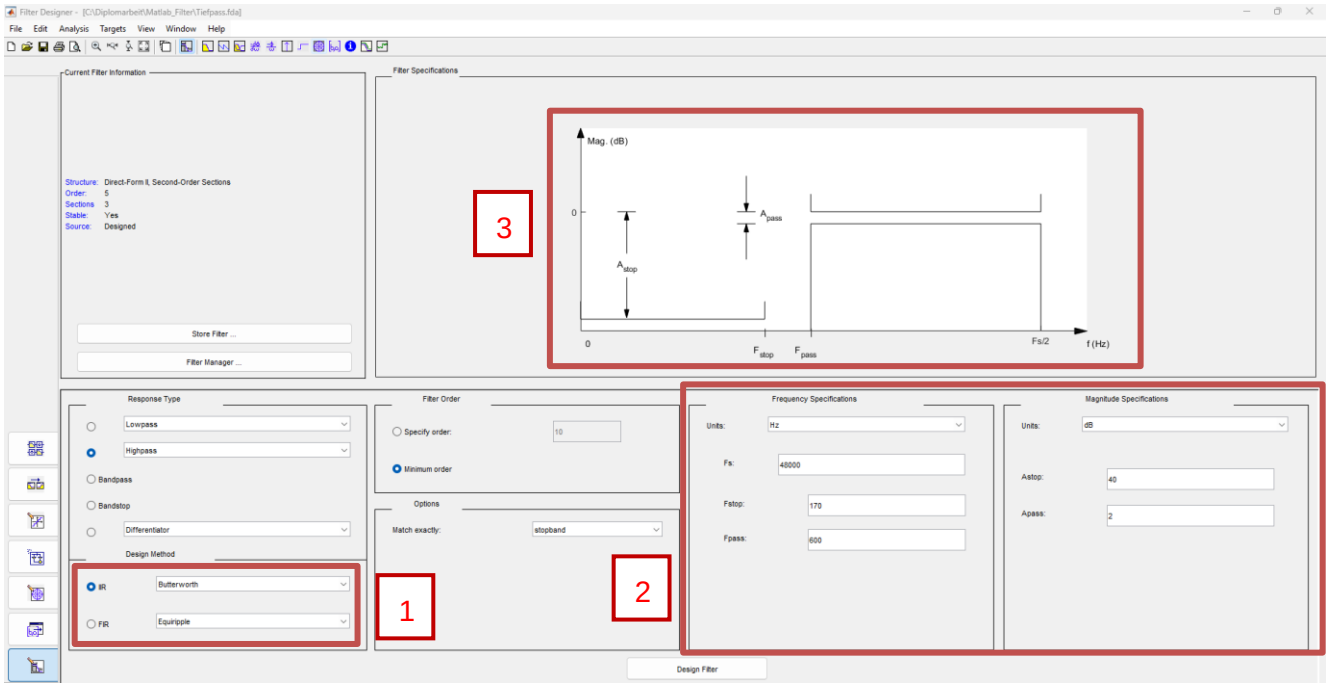



*Abbildung 126: Matlab Filter Designer*

Beispiel für den Hochpass:

1) Hier wird eingestellt, dass ein IIR-Filter verwendet wird und dass dieser die „*Butterworth*“ Filterform haben soll, welche etwa die Standard-Filterform ist.
2) Hier wird die verwendete Abtastrate von 48kHz eingestellt und unter welcher Frequenz der Filter zum Filtern beginnen soll (Grenzfrequenz = 600Hz). Die Stop-Frequenz wird benötigt, um bei dieser Frequenz eine Dämpfung in dB anzugeben. Diese Dämpfung ist dann ab der Stop-Frequenz erreicht.
3) Hier kann der Amplitudengang des Filters mit den angegebenen Frequenzen und Dämpfungen angesehen werden.

Die Stop-Frequenz kann nicht viel näher an die Grenzfrequenz geschoben werden, da sonst die Filter-Ordnung zu hoch werden würde und dadurch der Rechenaufwand zu groß werden würde.

Wenn alles eingestellt ist, kann „*Design Filter*“ gedrückt werden.

Unter „*Targets*“ → „*Generate C Header* …“ können die Filter-Koeffizienten als C-Variablen exportiert werden. Dazu wurde noch bei „*Export as*:“ „*Single-precision float“* ausgewählt.

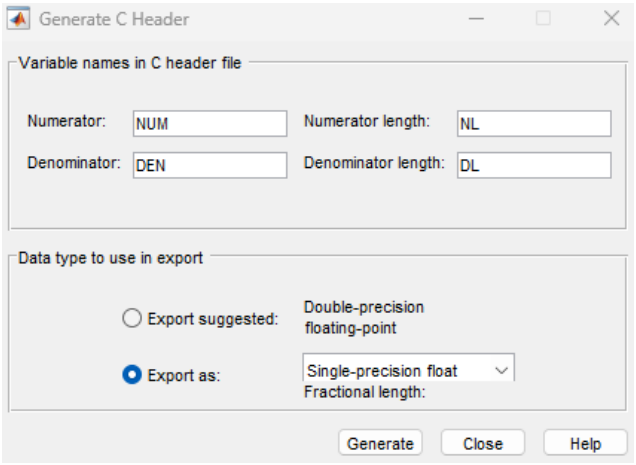



*Abbildung 127: C-Header exportieren*

### *7.9.2.4 Code*

Für die Berechnung des IIR-Filter wurde eine Funktion erstellt:

```
983. int iir_filter_process(int x) {
984.     float y = (float)x;
985.
986.     for (int i = 0; i < MWSPT_NSEC; i++) {
987.         float w0 = y - pDEN[i][1] * pState[i][0] - pDEN[i][2] * pState[i][1];
988.         y = pNUM[i][0] * w0 + pNUM[i][1] * pState[i][0] + pNUM[i][2] * pState[i][1];
989.
990.         // Shift delay elements
991.         pState[i][1] = pState[i][0];
992.         pState[i][0] = w0;
993.     }
994.
995.     return (int) y;
996. }
```

5 Biquad-Stufen

Die Funktion übergibt den Eingangswert x und retourniert den gefilterten Wert y.
Die Berechnung folgt dem Blockschaltbild von einer Biquad-Stufe der Kaskaden-Struktur.
Die Variable `MWSPT_NSEC` beträgt für Hochpass- und Tiefpass-Filter 5 und sagt aus, wie oft eine Biquad-Stufe für diesen Filter notwendig ist und wiederholt die Berechnung einer Biquad-Stufe anhand dieser Zahl.

Da diese Funktion von dem Hochpass und dem Tiefpass genutzt wird, verwendet es für seine Koeffizienten und Speicher-Elemente Pointer, welche vor dem Aufrufen der Funktion gesetzt werden.

State → Hier werden die Werte gespeichert, die im Blockschaltbild nach $\mathbf{z^{-1}}$ sind.
DEN → Hier sind die Koeffizienten gespeichert, die im Blockschaltbild a oder c sind.
NUM → Hier sind die Koeffizienten gespeichert, die im Blockschaltbild b oder d sind.

In Zeile 987 & 988 wird die Biquad-Stufe berechnet und in Zeile 991 & 992 werden die Werte, die für den nächsten Durchlauf gebraucht werden, gespeichert.

Code-Ausschnitt von `calculate_Output();` :

```
878.     left_out = (left_in>>1)+(right_in>>1); // mono the audio
879.     right_out = left_out;
880.
881.     if(isLeftHigh == 't')
882.     {
883.               pNUM = NUML; // Update Pointers for Lowpass-Filter
884.               pDEN = DENL;
885.               pState = wl;
886.               right_out = iir_filter_process(right_out);
887.
888.               pNUM = NUMH; // Update Pointers for Highpass-Filter
889.               pDEN = DENH;
890.               pState = wh;
891.               left_out = iir_filter_process(left_out);
892.     }
893.     else
894.     {
895.               pNUM = NUML; // Update Pointers for Lowpass-Filter
896.               pDEN = DENL;
897.               pState = wl;
898.               left_out = iir_filter_process(left_out);
899.
900.               pNUM = NUMH; // Update Pointers for Highpass-Filter
901.               pDEN = DENH;
902.               pState = wh;
903.               right_out = iir_filter_process(right_out);
904.     }
```




1) Zuerst wird die Audio von Stereo zu Mono berechnet, damit nicht beide `left_out` und `right_out` einzeln berechnet werden müssen.

2) Danach wird unterschieden auf welchem Ohr die hohen bzw. tiefen Freqeuenzen ausgegeben werden. Je nachdem bei welchem Ohr jetzt hohe oder tiefe Frequenzen zu hören sein sollen, werden die Pointer auf die Koeffizienten des gewollten Filter gesetzt und dann kann der Filter auch schon berechnet werden. Das gleiche passiert für das andere Ohr, wobei dann die Koeffizienten des jeweils anderen Filter für die Berechnung verwendet werden.

## 7.9.3 Pitch Shift (Mickey Mouse Voice)

*7.9.3.1 Allgemein*

Bei dem Pitch-Shift Audio-Effekt werden alle Frequenzen des Audio-Eingangs entweder erhöht oder vertieft. Ein klassisches Beispiel dafür ist der Stimmverzerrer bei dem Pitch-Shift eingesetzt wird, wobei die Stimme dann entweder höher oder tiefer ist als die Original-Stimme. Das Ziel ist es den Pitch im Programm nach oben oder unten dynamisch steuern zu können.

### *7.9.3.2 Grundprinzip*

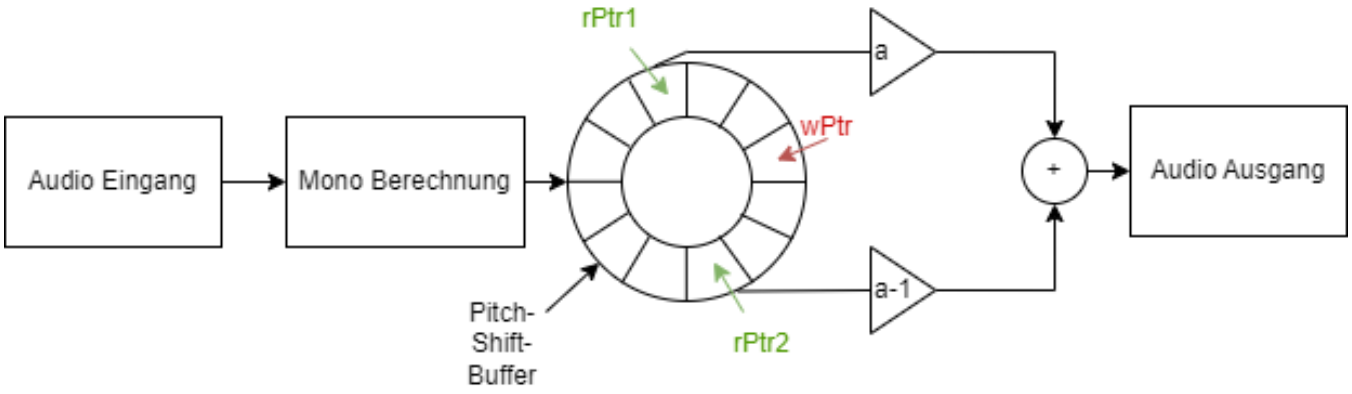


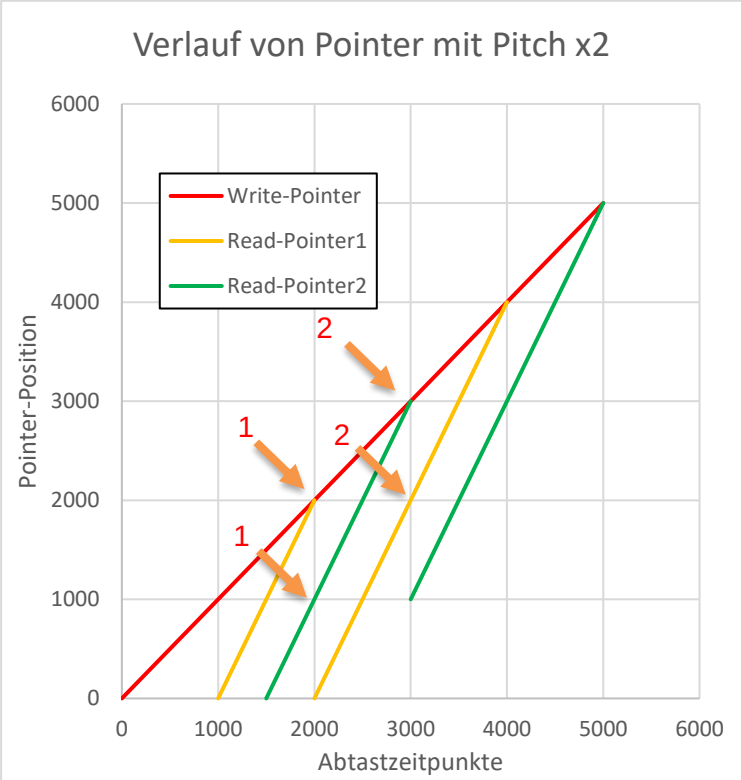

*Abbildung 128: Pitch-Shift Grundprinzip*

Zuerst wir die Stereo-Audio wieder zur Mono-Audio berechnet, damit das Programm nicht doppelt für beide Ohren laufen muss.

Danach gibt es einen Ring-Buffer, welcher 1000 Samples groß ist. Der Pointer wPtr ist der Write-Pointer. Dieser kopiert kontinuierlich das neue Sample an den nächsten Platz im Ring-Buffer.

Neben dem Write-Pointer gibt es noch 2 Pointer: rPtr1 und rPtr2 welche Read-Pointer sind und um 180° zueinander verschoben sind bzw. 500 Samples entfernt sind. Diese Read-Pointer rotieren mit der Geschwindigkeit `Shift`, was eine float Variable ist. Geschwindigkeit heißt, um wie viele Samples sie pro neuem Audio-Sample weiter im Buffer zeigen. Diese Variable kann von 0,5 – 2 betragen. 0,5 würde eine Oktav niedriger bedeuten und 2 eine Oktav höher.

Es braucht 2 Read-Pointer, da jedes Mal ein Knacksen zu hören wäre, wenn der Read-Pointer den Write-Pointer überholt bzw. überholt wird. Bevor das passiert, gibt es eine „Fade-Phase“ zum anderen Read-Pointer, wobei der aktuelle Read-Pointer immer weniger stark gewichtet wird und der andere immer mehr, bis der andere Read-Pointer den Audio-Ausgang übernommen hat. „a“ ist der Faktor von 0 – 1 und gibt an zu welchem Teil der Read-Pointer1 den Ausgang beeinflusst und a – 1 ist dann der fehlende Teil auf 1, der angibt zu welchem Teil der Read-Pointer2 den Ausgang beeinflusst.




In diesem Diagramm wird der Verlauf der 3 Pointer dargestellt, wenn der Pitch-Shift 2 beträgt. Auf der X-Achse sind die vergangen Abtastzeitpunkte aufgetragen und auf der Y-Achse ist die Pointer-Position im Buffer eingetragen. Hier wird angenommen, dass der Buffer-Platz unendlich ist. Es gilt jedoch dasselbe Prinzip. Die Wert-Auslesung startet im Read-Pointer1. Wenn der Read-Pointer1 den Write-Pointer überholen würde wechselt die Wert-Ausgabe auf den Read-Pointer2 (1). Wenn wiederum dieser den Write-Pointer überholen würde, wechselt die Wert-Ausgabe auf den Read-Pointer1 zurück (2). Dieses Schema geht kontinuierlich weiter.

*7.9.3.3 Code*

Funktion zur Berechnung des Pitch-Shift:

```
926. void apply_pitch_shift() {
927.     int32_t sum = (left_in>>1)+(right_in>>1); // mono the audio
930.
931.     Buf_Pitch_Shift[WriteP] = sum;
932.
933.     int32_t ReadP_Int = roundf(ReadP);
934.     int32_t ReadP_Int2 = 0;
935.     if(ReadP_Int >= BUFFER_SIZE_PITCH_SHIFT/2)
936.     {
937.               ReadP_Int2 = ReadP_Int - (BUFFER_SIZE_PITCH_SHIFT/2);
938.     }else
939.     {
940.               ReadP_Int2 = ReadP_Int + (BUFFER_SIZE_PITCH_SHIFT/2);
941.     }
942.
943.     float Read_val = (float) Buf_Pitch_Shift[ReadP_Int];
944.     float Read_val2 = (float) Buf_Pitch_Shift[ReadP_Int2];
```

In Zeile 927 wird die Mono-Audio berechnet.

In Zeile 931 wird das neue Audio-Sample auf den Speicher-Platz des Write-Pointer geschrieben.

Von Zeile 933 – 944 werden die Read-Pointer aktualisiert und anschließend werden beide Werte aus dem Buffer gelesen.

Rest der Funktion:

```
948.     if(Overlap >= (WriteP-ReadP_Int) && (WriteP-ReadP_Int) >= 0 && Shift != 1.0f)
949.     {
950.               int rel = WriteP-ReadP_Int;
951.               Crossfade = ((float)rel)/(float)Overlap;
952.     }else if((WriteP-ReadP_Int) == 0)
953.     {
954.               Crossfade = 0.0f;
955.     }
956.
957.     if(Overlap >= (WriteP-ReadP_Int2) && (WriteP-ReadP_Int2) >= 0 && Shift != 1.0f)
958.     {
959.               int rel = WriteP-ReadP_Int2;
960.               Crossfade = 1.0f - ((float)rel)/(float)Overlap;
961.     }else if((WriteP-ReadP_Int2) == 0)
962.     {
963.               Crossfade = 1.0f;
964.     }
965.
966.     sum = (Read_val*Crossfade + Read_val2*(1.0f-Crossfade));
967.
968.     ReadP += Shift;
969.     WriteP++;
970.     if(WriteP >= BUFFER_SIZE_PITCH_SHIFT)
971.     {
972.               WriteP = 0;
973.     }
974.     if(roundf(ReadP) >= BUFFER_SIZE_PITCH_SHIFT)
975.     {
976.               ReadP = 0.0f;
977.     }
978.     //Apply Pitch-Shift to output
979.     left_out = sum;
980.     right_out = sum;
981. }
```

`Crossfade` entspricht der Variable a von dem Blockschaltbild von vorher.
`Overlap` entspricht einer Zahl von 100. Wenn ein Read-Pointer in der Nähe von 100 Samples oder weniger des Write-Pointer ist, beginnt die Fade-Phase zu dem anderen Read-Pointer.

Zeile 948 – 964: Hier wird ermittelt, ob einer der Read-Pointer in die Nähe des Write-Pointer kommt. Wenn ja beginnt die Fade-Phase zum anderen Read-Pointer.

Zeile 966: Hier werden die beiden Werte der Read-Pointer mit ihrer Gewichtung zusammen addiert und im Ausgangs-Sample gespeichert.

Zeile 968 – 977: Write- und Read-Pointer werden aktualisiert.

Zeile 979 & 980: Jetzt müssen nur noch die Ausgangs-Buffer auf den ermittelten Pitch-Shift-Wert gesetzt werden.

In `calculate_Output();` wird dabei nur die Funktion `apply_pitch_shift();` aufgerufen.

## 7.9.4 Reverb

### *7.9.4.1 Allgemein*

Bei dem Reverb Audio-Effekt wird die eingehende Audio mit einem Echo beaufschlagt. Die Intensität des Echo-Effekts soll auch gesteuert werden können.

### *7.9.4.2 Grundprinzip*

Blockschaltbild des Reverb-Filter:

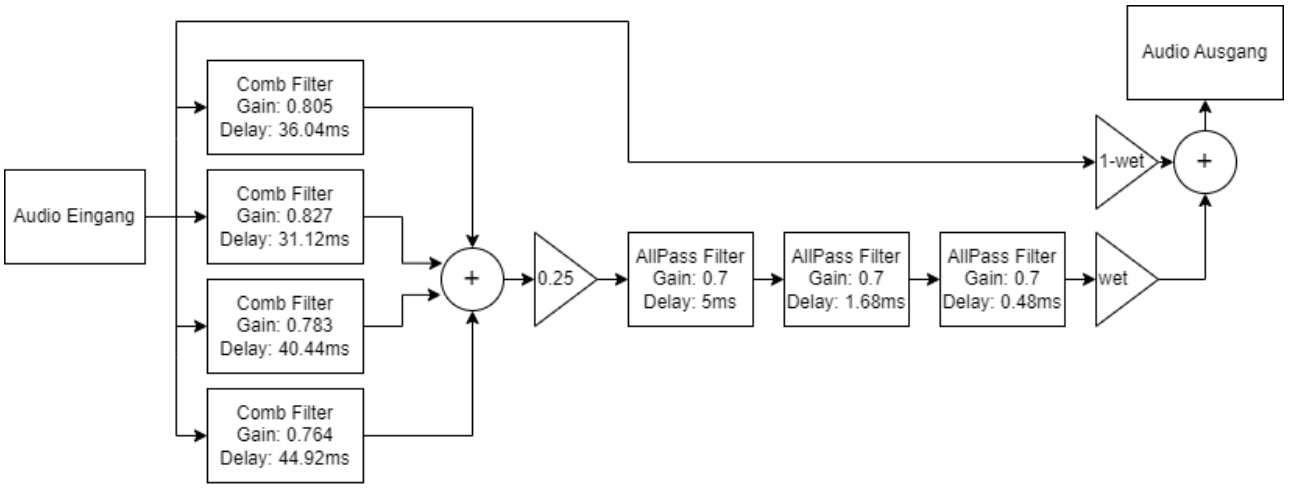



*Abbildung 129: Reverb Blockschaltbild*

Die Idee ist, dass neben dem Audio-Signal noch verzögerte, vergangene Audio gehört werden kann, die als Echo wahrgenommen wird. Dazu gibt es die Variable „*wet*“ die wieder 0 – 1 sein kann und angibt, wie groß der Echo-Effekt ist. 1 ist am stärksten und 0 ohne Echo-Effekt.

Bei dem Echo-Effekt selbst ist die Überlegung, dass die Schallwelle von 4 verschiedenen Wänden reflektiert wird und somit zeitverzögert am Beobachter ankommt. Diese Verzögerung wird mit den „*Comb Filter*“ realisiert. Danach werden die Signale addiert und durch 4 geteilt, da 4 Signale addiert worden sind und das Signal nicht verstärkt werden soll.

Danach wird das Signal noch durch 3 kurze Verzögerungs-Filter mit einer Verstärkung von 0,7 geschickt, bevor das Echo-Signal mit dem aktuellen Signal gewichtet addiert wird und auf den Audio-Ausgang gespeichert wird.

Alle Filter müssen eine Verstärkung von < 1 haben, da sie sonst instabil werden könnten.

Struktur des Comb- und AllPass Filter:

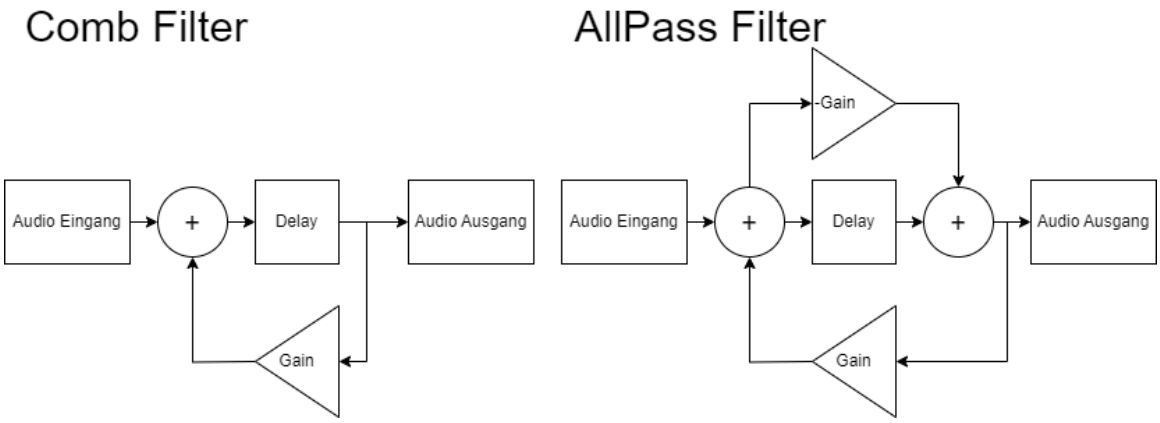



*Abbildung 130: Comb-Filter, AllPass-Filter*

Es gelten wieder dieselben Regeln zur Berechnung der Filter wie bei dem IIR-Kaskaden-Filter. Für beide Filter wurde eine Funktion erstellt, die nach dem Blockschaltbild-Schema arbeitet. Dabei gibt es immer einen Eingang und Ausgang des Filter. Da mehrere Filter von der Struktur Comb- und AllPass-Filter existieren, werden die Parameter wieder mit Pointer übergeben.

Bei beiden Filtern gibt es einen Buffer, welcher so viel Platz hat, dass die Multiplikation der Buffergröße mit der Abtastrate die im Blockschaltbild gezeigte Verzögerung ergibt.
Z.B.: Comb-Filter mit der größten Verzögerung: Buffergröße = 2156 * 1/48000s = 44.916ms

Es wird also der aktuelle Wert abgespeichert und der Wert vor z.B.: 44.916ms für die weitere Berechnung verwendet.

### *7.9.4.3 Code*

Code für die Comb_Filter-Funktion:

```
998. float Comb_Filter(float inputSample,float cf_buf[], int *cf_p, float *cf_g, int *cf_lim)
999. {
1000.    float readback = cf_buf[*cf_p];          // Read Most Delayed Value of cf_buf
1001.    float newBufSample = readback * (*cf_g) + inputSample; // Calculate Value for cf_buf
1002.    cf_buf[*cf_p] = newBufSample;  // Overwrite Oldest Value in cf_buf with the new one
1003.    (*cf_p)++;
1004.    if(*cf_p == *cf_lim)  // Reset Pointer Value to 0 if the end of cf_buf has been reached
1005.    {
1006.             *cf_p = 0;
1007.    }
1008.    return readback;
1009.}
```

**cf** steht für Comb.

**cf_buf:** Hält die vergangenen Werte des Filters nach dem Delay.
**cf_p:** Hält die Adressierung des ältesten Werts von cf_buf.
**cf_g:** Hält den Wert des Gain-Faktors
**cf_lim:** Hält die Anzahl der Buffer-Größe von cf_buf.

Code für die AllPass_Filter-Funktion:

```
1011. float AllPass_Filter(float inputSample,float ap_buf[], int *ap_p, float *ap_g, int *ap_lim)
1012. {
1013.     float readback = ap_buf[*ap_p]; // Read Most Delayed Value of ap_buf
1014.     readback += (-(*ap_g))*inputSample;
1015.     float newBufSample = readback * (*ap_g) + inputSample; // Calculate Value for ap_buf
1016.     ap_buf[*ap_p] = newBufSample; // Overwrite Oldest Value in cf_buf with the new one
1017.     (*ap_p)++;
1018.     if(*ap_p == *ap_lim)  // Reset Pointer Value to 0 if the end of ap_buf has been reached
1019.     {
1020.               *ap_p = 0;
1021.     }
1022.     return readback;
1023. }
```

**ap** steht für AllPass.

**ap_buf:** Hält die vergangenen Werte des Filters nach dem Delay.
**ap_p:** Hält die Adressierung des ältesten Werts von ap_buf.
**ap_g:** Hält den Wert des Gain-Faktors
**ap_lim:** Hält die Anzahl der Buffer-Größe von ap_buf.

Code der Funktion `void apply_Reverb()`:

```
1025. void apply_Reverb()
1026. {
1027.     float sum = (float) ((left_in>>1) + (right_in>>1)); // calculate mono
1028.     // calculate echo based on the block diagram using the Comb_Filter and AllPass_Filter
1029.     float newSample = (Comb_Filter(sum, cf_buf0, &cf_p0, &cf_g0, &cf_lim0)
1030.                       +Comb_Filter(sum, cf_buf1, &cf_p1, &cf_g1, &cf_lim1)
1031.                       +Comb_Filter(sum, cf_buf2, &cf_p2, &cf_g2, &cf_lim2)
1032.                       +Comb_Filter(sum, cf_buf3, &cf_p3, &cf_g3, &cf_lim3)) / 4.0f;
1033.     newSample = AllPass_Filter(newSample, ap_buf0, &ap_p0, &ap_g0, &ap_lim0);
1034.     newSample = AllPass_Filter(newSample, ap_buf1, &ap_p1, &ap_g1, &ap_lim1);
1035.     newSample = AllPass_Filter(newSample, ap_buf2, &ap_p2, &ap_g2, &ap_lim2);
1036.     // add original sample and echo-sample together proportionally to wet
1037.     sum = (1.0f - wet) * sum + wet * newSample;
1038.
1039.     left_out = (int) sum;
1040.     right_out = left_out;
1041. }
```

In dieser Funktion werden die Comb- und AllPass-Filter verwendet.

Zeile 1027: Zuerst wird die Eingangs-Audio wieder zu Mono konvertiert.

Zeile 1029 – 1035: Alle im Blockschaltbild gezeigten Filter werden nach der Reihe mit ihren Parametern aufgerufen und das Signal wird nach den Comb-Filtern geviertelt.

Zeile 1037: Hier wird der Ausgangswert berechnet, in dem das aktuelle Audio-Sample mit dem Echo-Effekt addiert wird. Die Gewichtung von Echo-Effekt und aktueller Audio hängt wie gesagt von der Variable „*wet*“ ab, welche durch den Benutzer verändert wird.

Zeile 1039 & 1040: Der berechnete Hall-Effekt wird ausgegeben.

In `calculate_Volume();` wird wieder nur die Funktion `apply_Reverb();` aufgerufen.

## 7.9.5 Wah-Wah-Effekt

### *7.9.5.1 Allgemein*

Bei dem Wah-Wah Audio-Effekt wird die eingehende Audio mit einem Wah-Sound beaufschlagt. Dabei wird der Wah-Effekt durch einen Bandpass auf dem Eingangssignal realisiert. Das Frequenzband des Bandpass oszilliert dabei auf und ab, wodurch das fallende und steigende Audio-Signal bei diesem Effekt gehört werden kann. Dieser Effekt wird typischerweise bei Gitarren-Musik benutzt.

### *7.9.5.2 Grundprinzip*

Blockschaltbild des Wah-Wah-Effekt:

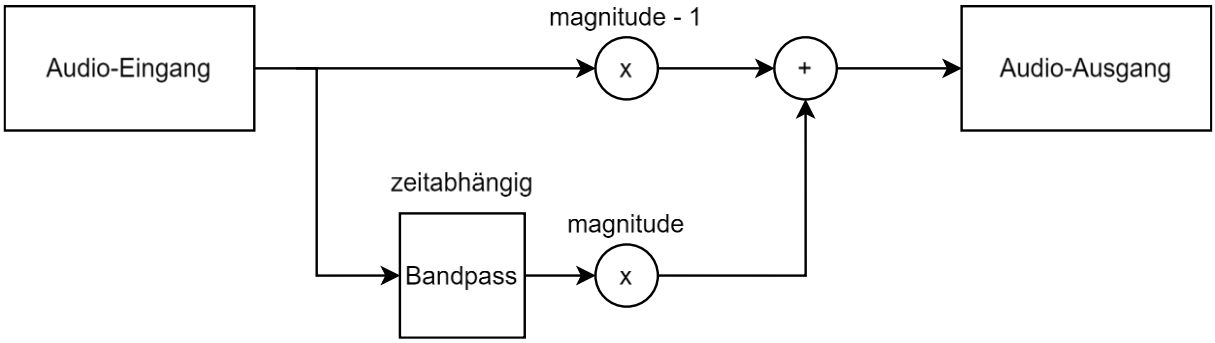



*Abbildung 131: Blockschaltbild des Wah-Wah-Effekt*

Dieses Blockschaltbild zeigt die Berechnung des Wah-Wah-Effekt. Der Effekt berechnet sich dabei aus dem originalen Audio-Signal und dem zeitabhängigen bzw. sich ändernden Bandpass-gefilterten-Audio-Signal. Die Gewichtung der beiden Signale hängt dabei von der Variable magnitude ab (0 – 1), welche vom Benutzer eingestellt werden kann.

Blockschaltbild zur Berechnung des Bandpass-Ausgangs:

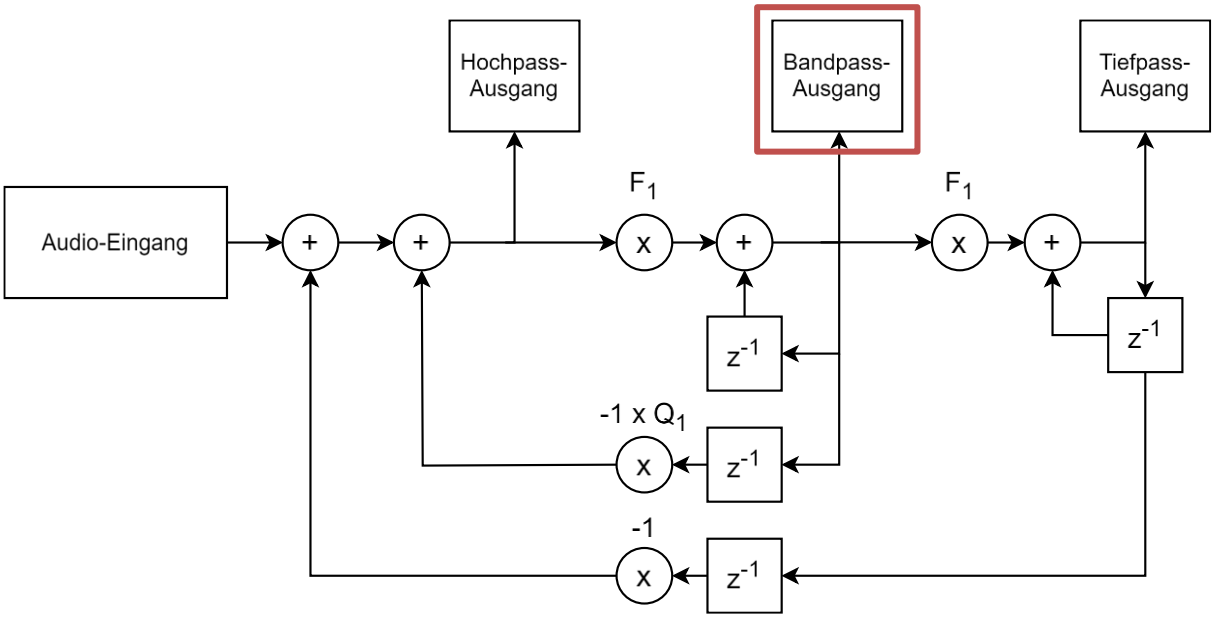



*Abbildung 132: Blockschaltbild Berechnung Wah-Wah-Bandpass*

In diesem Blockschaltbild wird die Berechnung des dynamischen Bandpass beschrieben. Der Wert bei „*Bandpass-Ausgang*“ ist also der Wert nach dem „*Bandpass*“-Block im vorherigen Blockschaltbild.

In dem Blockschaltbild gibt es 2 Koeffizienten $\mathbf{F_1}$ und $\mathbf{Q_1}$, welche mit folgenden Formeln berechnet werden:

$$F_1 = 2\sin\left(\frac{\pi f_c}{f_s}\right) \qquad\qquad Q_1 = 2d$$

Hier wird gesehen, dass es noch 2 unbekannte Variablen gibt:

**$f_c$**……**Center-Frequency**: Diese Frequenz bestimmt, wo die Mittelfrequenz des Bandpasses ist. Das bedeutet, dass um diese Frequenz herum das Wah-Wah-Signal gehört werden kann.

**d**……..**damp**: Diese Variable gibt an wie präzise das Wah-Wah-Signal gehört werden soll. Je größer **damp** ist, desto breiter ist das Frequenzband des Wah-Wah-Signal.

*7.9.5.3 Code*

```
1043. void apply_WahWah()
1044. {
1045.     float input = (float) ((left_in>>1) + (right_in>>1)); // calculate mono
1046.
1047.     sweep_rate = Fw/Fs;
1048.
1049.     // Wah frequency sweep (triangle wave)
1050.     Fc += sweep_dir * sweep_rate;
1051.     if (Fc > maxFc) {
1052.           Fc = maxFc;
1053.           sweep_dir = -1;
1054.     } else if (Fc < minFc) {
1055.           Fc = minFc;
1056.           sweep_dir = 1;
1057.     }
1058.
1059.     // Update coefficient
1060.     Q1 = 2.0f * damp;
1061.     F1 = 2.0f * sinf( 3.1415926535f * Fc / Fs);
1062.
1063.     // State Variable Filter Equations
1064.     yh = input - yl - Q1 * yb;
1065.     yb = F1 * yh + yb;
1066.     yl = F1 * yb + yl;
1067.
1068.     left_out = (int) (input * (1 - magnitude) + yb * magnitude);
1069.     right_out = left_out;
1070. }
```

Zeile 1045: Zuerst wird die Eingangs-Audio wieder zu Mono konvertiert.

Zeile 1047: In der Variable $\mathbf{f_w}$ wird gespeichert wie viele Hz die Center-Frequency $\mathbf{f_c}$ sich in der Sekunde ändern soll. Zusätzlich mit der Abtastrate kann die `sweep_rate` berechnet werden, welche angibt um wie viele Hz sich $\mathbf{f_c}$ in diesem Methodenaufruf ändern muss.

Zeile 1050: Hier wird $\mathbf{f_c}$ aktualisiert. `sweep_dir` gibt noch an, ob $\mathbf{f_c}$ gerade am Steigen oder Fallen ist.

Zeile 1051 – 1057: Hier wird `sweep_dir` umgedreht, wenn $\mathbf{f_c}$ den maximalen bzw. minimalen Wert erreicht.

Zeile 1060 & 1061: Hier werden die beiden Koeffizienten aktualisiert. Diese müssen aktualisiert werden, da diese Koeffizienten von Werten wie `damp` abhängig sind, welche von dem Benutzer geändert werden können.

Zeile 1051 – 1057: Berechnung des Bandpass-Filters wie im vorherigen Blockschaltbild beschrieben.

Zeile 1068 & 1069: Berechnung des Ausgangs-Wert. Die Variable magnitude beschreibt dabei wie stark das originale Signal und das Wah-Wah-Signal gewichtet werden sollen.

In `calculate_Volume();` wird wieder nur die Funktion `apply_WahWah();` aufgerufen.

# 8 Inbetriebnahme des Systems

Damit das System in Betrieb genommen werden kann werden folgende Dinge benötigt:

- MDDS-System mit dem Mikrocontroller **STM32F303RET6**
  (Standardmäßig: STM32F103RBT6)
- Audioplatine
- Software

Wird ein eigenes MDDS-System verwendet müssen noch folgende Schritte getätigt werden:

Zuerst muss das C-Programm der Diplomarbeit auf den Mikrocontroller geflasht werden. Danach muss die Audioplatine über die PMOD-Schnittstelle mit der Mikrocontroller-Platine verbunden werden. Als letztes muss noch das Nextion-Display mit unserer Software geflasht werden und schon ist das System funktionsfähig.

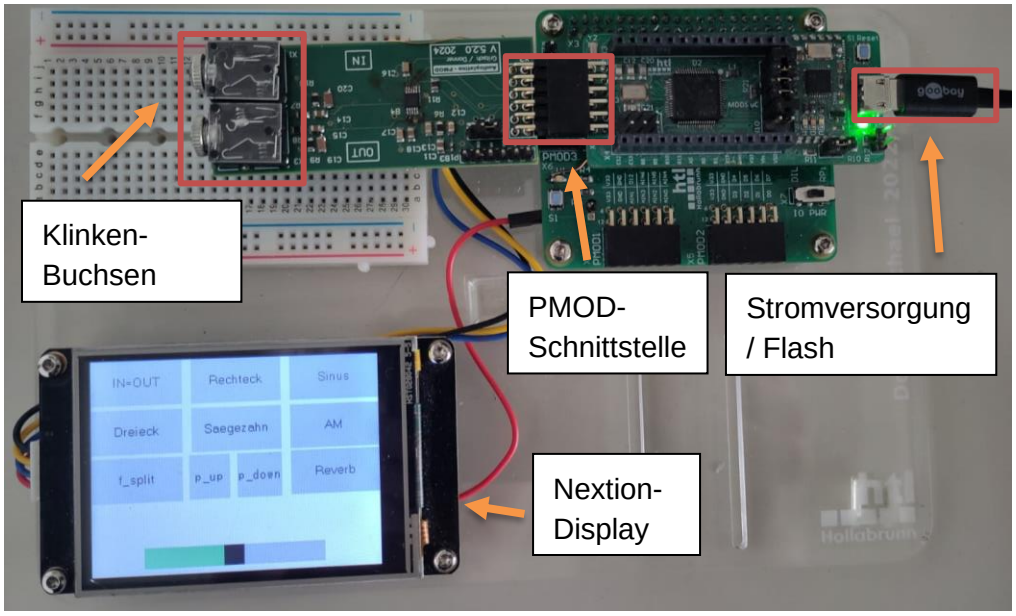



*Abbildung 133: System Inbetriebnahme*

Um Audio ein- oder auszugeben, müssen Klinken-Stecker in die zwei Klinken-Buchsen gesteckt werden. Dabei ist mit „*IN*“ oder „*OUT*“ gekennzeichnet, wo der Eingang und wo der Ausgang ist.

**Nextion-Display:**
Mit dem Nextion-Display kann der Modus der Audio-Verarbeitung und die Lautstärke eingestellt werden.

Der grüne Regler ist für die Lautstärke zuständig. Diese skaliert die Audio von 0 – 2, also ist der Regler ganz links kann nichts gehört werden und ist dieser ganz rechts wird die eingehende Lautstärke doppelt so laut ausgegeben.

Ansonsten gibt es noch die Buttons. Wenn diese gedrückt werden, ändern sie den Modus auf das Gedrückte um.

## Extra-Funktionen:

**f_split:** Wird doppelt auf f_split gedrückt ändert dieses auf welchem Ohr tiefe und auf welchem Ohr hohe Frequenzen ausgegeben werden.

**Pitch-shift:** Die Höhe des Pitch-Shifting kann durch „*p_down*“ verringert und durch „*p_up*“ erhöht werden.

**Reverb:** Wird auf „*Reverb*“ öfter gedrückt, wird der Echo-Effekt stärker. Wenn der Button oft genug gedrückt wurde, fängt der Effekt wieder von klein an.

**Wah-Wah:**
Wird auf „*W_mag*“ öfter gedrückt, wird der Wah-Wah-Effekt stärker. Wenn der Button oft genug gedrückt wurde, ist der Effekt wieder klein.

Der Button „*W_damp*“ bestimmt die Präzision des Effekts. Je öfter auf den Button gedrückt wird, desto „schwammiger“ ist der Effekt. Wenn der Button oft genug gedrückt wurde, fängt der Effekt wieder von sehr präzise an.

Der Button „*W_speed*“ bestimmt die Geschwindigkeit des Wah-Wah-Effekts. Dabei gibt es langsam, mittel und schnell. (Startend bei mittel)

Die Buttons „*W_min_f*“ und „*W_max_f*“ bestimmen die Grenzen der Center-Frequency $\mathbf{f_c}$. Je öfter auf die Buttons gedrückt wird, desto näher rückt entweder die Ober- oder Untergrenze an die jeweils andere Grenze. Wenn die Grenzen zu nah aneinander sind, werden beide Grenzen wieder an den maximalen Abstand zueinander zurückgesetzt.

**Bemerkung:** Jeder Wert ist zu Beginn mit einem vorgeschlagenen Standartwert initialisiert. Wenn diese Werte wieder hergestellt werden sollen, kann der „*Reset-Button*“ gedrückt werden.

# 9 Verwendete Software

## 9.1 Altium Designer

Altium Designer ist ein EDA-Programm, was bedeutet das es für die Entwicklung, Simulation, Verifizierung, Fertigung und Prüfung elektronischer Systeme eingesetzt wird. [SIE25]

In der Diplomarbeit wurde das Programm dazu benützt, Schaltungen aufzubauen und aus ihr eine Leiterplatte zu entwickeln.

*Abbildung 134: Logo von Altium Designer [AMA25]*

## 9.2 STM32CubeMX

STM32CubeMX ist ein von STMicroelectronics entwickeltes Initialisierungs- und Konfigurationswerkzeug für STM32-Mikrocontroller. Es ermöglicht eine grafische Auswahl und Konfiguration der Peripheriegeräte und die automatische Generierung des C-Quellcodes für die Projektentwicklung.

In der Diplomarbeit wurde das Programm für die Konfiguration des I2S-Protokoll, Clock-Versorgung und USART3-Protokoll verwendet.

*Abbildung 135: STM32CubeMX Logo [WIK25]*

## 9.3 Keil uVision5

Keil µVision ist eine integrierte Entwicklungsumgebung (IDE) zur Erstellung von Programmen für Mikrocontroller. Es ermöglicht sowohl die Entwicklung von Programmen als auch das direkte Flashen auf den Controller.

In der Diplomarbeit wurde das Programm für die Entwicklung und Flashen aller Software verwendet.

*Abbildung 136: Keil uVision5 Logo [EMB25]*

## 9.4 Logic 1.1.15

Logic 1.1.15 ist eine Software zur Analyse digitaler Signale mit Saleae Logic-Analyzern. Sie ermöglicht die Erfassung, Visualisierung und Protokoll-Dekodierung von Signalen.

In der Diplomarbeit wurde das Programm für die Überprüfung und Fehler-Analyse der Signale des I2S-Protokoll.

## 9.5 Matlab R2024a

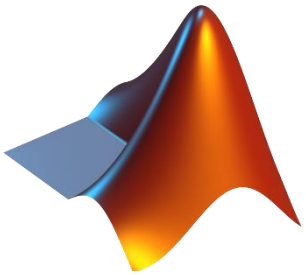


*Abbildung 137: Matlab Logo [ENM25a]*

MATLAB ist eine Entwicklungsumgebung für numerische Berechnungen und technische Simulationen. Es ermöglicht die Analyse von Daten und die Entwicklung von Algorithmen.

In der Diplomarbeit wurde das Programm für das Design und exportieren von IIR-Filter benutzt.

## 9.6 Nextion Editor




*Abbildung 138: Nextion Editor Logo [ELE25]*

Nextion Editor ist eine Entwicklungsumgebung zur Erstellung von Benutzeroberflächen für Nextion-Displays. Es ermöglicht das Design, Programmierung und Simulation auf einer grafischen Oberfläche.

In der Diplomarbeit wurde das Programm für das Design und die Simulation des Nextion-Display verwendet.

## 9.7 Draw.io



*Abbildung 139: Draw.io Logo [ENM25b]*

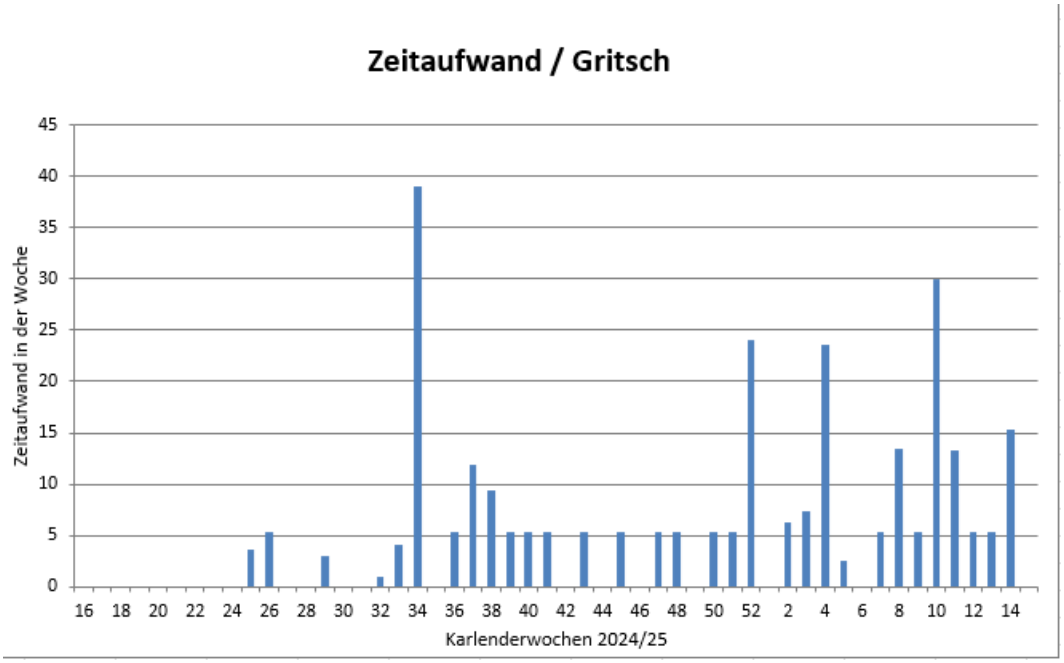
Draw.io ist eine Open-Source Website auf der digitale Diagramme erstellt werden können.

In der Diplomarbeit wurde das Programm für die Erstellung von Blockschaltbilder, Flussdiagramme und Symbole verwendet.

# 10 Zeitaufwand

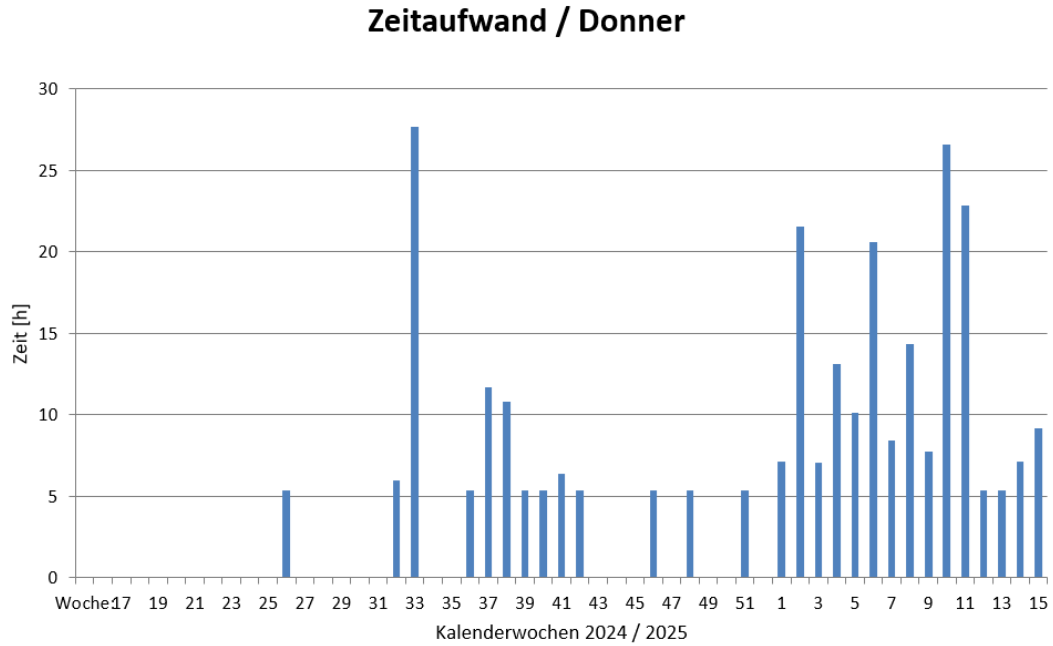
Der Zeitaufwand wurde in einem Projekt-Protokoll dokumentiert. Der Zeitaufwand wurde aus dieser Aufzeichnung entnommen.

## 10.1 Lukas Gritsch




| | |
|---|---|
| Freizeit: | 178,73h |
| Schule: | 108,80h |
| Insgesamt: | 287,53h |

## 10.2 Michael Donner




| | |
|---|---|
| Freizeit: | 195,74h |
| Schule: | 95,83h |
| Insgesamt: | 291,57h |

# 11 Kostendarstellung

| Firma | Kosten |
|---|---|
| PCB-Way | 255,29€ |
| Mouser Elektronics | 123,91€ |
| Digikey | 155,98€ |
| Reichelt | 46,30€ |
| RS-Components | 234,49€ |

In Summe hat die Diplomarbeit Kosten von 815,97€ verursacht.

# 12 Projektplan

Am Beginn der Diplomarbeit wurden Projektpläne ausgearbeitet nach den sich versucht wurde zu richten:

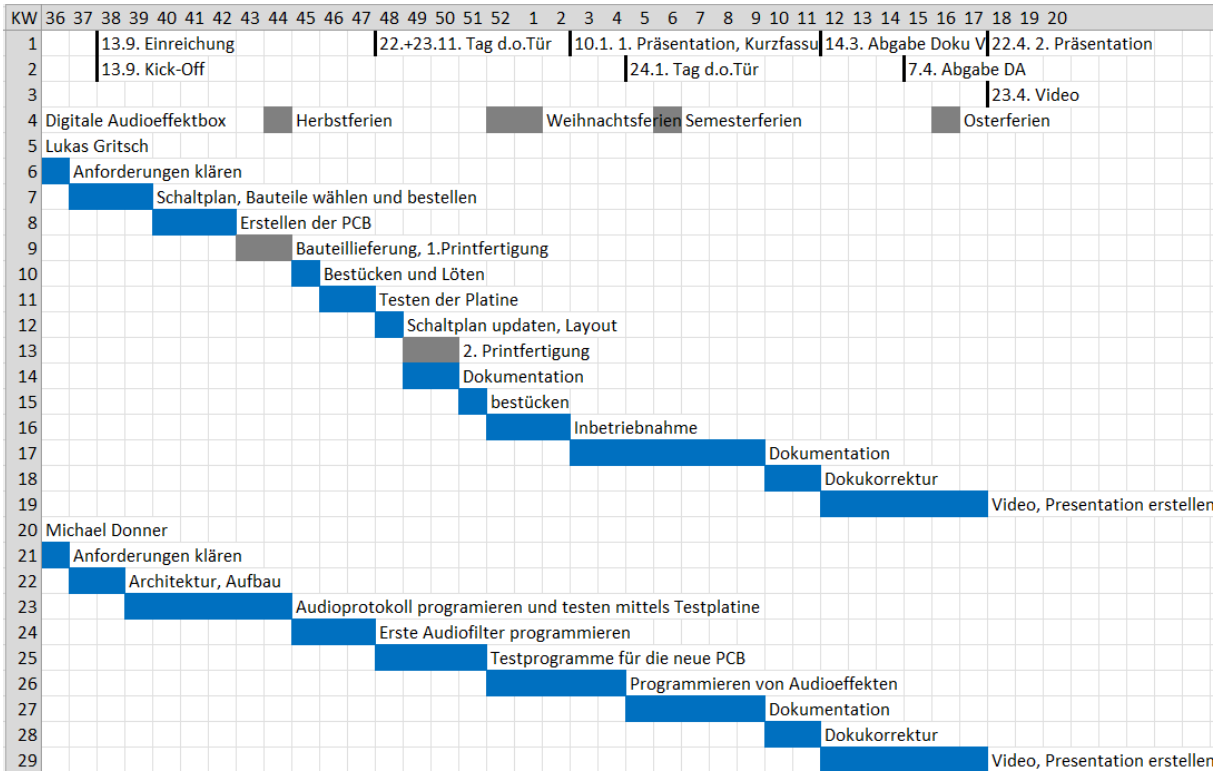



*Abbildung 140: Projektplan*

# 13 Quellen

## 13.1 Gedruckte Medien

[GLA25] Schaltung der Audioplatine V4
GLADOS – Guided Low Altitude Detection and Observation System
Von Benedikt Kneissl, Felix Peterka, Julian Kandlhofer von Killian Smith
Veröffentlicht am 03.04.2020 im Archiv der HTL Hollabrunn

## 13.2 Online

[SIE25] Erklärung einer EDA
https://www.sw.siemens.com/de-DE/technology/electronic-design-automation-eda/
(Letzter Aufruf: 08.03.2025)

[AMA25] Altium Designer Logo
https://www.amazon.in/Altium-Designer-Perpetual-License-Delivery/dp/B0CQ65YVHD
(Letzter Aufruf: 08.03.2025)

[MOU25a] Datenblatt des CS5343
https://www.mouser.at/datasheet/2/76/CS5343_44_F5-1142054.pdf
(Letzter Aufruf: 08.03.2025)

[AUD25a] Beispielschaltung für die Platine V5.2.0.
https://audiodsplab.wordpress.com/pmod-i2s2/
(Letzter Aufruf: 08.03.2025)

[MOU25b] Datenblatt des CS4344
https://www.mouser.at/datasheet/2/76/CS4344_45_48_F2-1141644.pdf
(Letzter Aufruf: 08.03.2025)

[TI25a] Übersicht über den TAC5142
https://www.ti.com/lit/ds/symlink/tac5142.pdf
(Letzter Aufruf: 08.03.2025)

[MIK25] Vergleich zwischen analoger und digitaler Masse
https://www.mikrocontroller.net/topic/28651
(Letzter Aufruf: 08.03.2025)

[TI25b] Datenblatt des TLV320AIC32
https://www.ti.com/lit/ds/symlink/tlv320aic32.pdf
(Letzter Aufruf: 08.03.2025)

[MOU25c] Datenblatt des MAX38640BELT18+T
https://www.mouser.at/datasheet/2/609/max38640_max38643-3469600.pdf
(Letzter Aufruf: 08.03.2025)

[MOU25d] Funktionsweise des Taktgenerator ASEMB-12.000MHZ-LY-T
https://www.mouser.at/datasheet/2/3/ASEMB-3318235.pdf
(Letzter Aufruf: 08.03.2025)

[TI25c] Datenblatt des TS5A22364
https://www.ti.com/lit/ds/symlink/ts5a22364.pdf
(Letzter Aufruf: 08.03.2025)

[AUD25] Audio-Stream Projekt-Beispiel
https://audiodsplab.wordpress.com/ping-pong-buffer-audio-stream/
(Letzter Aufruf: 03.03.2025)

[DIG25] Test-Audiocodec Reference Manual
https://digilent.com/reference/pmod/pmodi2s2/reference-manual
(Letzter Aufruf: 03.03.2025)

[YOU25a] IIR-Filter Tutorial
https://www.youtube.com/watch?v=lNBrGOk0XzE
(Letzter Aufruf: 11.03.2025)

[YOU25b] pitch-shift Tutorial
https://www.youtube.com/watch?v=p_a8mDcAvOg
(Letzter Aufruf: 11.03.2025)

[YOU25c] Reverb Tutorial
https://www.youtube.com/watch?v=nRLXNmLmHqM
(Letzter Aufruf: 12.03.2025)

[SCH25] IIR-Filter Kaskadenstruktur
https://schaumont.dyn.wpi.edu/ece4703b21/lecture3.html
(Letzter Aufruf: 11.03.2025)

[WIK25] Logo von STM32CubeMX
https://wiki.stmicroelectronics.cn/stm32mpu/wiki/STM32CubeMX
(Letzter Aufruf: 13.03.2025)

[EMB25] Logo von Keil uVision5
https://embeddedtools.store/product/keil-mdk-arm-essential-microcontroller-development-kit-pay-by-month-term/
(Letzter Aufruf: 13.03.2025)

[ENM25a] Logo von Matlab
https://en.m.wikipedia.org/wiki/File:Matlab_Logo.png
(Letzter Aufruf: 13.03.2025)

[ELE25] Logo von Nextion Editor
https://www.electronics-lab.com/design-hmi-projects-easily-nextion/
(Letzter Aufruf: 13.03.2025)

[ENM25b] Logo von Draw.io
https://en.m.wikipedia.org/wiki/File:Diagrams.net_Logo.svg
(Letzter Aufruf: 13.03.2025)

[STC25] Hardware-Reference-Manual STM32F303
https://www.st.com/en/microcontrollers-microprocessors/stm32f303/documentation.html
(Letzter Aufruf: 13.03.2025)

# 14 Verzeichnis der Abbildungen

Abbildung 1: Blockschaltbild des Gesamtsystems ..... 15
Abbildung 2: Ports des I2S-Protokoll ..... 18
Abbildung 3: I2S Timing-Diagramm [STC25] ..... 18
Abbildung 4: Blockschaltbild des Audio ..... 19
Abbildung 5: Schemetic der Audioplatine V4 [GLA25] ..... 21
Abbildung 6: Musterschaltung aus dem Datenblatt des TLV320AIC32 [TI25b] ..... 22
Abbildung 7: Schaltung der Trennung von digital und analog ..... 23
Abbildung 8: Schaltung des Buck Converters ..... 23
Abbildung 9: Buck-Converter aus dem Datenblatt [MOU25c] ..... 24
Abbildung 10: Pinning aus dem Datenblatt [MOU25c] ..... 24
Abbildung 11: Tabelle zur Konfiguration der Ausgangsspannung aus dem Datenblatt [MOU25c] ..... 24
Abbildung 12: Schaltung mit dem Audioswitch ..... 25
Abbildung 13: Pinning des Audioswitchs [TI25c] ..... 25
Abbildung 14: Pinningtabelle des Audioswitch [TI25c] ..... 26
Abbildung 15: Schaltung des Taktgenerators ..... 26
Abbildung 16: Pinning des TLV320AIC32 [TI25b] ..... 27
Abbildung 17: Pinningtabelle des TLV320AIC32 [TI25b] ..... 27
Abbildung 18: Schemetic der Platine V5.0.0 ..... 29
Abbildung 19: Layout der Platine V4 [GLA25] ..... 30
Abbildung 20: Layout der Platine V5.0.0 ..... 31
Abbildung 21: Platine an MDDS-System angeschlossen ..... 32
Abbildung 22: Messung der Versorgung mit TLV320AIC32 ..... 32
Abbildung 23: Aufbau mit externem Netzgerät ..... 33
Abbildung 24: Wärmebildaufnahme der Platine mit 100mA Strombegrenzung ..... 33
Abbildung 25: Wärmebildaufnahme bei 2,4A ..... 34
Abbildung 26: Ohmmeter zwischen Versorgung und Masse gemessen ..... 34
Abbildung 27: Beispielschaltung für den TAC5142 [TI25a] ..... 36
Abbildung 28: Pinning des Audiocodec TAC5142 [TI25a] ..... 37
Abbildung 29: Ausgangsbeschaltung des TAC5142 ..... 38
Abbildung 30: Pseudo-differential Output des TAC5142 [TI25a] ..... 38
Abbildung 31: Single-Ended Eingang des TAC5142 [TI25a] ..... 39
Abbildung 32: Konfigurationspins des TAC5142 ..... 39
Abbildung 33: Versorgung des TAC5142 ..... 39
Abbildung 34: Schaltung der Platine V5.1.1 ..... 40
Abbildung 35: Layout der V5.1.1 ..... 41
Abbildung 36: Konfigurationspins im Überblick[TI25a] ..... 41
Abbildung 37: Konfiguration des MD0-Pins [TI25a] ..... 42
Abbildung 38: Konfiguration von MD1 und MD2 [TI25a] ..... 42
Abbildung 39: Konfiguration des MD3-Pins [TI25a] ..... 42
Abbildung 40: Konfiguration von MD4 und MD5 [TI25a] ..... 42
Abbildung 41: I2S-Protokoll des Audiocodecs TAC5142 [TI25a] ..... 43
Abbildung 42:Konfiguration der GPIO-Pins ..... 43
Abbildung 43: Konfiguration von I2S2 in CubeMX ..... 44
Abbildung 44: Konfiguration von I2S3 in CubeMX ..... 44
Abbildung 45: Schaltung des Breakoutboards ..... 45
Abbildung 46: Layout des Breakoutboards ..... 46

Abbildung 47: MDDS System mit V5.1.1. ..... 47
Abbildung 48: Messchaltung für den TAC5142 ..... 47
Abbildung 49: Messung von Word Select ..... 48
Abbildung 50: Messung der BCLK ..... 48
Abbildung 51: Messung der Datenleitung ..... 49
Abbildung 52: Messung des Eingangssignals ..... 49
Abbildung 53: Tabelle der unterstützten Frequenzen [TI25k] ..... 50
Abbildung 54: Schaltung des Schaltmoduls von Digilent Development Kit [AUD25a] ..... 51
Abbildung 55: Master/Slave-Auswahl [MOU25a] ..... 52
Abbildung 56: Schaltungsumsetzung der Master/Slave-Auswahl ..... 52
Abbildung 57: Schaltung des V5.2.1 ..... 53
Abbildung 58: Layout der V5.2.1 ..... 53
Abbildung 59: Layout der V.5.2.0 ..... 54
Abbildung 60: Audioplatine V5.2.1 am MDDS-System angesteckt ..... 55
Abbildung 61: I2S-Protkoll mit Logikanalysator gemessen ..... 55
Abbildung 62: Sägezahnsignal von der Audioplatine erzeugt ..... 56
Abbildung 63: Rechtecksignal von der Audioplatine erzeugt ..... 56
Abbildung 64: Dreiecksignal von der Audioplatine erzeugt ..... 56
Abbildung 65: Sinussignal von der Audioplatine erzeugt ..... 57
Abbildung 66: Allgemeines Blockschaltbild der Software-Architektur ..... 58
Abbildung 67: Test Audio-Codec ..... 59
Abbildung 68: Test-Platine für Test-Audio-Codec ..... 59
Abbildung 69: PMOD Base Board Platine ..... 60
Abbildung 70: STM32407G-DISC1 ..... 61
Abbildung 71: Mikrocontroller STM32F303RET6 auf MDDS-System ..... 61
Abbildung 72: STM32CubeMX Logo [WIK25] ..... 62
Abbildung 73: Demo-Projekt STM32CubeMX ..... 63
Abbildung 74: STM32F107RCT6 Master-Master Blockschaltbild ..... 63
Abbildung 75: WS_I2S2 und WS_I2S3 Diagramm ..... 63
Abbildung 76: I2S3 Frequenz ..... 63
Abbildung 77: I2S2 Frequenz ..... 63
Abbildung 78: Digitale Audio Effektbox Slave-Version ..... 64
Abbildung 79: STM32F107RCT6 Master-Slave Blockschaltbild ..... 64
Abbildung 80: Projekt mit Cortex STM32F407G ..... 64
Abbildung 81: STM32CubeMX neues Projekt ..... 65
Abbildung 82: Mikrocontroller Suche ..... 65
Abbildung 83: Mikrocontroller Auswahl ..... 65
Abbildung 84: System Konfiguration STM32CubeMX ..... 66
Abbildung 85: Quarz-Oszillator für Clock-Versorgung STM32CubeMX ..... 66
Abbildung 86: Quarz-Oszillator Pins ..... 66
Abbildung 87: Clock Configuration STM32CubeMX ..... 67
Abbildung 88: I2S Parameter Settings ..... 68
Abbildung 89: I2S DMA Konfiguration ..... 68
Abbildung 90: I2S NVIC Konfiguration ..... 68
Abbildung 91: DMA Konfiguration ..... 69
Abbildung 92: GPIO Konfiguration ..... 70
Abbildung 93: USART3 Konfiguration ..... 71
Abbildung 94: Pinout ..... 72

Abbildung 95: Project Manager Menü ..... 73
Abbildung 96: Generate Code Button ..... 73
Abbildung 97: Ping-Pong buffer-audio-stream ..... 74
Abbildung 98: Ping-Pong buffer Kreis-Diagramm [AUD25] ..... 75
Abbildung 99:Allgemeiner Software Ablauf Flussdiagramm ..... 76
Abbildung 100: Interrupt Auslösung ..... 76
Abbildung 101: Lautstärken-Symbol ..... 80
Abbildung 102: Sägezahn-Signal ..... 81
Abbildung 103: Dreieck-Signal ..... 82
Abbildung 104: Rechteck-Signal ..... 83
Abbildung 105: Sinus-Signal ..... 84
Abbildung 106: AM-Signal ..... 85
Abbildung 107: Nextion-Display im MDDS-System ..... 86
Abbildung 108: Nextion-Editor New File ..... 86
Abbildung 109: Nextion-Display Art ..... 87
Abbildung 110: Nextion-Display Auswahl ..... 87
Abbildung 111: Display Ausrichtung & Character Encoding ..... 87
Abbildung 112: Nextion-Editor Haupt-Editor-Bereich ..... 88
Abbildung 113: Nextion-Editor Font Creator ..... 88
Abbildung 114: Nextion-Editor Font Generator ..... 88
Abbildung 115: Nextion-Editor File Output ..... 89
Abbildung 116: SD-Karte & Adapter ..... 89
Abbildung 117: SD-Karten-Slot ..... 89
Abbildung 118: Debug auf Tool-Leiste ..... 89
Abbildung 119: Nextion-Display Simulator ..... 90
Abbildung 120: UART3 Port-Besetzung ..... 90
Abbildung 121: UART3 Protokoll [STC25] ..... 91
Abbildung 122: Lautstärkeregler verschoben ..... 91
Abbildung 123: Button IN_OUT gedrückt ..... 91
Abbildung 124: 1 Byte UART-Übertragung ..... 92
Abbildung 125: IIR-Filter Kaskaden-Struktur [SCH25] ..... 93
Abbildung 126: Matlab Filter Designer ..... 94
Abbildung 127: C-Header exportieren ..... 95
Abbildung 128: Pitch-Shift Grundprinzip ..... 97
Abbildung 129: Reverb Blockschaltbild ..... 100
Abbildung 130: Comb-Filter, AllPass-Filter ..... 101
Abbildung 131: Blockschaltbild des Wah-Wah-Effekt ..... 103
Abbildung 132: Blockschaltbild Berechnung Wah-Wah-Bandpass ..... 103
Abbildung 133: System Inbetriebnahme ..... 106
Abbildung 134: Logo von Altium Designer [AMA25] ..... 108
Abbildung 135: STM32CubeMX Logo [WIK25] ..... 108
Abbildung 136: Keil uVision5 Logo [EMB25] ..... 108
Abbildung 137: Matlab Logo [ENM25a] ..... 109
Abbildung 138: Nextion Editor Logo [ELE25] ..... 109
Abbildung 139: Draw.io Logo [ENM25b] ..... 109
Abbildung 140: Projektplan ..... 112